



Objective FP7-ICT-2007-1-216041/D-5.3

The Network of the Future

Project 216041

“4WARD – Architecture and Design for the Future Internet”

D-5.3

Evaluation of Generic Path architecture and mechanisms

Date of preparation: **10-06-14**
Start date of Project: **08-01-01**
Project Coordinator: **Henrik Abramowicz**
Ericsson AB

Revision: **1.0**
Duration: **10-06-30**



Document Properties:

Document Number:	FP7-ICT-2007-1-216041-4WARD/D-5.3	
Document Title:	Evaluation of Generic Path architecture and mechanisms	
Document Responsible:	Hagen Woesner (TUB)	
Authors:	Peter Schefczik (ALUD), Heidrun Grob-Lipski (ALUD), Esther Le Rouzic(FT), Thi-Mai-Trang Nguyen (LIP6), Philippe Bertin (FT), Thorsten Biermann (UPB), Sebastian Dröge (UPB), Sergio Figueiredo (IT), Anders Gunnar (SICS), Dariusz Bursztynowski (TPSA), Manolis Sifalakis (UBasel), Lucio Studer Ferreira (IST-TUL), Petteri Pöyhönen (NSNF), Luca Muscariello (FT), Michael Soellner (ALUD), Horst Roessler (ALUD), Nicolas Brochier(FT), Xiaofei Zhang (LIP6), Guozhi Wei (FT), Martin Dräxler (UPB), Carsten Rösnick (UPB), Augusto Neto (PTIN), Mats Folke (EAB), Tomasz Janaszka (TPSA), Christian Tschudin (UBasel), Luis M. Correia (IST-TUL), Enrique Izaguirre (TID), Sara Oueslati (FT),	
Target Dissemination Level:	PU	
Status of the Document:	Final	
Version:	1.0	

Production Properties:

Reviewers:	Martin Johnsson (EAB)
------------	-----------------------

Document History:

Revision	Date	Issued by	Description
1.0	2010-06-14	Hagen Woesener	Version 1.0, released for EC final project review

Disclaimer:

This document has been produced in the context of the 4WARD Project. The research leading to these results has received funding from the European Community's Seventh Framework Programme (FP7/2007-2013) under grant agreement n° 216041. All information in this document is provided "as is" and no guarantee or warranty is given that the information is fit for any particular purpose. The user thereof uses the information at its sole risk and liability. For the avoidance of all doubts, the European Commission has no liability in respect of this document, which is merely representing the authors view.

**Abstract:**

This deliverable D5.3 "Evaluation of Generic Path architecture and mechanisms" of the workpackage WP5, "Formux"(FORwarding and MULTipleXing), presents the results of the final evaluation and prototyping activities of the Generic Path (GP) framework concept, from the perspectives of architectural framework, mobility, routing, resource management and the wireless mesh networks use case. In addition to the presentation of the main achievements and practical validation results, the concluding assessment of the GP architecture approach is discussed. The appendices give a detailed description of the developed prototypes and their coding, parts of which will be published under open source licensing conditions.

Keywords:

Generic Path, Compartment, Node Compartment, Endpoint, Entity, Mediation Point, Hook, Port, Routing, Resource Management, Mobility, Validation, Prototyping

Executive Summary

This deliverable presents the results of evaluation and prototyping activities of the Generic Path (GP) framework concept. The architecture has basically been presented in deliverable D-5.1 [1] and deliverable D-5.2 [2]; here we concentrate on practical and tangible results regarding this architecture. Other aspects such as cross-relations of GPs with In-Network-Management or performance of specific algorithms are dealt with in deliverable D-5.4 [3]. It should be noted that the final validation results concerning the cooperation and coding framework in the GP architecture had already been reported in deliverable D-5.2.

After a brief recap of the why and how, we show how work in the main research areas has converged during the project. All chapters follow a similar structure: we present a use case (problem to solve) and our approaches to solving it.

We start in chapter 3 by discussing three approaches to mobility management showing the flexibility of Generic Path constructs: mobile anchors, anchor-less and multi-homed end-to-end mobility. We present results and give a comparison of the solutions.

The next chapter 4 is routing-related: We investigate the usability of the GP for inter-domain routing, supported by a migration tool allowing inter-domain routing experiments and corresponding evaluation results. In an additional section, robust GP routing mechanisms and their relation to resource management are considered.

A third major building block in chapter 5 is dealing with GP resource management in two different variants: one for "managed" networks that is elaborated along the use case of Optical Transport Network (OTN) and explains how a network provider would describe and compose OTN services; a second one that is considering mechanisms for a light-weight GP management to be applied locally in a Mediation Point (MP).

The last use case in chapter 6 provides an example for the design process using GPs: Wireless Mesh networks. The introduction of an additional compartment (providing a generic link layer) is demonstrated here.

In addition to the presentation of the main achievements and validation results, the concluding assessment of the GP architecture approach is discussed in chapter 7.

The appendix of the document contains more detailed descriptions of the prototyping work performed within WP5:

The combined simulator/emulator developed at UPB provides an open source environment that shall foster the implementation of GPs in POSIX-based systems. Appendix A provides the documentation of this prototype.

In close relation to this environment and its OMNet++ based part, appendix B provides another evaluation result for the GP-based transport services selection, the Link Quality Ordering (LQO)-based Routing.

In addition, appendix C explains how dynamic service composition can be performed and presents insights based on prototyping and performance evaluation results. [D-5.2 had explained that though service composition is needed this can be distributed as well. The chapter here explains how a lot of the discussion that kept us busy during the project was entirely philosophic.]

Finally, appendix D extends the discussion of chapter 5 on OTN resource management by explaining the selected validation approach in an OMNet++ based environment.



Document:	FP7-ICT-2007-1-216041-4WARD/D-5.3		
Date:	June 14, 2010	Security:	Public
Status:	Final	Version:	1.0

We conclude with a discussion of achievement and lessons learned from two and a half year of joint research on Future Internet (FI) architecture. A number of problems that were identified before the start of the project could actually be solved, while other questions remain open and require more implementation and testing in real life conditions. An outlook on standardization opportunities concludes the document.

Contents

Executive Summary	iii
1 Introduction	1
1.1 General	1
1.2 Background	1
1.2.1 Slowing evolution: Global deployment (No one suffers alone, we suffer altogether)	2
1.2.2 Obstructing convergence: Rigidness of the layer model	2
1.3 Revisiting our model: Object Orientation	3
2 Overview of the Generic Path Model	5
2.1 The GP service model	5
2.1.1 The Structure of a Generic Path	5
2.1.2 Using Generic Paths to compose network services	6
2.1.3 Compartments and contextualisation of network services	7
2.1.4 GP and Compartment Programming Interfaces: GP service creation and access management	7
2.2 Related work	8
3 Mobility	11
3.1 Mobility Use Cases	11
3.2 Setup of the GP	12
3.3 Three Ways of Managing Mobility	12
3.3.1 Dynamic Mobility Anchoring	12
3.3.2 Anchorless Mobility Design	13
3.3.3 Multi-Homed End-to-End Mobility	14
3.4 Triggers and Handover Decision	15
3.5 Mobility Evaluation Framework	17
3.5.1 Simulations for DMA	17
3.5.2 Simulation and prototype for MEEM	19
3.6 Mobility Comparison and Combination	22
3.6.1 Dynamic Mobility Combinations	24
3.6.2 Mobility Comparison Conclusion	26
3.7 Mobility Conclusion	26
4 Routing evaluations	27
4.1 Inter-domain routing	27
4.1.1 Introduction	27
4.1.2 GP-BGP evaluation	28
4.1.3 AS pairing evaluation	31
4.1.4 Conclusions	36
4.2 GP routing and robust resource management	37
4.2.1 Introduction	37



4.2.2	Robust multi-path routing	37
4.2.3	QoS-aware Routing and Resource Control (QoS-RRC)	37
4.2.4	Conclusions	39
5	GP Resource Management	41
5.1	Optical Transport Network (OTN): A use case for the GP framework	41
5.1.1	Introduction	41
5.1.2	The OTN GP Model	43
5.1.3	Validation of the Approach	50
5.2	GP light-weight resource management: Evaluation results	53
5.2.1	SQF description	53
5.2.2	Analysis	54
5.2.3	Experiments	56
5.2.4	SQF implementation in the context of the GP framework	58
6	WMN: A design exercise for the GP Model	61
6.1	Wireless Mesh Networks, the wireless Internet paradigm	61
6.2	Research scope: Wireless connectivity management	62
6.3	Object-oriented engineering in the GP model	63
6.3.1	Wireless resource abstraction	63
6.3.2	Channel assignment strategy	64
6.4	Discussion	65
6.5	Proof of concept validation	66
7	Conclusions	71
7.1	Prototyping and performance evaluation activities	71
7.2	Solved problems	72
7.2.1	Session mobility and terminal mobility	72
7.2.2	Introduction of innovation in the network stack	72
7.2.3	Investigation of internal state of a path	72
7.2.4	Flexible resource description and composition	72
7.3	Open issues	73
7.3.1	Unified control plane vs. multiple compartments	73
7.3.2	Light-weight vs. heavy-weight resource description and control	73
7.3.3	Performance gains?	73
7.4	Perspective	74
	Appendix	75
A	GP Prototype Design and Implementation	75
A.1	Introduction & Fundamental Concepts	75
A.1.1	Motivation	75
A.1.2	Basic structures and their relationship	75
A.2	Technical concepts	78
A.2.1	Overall structure	78
A.2.2	Capabilities	82
A.2.3	Resolutions	82
A.2.4	Generic Paths & Endpoints	85
A.3	Developing with the ForMux/GP Testbed	89
A.3.1	Installing & Building	89



A.3.2	API along with exemplary implementations	91
A.3.3	Included Demo Applications	98
B	GP-based Transport Services Selection for LQO-based Routing	101
B.1	Introduction	101
B.2	LQODV Protocol - General Idea	101
B.3	Modelling using GP-architecture and UPB's framework	102
C	Dynamic Functional Composition at the Mediation Point	105
C.1	General	105
C.2	Functional composition framework	105
C.2.1	System Overview	106
C.2.2	Modularisation: Component based, message passing	106
C.2.3	Composition model: Federating and organising functionality	108
C.2.4	Virtualisation through recursive composition and resource management	109
C.3	Evaluation of the functional composition framework	109
C.3.1	Adaptation capacity	110
C.3.2	Performance	111
C.4	The functional composition framework in the GP Node framework	113
C.5	Related work	113
D	Optical Technologies: Simulating a Generic Network Environment	117
D.1	Introduction	117
D.2	Infrastructure	117
D.3	Equipments : transmission	118
D.4	Equipments : switching	118
D.5	The Abstract Resources	118
D.6	The Abstract Representation	119
D.7	Examples of NED code	120
	List of Abbreviations, Acronyms, and Definitions	127
	Bibliography	131



Document:	FP7-ICT-2007-1-216041-4WARD/D-5.3		
Date:	June 14, 2010	Security:	Public
Status:	Final	Version:	1.0

1 Introduction

1.1 General

An observation regarding Internet related research is that, by antithesis to the progress made in software engineering, little progress has been made in network engineering in terms of improving, adopting, formalising, and evolving our models, tools and design patterns. For many years, the current layered architecture provided a powerful software engineering foundation for abstracting and structuring functionality and separating engineering concerns. Yet today, more often than not, the rigidity of this model is coming short of embracing or describing new functionality and semantic interactions, between layers and across divergent technologies, imposing interface limitations (that lead to architectural violations) and encouraging repetition of functionality in application overlays due to limited functional plasticity and customisability. Most new developments use the OSI layer reference model [4] to describe the advocated functionality, but yet implement it in ad-hoc ways and using proprietary interfaces that either do not abide or violate the OSI model. This extends also to most current operating system stack implementations that only clearly respect two inter-layer boundaries of this model: the one “south” of the application layer (socket interface) and the one “north” of the link layer (device driver interface). This tolerated, in the past, practice for engineering basic network functions, appears to be rather restrictive for engineering complex services, dynamic systems, and implementing new communication paradigms that promote network convergence.

As software engineers we have available a wide range of models and formal tools to tackle the design complexity, but as network engineers we are bounded by the simplistic patterns imposed by a rigid old frozen-in-time network and communications model. Understandably, this narrows our perspective for what is possible, where to place new functionality, and how to implement it in accordance with the Internet architecture. It would be thus, of no surprise if what is perceived as ossification is simply the result of narrowed or conservative thinking by the lack of effective design tools and in face of all the emergent complexity.

Starting from this tenet in this article we explore the potential of adopting object orientation for modelling and engineering network functions and services. We present the *Generic Path* model, that provides object-oriented constructs for describing in-network processing and transportation services on data flows and across (heterogeneous) network(s). We explain how it is possible through this model, to preserve service modularity, enable abstraction (agnostic to underlying technology), provide access to the dynamic state of the network (and in doing so effect cross layering), re-use algorithms, and compose networks and network services in a structured way.

1.2 Background

The experience of Internet engineering in nearly half century of its existence has taught us a number of important lessons on how to design robust and scalable networks. The same experience has revealed a number of issues in its original design that we need to address in order to effect network convergence in future Internet engineering. Two important limitations are the global requirement for the TCP/IP protocol suite [5] which slows Internet’s evolutionary pace, and the rigidity of the layer model which attributes to the limited plasticity of the Internet architecture and complicates cross-technological interfacing and



convergence.

1.2.1 Slowing evolution: Global deployment (No one suffers alone, we suffer altogether)

TCP/IP was initially intended and served as a least common denominator for heterogeneous networks allowing application inter-operation across them, thus necessitating its global deployment. However, the long experience from this early approach of facilitating network convergence has allowed us a number of realisations:

- A full-fledged protocol suite is neither necessary nor suitable everywhere: Grids of very small devices such as sensors and RFIDs may not have the resources to operate a complete TCP/IP stack, or they may not need it (e.g. full mesh networks do not require routing). Underlying network technologies may already have their own transport and network functionality (3GPP[6], ATM[7], X.25[8]). Reinforcing TCP and IP over them for network-convergence is far from efficient.
- The requirement for global deployment imposes scalability and deployment constraints since any modification or extension of protocol functionality often needs to be adopted everywhere or in large parts of the global infrastructure. At the same time, as the size and use of the Internet increased dramatically and its stake-holders are (rightfully) less willing to risk, technical standardisation processes and market acceptance for new functionality in a global infrastructure take time and come often with competing incentives.
- The protocol-centric (as opposed to abstraction-oriented) evolution of the Internet, has created technological dependencies, which are difficult (if possible at all) to get rid of, such as the address/locator overloading, TCP's check-summing, and so forth.

As a result the current set of protocols and mechanisms were easier adopted than they can be displaced. The accumulated functional constraints, make Internet evolution resemble more an aggressive search algorithm, that rewards the accumulation of dependencies, and less opportunistic to mutations and experimentation with new variants. In this metaphor, Internet's ossification is not about evolutionary robustness, but rather entrapment in a local optimum, from which there is no facility of escape. By contrast, evolutionary adaptation in nature starts locally and propagates to larger scales. Localisation and federation of functionality (where common denominator functionality is present only at the federated identity) leverages evolution in face of heterogeneity. Modularity allows easy integration of new functionality, and fast deprecation/replacement of existing, and permits invariance of mechanisms.

1.2.2 Obstructing convergence: Rigidity of the layer model

Network convergence requires more than mere bridging of different link-level transports, with a common overlay. One cannot factor out the additional diversity and heterogeneity of applications and resources. Today more clearly than ever before the current layer model falls short in describing the dynamism and plasticity necessary for integration of new and diverse technologies [9, 10]; its strict opaqueness has performance impacts [11] and the strong coupling between transport and network layers create dependencies (address/identifier coupling, checksumming, MSS calculation at the network layer, etc) that limit the adoption of new functionality.

Founded on Dijkstra's "THE" multiprogramming model [12], layered abstraction in the Internet model empowered technological advancements, and enabled incremental abstraction and decomposition of the interaction between the user and the hardware infrastructure. As illustrated schematically in figure 1.1a, there were four key elements taking part in this interaction: (a) the user (b) the *user application*, which is the user's interface to the network and generates *information objects* (units of information exchanged over the

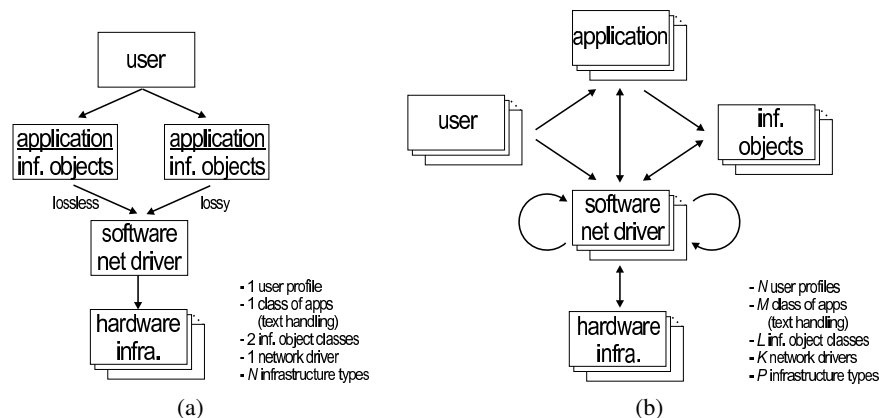


Figure 1.1: Original versus today's interaction models

network), (c) the software network driver, which implements network semantics and provides communication primitives, and (d) the different hardware infrastructures over which communication is provided. The two, communication determinant, role players in this interaction were the user application which generates information objects, and the multiplicity of hardware infrastructures over which information objects must be exchanged. As most information objects envisioned at the time were of a static nature and text based (remote terminal, e-mail, newsgroups, static file transfer, etc), their requirements were explicitly and statically enumerated in the interaction between the application and the network driver. The network driver is able to abstract and provide lossy or lossless data-delivery of information objects over different (point-to-point and point-to-multipoint) hardware infrastructures. To this end, layers allowed separation of engineering concerns in three abstractions: hop-by-hop communication, multi-hop data transportation, and end-to-end application requirements satisfaction (lossy versus lossless delivery).

The challenge faced today however, is more realistically depicted in fig 1.1b. The key interacting elements remain the same, but the number of possible interactions is larger and the scope more dynamic. The emergence of multimedia traffic introduced a dynamic set of (often interacting) information objects with diverse delivery requirements. The application element is not always under user control as it often interacts with the network in a more varied way than simply expressing reliability requirements (e.g. handling of mobility). Moreover it is decoupled from the generated information objects, which in turn may interact with the network (e.g. media objects being cached, transcoded or invoking in-network manipulation functions). The user on the other hand, directly interacts with the network driver (e.g. establishing vpn connections, expressing delivery requirements, performing admission control, etc). The network driver needs to be dynamic, not only in terms of routing path determination, but also in the selection of service functions and expression of interfaces towards a physical infrastructure, which is more diverse in nature and operational constraints (wired, wireless, multi-hop, ad-hoc, infrastructure, and other). Finally, new infrastructure technologies often depend on interactions with the network driver before satisfying user and information delivery requirements. In face of such complex, dynamic, and often symmetric or cyclic interactions, separation and organisation of engineering concerns require a model that supports a larger and more volatile set of often orthogonal abstractions, than the current fixed three (hop-by-hop, multi-hop, end-to-end).

1.3 Revisiting our model: Object Orientation

In software engineering, object-oriented design (OOD) was introduced to formalise sound engineering practices into architectural constructs and design patterns for the engineering of large, scalable and extensible



systems, and with emphasis on expressibility of dynamic functionality, effective abstraction and modularity.

Object orientation provides generic constructs for separating functionality from mechanism. When one thinks of network functionality today, she often use IPv4 (not even IPv6) as a model and tailors or orients any other (layer) function and its design to the specifics of this protocol instead of its abstract functionality and service. By contrast, in OOD, there is a clear separation between functionality and mechanism: a (abstract) class provides the representation and specification of a type of function or service, its interface represents generic semantics for accessing this function, and its objects are embodiment of mechanisms providing this function. This distinction allows a clear separation of incentives and roles for a *user* and a *provider* of a network service. A user for example can concentrate on the service provided as opposed to the intrinsics of the service fabric.

Encapsulation of mechanism in service classes, on the other hand, promotes invariance of mechanisms (e.g. for robustness). Selective information hiding allows controlled and generic access to mechanisms and helps the tractability of cross dependencies between different functions. This is one of the major concerns currently against the adoption of ad-hoc cross-layer optimisations. In the current model, by analogy, layered abstraction also promotes information hiding and controlled access through specified interfaces, as layers are stacked. However, layer stacking only offers a single-lined hierarchy of abstraction, which significantly limits the number and scope of “legal” interactions across service functions.

OOD in classes enable other features as well, which are not present in a layered model. One can create new services from existing ones, encouraging algorithmic or functional reuse through dynamic abstraction. Incrementally complex functionality may extend on existing one across one or multiple *inheritance* line(s), or integrate functions into services, through *composition*. Inheritance and composition suggest a canonical methodology of evolving functionality in an architecturally tractable and consistent way.

Polymorphism, the third design principle in OOD, provides a consistent methodology for representing and extending operations and interactions among classes (service functions). Operator *overloading* can be used to extend the definition (semantics) and mechanics of an operation or interaction. Function *templates* instantiate generic operations on multiple types of service classes. And *virtual interfaces* may be used to provide access to encapsulated service functions through their encapsulating ones, thus enabling a formally correct way of acquiring “cross-layer” type of information in an entire composite system.

In OOD, apart from a toolbox of architectural constructs for designing dynamic, modular, and scalable systems, we can find a multitude of design methodologies (a.k.a. design patterns [13]), which encouraged structured abstract thinking about re-usability of mechanisms, organisation (federation, distribution) and semantic placement of functionality. They may help us understand how to make more effective use of service functions which are present in various forms and contexts across the Internet fabric (e.g. name resolution and routing); how to explore optimisation decomposition of in-system functionality[10]; and how to formalise and re-use common practices from our Internet engineering experience so far.

2 Overview of the Generic Path Model

2.1 The GP service model

The *Generic Path* (GP) model aims to open the path for faster technological convergence in the current Internet and promote functional evolution by introducing object-oriented design concepts in network engineering. It opts to satisfy the following objectives

- *Expressibility* of functionality, for describing and designing data communication and network services independently of technology.
- *Abstraction* of different communication and service paradigms.
- *Usability*, through a set of primitives that can support generic, selective and polymorphic access to the mechanisms and technology that implement services and service functionality.

From a high-level view a *Generic Path* refers to a generalised data service path in a network, that data flow across, and in doing so they are subjected to processing ($Tp(x)$) and transportation operations ($Tf(x)$). We can symbolically express the service a GP provides on data x as

$$GP(x) = \{Tp(x), Tf(x)\}$$

2.1.1 The Structure of a Generic Path

Our aim is to provide primitives for generic and implicit access to data manipulation, transportation operations and state, such as the underlying service mechanisms may permit. As a first step to meet this goal, service functionality is provided in our model by GP classes, which derive from an abstract base class that defines a generic interface. Access to any functionality is therefore possible through this interface and any extended interfaces that derived classes may provide. Figure 2.1 shows the structure of a GP class object (instantiated service) containing a list of elements called *GPItems* and which may be one of the following.

- *Endpoint*: Identifiable data entry/exit points for a GP service. Data appearing at an exit point may or may not be in the same form as data entering the GP, subject to the service processing encountered or the experienced transportation conditions.
- *Mediation point*: Access points along a service path where configuration operations related to the service provided may be possible.
- *Attribute*. General parameters or metric information and state relevant to the service provided by the GP class.
- *Composing (Sub-GP)*. Service composition (e.g. ala layering) is possible by combining several GPs using a composition function. When the service provided by a GP class is composed by services provided by other GP objects, these are listed as sub-GPs.

All GP elements alike, as seen in the diagram in figure 2.1, may export a set of *Knobs* and *Dials*, which are configuration and introspection capabilities for information and parameters related to each GP class objects. For instance, through endpoint knobs one may set address information for the endpoints, or through the attribute dials one can read available QoS information about the provided service. Note that when one

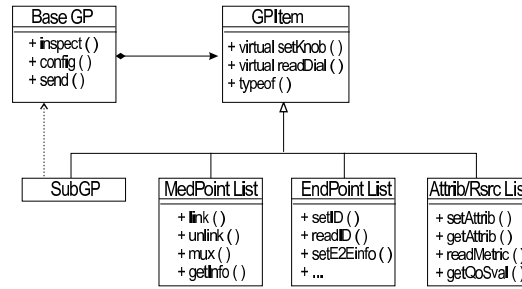


Figure 2.1: Base GP class layout

can access through a GP interface its composing sub-GPs, it is possible to (a) acquire attribute information about them recursively and configure their service parameters respectively, and (b) to access directly and modify the service composite that instantiates the GP service.

2.1.2 Using Generic Paths to compose network services

Data transformation and transportation operations in a GP represent mechanisms and algorithms implementing network service functions (for communication protocols and data processing). The developer (provider) of these mechanisms may extend the existing GP infrastructure by providing new GP classes and re-using the existing ones. For example, and in accordance with OOD, deriving along a single or multiple *inheritance* lines of GP classes it is possible to enrich existing service functionality and extend or federate at the interface level access to the composing mechanisms. On the other hand, following an iterative approach, *service composition* of any sort (horizontal such as peering, or vertical such as layering) can be described through the combination of GP classes to form new GP service classes using the object-oriented notion of *generic operators*. Symbolically, a composition function f_c over a set of GP classes may be an expression prescribing interface operations on $Tp(x)$ and $Tf(x)$, to form more complex services:

$$GP_{n+1} = f_c(GP_n, GP_{n-1}, ..)$$

Such a generic function can prescribe: (a) data flow operations on one GP type such as concatenation, aggregation, splitting, multiplexing; (b) transformation operations for similar types of GPs (in the same class hierarchy) such as proxying, protocol translation, and other middlebox operations; or (c) cross-protocol operations on heterogeneous types of GPs such as encapsulation, fragmentation and other. In this sense, any active element in a network (router, gateway, etc) can be represented as an automaton capable of providing GP operations.

The provider of a new GP service needs to decide how to divide the service functionality across components and subsequently identify what other GP services each component needs to use. This will determine the relationships among the components (interfaces) and their mode of integration with the underlying GP infrastructure (embedding, inheritance, etc). By deciding the flow-level operations and transformations to be supported at the intended service level (e.g. aggregation, coding, and so forth), one identifies functions that need to be implemented as class operators for the manipulation on objects of this service class. New types of such operators may be added at any time as new paradigms emerge, without impacting modularity. Finally, through the base GP interface, the developer can selectively specify what state and configuration parameters may be exposed in attributes, and through service elements (GPItems), for inspection and tuning.

In such a methodical approach that distinguishes function from mechanism, service designers are not confronted with issues that lead to technology-oriented artifacts and dependencies. Interfaces are easy to extend without compromising modularity or backwards compatibility. Modification or extension of functionality is incremental and localised without major infrastructure changes.

2.1.3 Compartments and contextualisation of network services

The service of a GP (or a semantic class thereof) is collectively provided by a distributed set of cooperating software and hardware modules that form a *Compartment* [14, 15]. From a top-down perspective, the compartment is the *context* in which the service is delivered. It captures implementation details such as data unit format(s), semantic for transportation and processing, as well as the scope of communications. For instance, in the scope of network interconnection, a compartment uses protocols for datagram transportation (e.g. IPv4[16], IPv6[17] and IPX[18]) and optional processing on those packets (e.g. IPSec [19] or inline measurements [20]). Likewise, the context of GPs providing a data stream abstraction, would be defined in terms of *stream segments* and involve communication protocols like RTP [21] or TCP[22] and services such as TLS [23].

Compartments also reflect the presence of realms [24] – policy domains – for decisions. How the specific service is implement in a compartment does not impact service users, nor does it impact decisions on policy, access control and quality of service, in other compartments.

In our object-oriented networking model, compartments essentially act as *extended namespaces* for service points and elements that need to be locally – within the scope of that compartment – identifiable. The analogy of compartments and GP classes (in our model) to namespaces and classes in object-oriented design serve the purposes of addressing/accessing and providing functionality respectively and promote *modularity at the policy level as well as at the functional level*.

Compartments contextualise communication and service infrastructures of different scales or functional complexity; from a single link, a domestic network, a campus, an enterprise, to large federated structures such as PlanetLab [25] or GÉANT [26]. To that respect, and given that a large compartment will likely be built using GPs offered by smaller or simpler ones, it naturally draws an analogy to layers of the current Internet and OSI models. A layer, however, assumes that the same set of functionalities is available at all nodes, while compartments capture the boundaries of administrative realms and reflects the fact that, in different realms, availability and behaviour of features such as QoS and encryption may vary greatly. Interoperability of networks is maintained through a core set of abstract GP classes (and thus services) that those networks collectively decide to use as a base class of their derived services.

In summary, the ability to decouple functionality (GP services) from the context it is employed in (compartments)¹ is a premise for *functional plasticity* (as may be required for optimisation decomposition for re-layering [10]) and *virtualisation* (resource federation and distribution) so that the same base functionality may be instantiated in multiple contexts and in a polymorphic way.

2.1.4 GP and Compartment Programming Interfaces: GP service creation and access management

GP services are provided within compartments and between named service points (i.e. GP endpoints) in the namespace of the respective compartments. A compartment provides a generic interface that one may use to establish a *GP service topology* for a certain type of service relevant to that compartment. This interface is based on two primitives (a) one for publishing (registering) service points and (b) another for establishing GP services at or between service points.

To exemplify the creation and management of GP services, we introduce the scenario (figure 2.2), where two applications establish a call over a Voice compartment *cVoice*. *cVoice* integrates and abstracts the services of two candidate technologies (called Skype and GSM); provided in separate respective compartments. Initially, the voice communication (GP in the voice compartment) uses an underlying GP in the *Skype* compartment. When the observed quality drops below a threshold level, it transparently switches

¹A mechanism however, may still be associated with a context that is it implemented for.

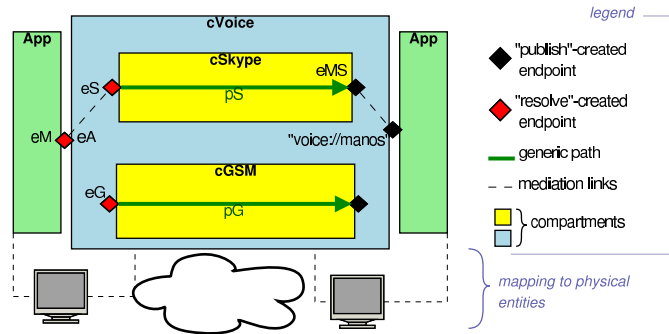


Figure 2.2: an abstract “voice” compartment, federating “skype” and “GSM” infrastructure compartments

from the Skype GP and to a new GP in the GSM compartment.

The code fragments in figure 2.3 exemplify the use of this interface for the creation and management of GP services (objects). In figure 2.3a a `publish()` call (l. 2) instructs the compartment to create an end-point (registration) and associate it with the name “sylvain”, which would be the listening point for accepting an incoming connection. The endpoint “voice://manos” (figure 2.2) should have been created in a similar way. The mechanisms for checking the validity of names (identifiers), and subsequently maintaining, distributing and resolving name information are intrinsic to every compartment and therefore not prescribed. The second compartment primitive, `resolve()` (l. 4) is then used to create a GP *gM* that will be connected to the endpoint named “manos” in *cVoice* through the generic path *gM*. *gM* can then be used (l. 5-6) as a regular socket to exchange information objects (in this case, voice samples).

The code implementing *cVoice* in Sylvain’s end-system (figure 2.3b) has to establish “carrier” GPs over one of the infrastructure compartment *cSkype* or *cGSM*. In l. 4, it receives connection requests by associating application endpoints (such as *eA*) to local ones in the *cVoice* scope. Using intra-compartment facility, *cVoice* then looks for an association of name “manos” in *cSkype* (l. 5), and by means of `resolve()` (l. 6) establishes the GP *pS* to the corresponding endpoint in the *cSkype* compartment. To complete the creation of the end-to-end GP in *cVoice*, so that every every sample exchanged over *pS* is delivered to the application, the binding of the *eM* to the endpoint *eS* of the underlying GP in *cSkype* is needed. This is established through the `link` call to the system-local mediation point abstraction (l.7).

Finally, to illustrate the re-compositional capability enabled through the *Mediation point* in this model, in figure 2.3c a call quality monitor process uses a *Dial* to access (in a technology agnostic way) measurement data collected at *cSkype* (l. 2), and when quality drops below a threshold level (stored as a *cVoice* GP attribute), the mediation point is used to transparently replace the *cSkype* GP with a new (similarly established) one in the *cGSM* compartment (l. 5-7).

2.2 Related work

Several efforts have appeared in the literature to address the shortcomings of the Internet architecture. While the majority of this work provide solutions within the current model by proposing enhancements, ad-hoc layer interfaces and extensions, which are often perceived as architectural violations (cross-layer literature and active networking), fewer have produced and proposed alternative architectures and communication paradigms for enabling network convergence.

In the role-based architecture [27], the authors consider a component-based model founded on service roles as an alternative to the current layer model. Although an important step is being made towards more modular, flexible and extensible model, the authors do not go as far as proposing the adoption of object



```

1: Compartment cV = System.getCompartment("voice");
2: Endpoint eI = cV.publish("sylvain");
3: startIncomingCallsAcceptor(eI);
4: GenericPath gM = cV.resolve("manos");
   while (gM.connected())
5:     Sound.play(gM.getInputStream().read(25ms));
6:     gM.getOutputStream().write(Sound.capture(25ms));

```

(a) Simple client voice application

```

1: Compartment cS = System.getCompartment("SkypeOverlay");
2: Compartment cG = System.getCompartment("GSMNetwork");
3: MediationPoint m = System.localMediationPoint();
   while (1)
4:     Endpoint eA = resolve_requests.dequeue();
5:     String skid = name_resolution(eA.getAttr(NAME),SKYPE);
6:     GenericPath pS = cS.resolve(skid);
7:     m.link(eA, pS.endpoint(LOCAL));
8:     monitor(eA, pS);

```

(b) Voice compartment, call establishment

```

periodicMonitoring()
1: [Endpoint eA, GenericPath pS] = monitored.dequeue();
2: int latency = pS.getDial(LATENCY);
3: int goal = eA.getAttr(MAX_LATENCY);
4: if (!acceptable(latency, goal))
5:     String telno = name_resolution(eA.getAttr(NAME),GSM);
6:     GenericPath pG = cG.resolve(telno);
7:     m.unlink(eA, pS); m.link(eA, pG);

```

(c) Voice compartment: Quality monitor and call management

Figure 2.3: GP and compartment API example

orientation in their work. A step towards the opposite direction is taken in the NIPCA [28] architecture whereby a flat process-based modelling of network communication is advocated, although we believe that adopting inter-process communication as a network paradigm is analogous to moving back to functional programming in software engineering. In i3 [29], the authors adopt indirection as a fundamental construct for flexibly building network service functionality, dynamically inter-stitching networks, and describing network communication through a publish-subscribe model. The power of indirection in leveraging transparent customisation of functionality has been also exploited in the network pointers model [30], and has been architecturally adopted in the ANA architecture [14] – by analogy, in the GP model indirection is facilitated through polymorphism. Another aspect of object orientation, namely abstract services (as in GP classes) and recursion of protocol functionality is being architecturally explored in the Recursive Network Architecture [31]. An influential proposal which (similarly to the GP model) also promotes indirection and localisation of service functionality is presented in [32], whereby global inter-network connectivity can be established across service contexts (by analogy to compartments) and through their inter-stitching functions (by analogy to the generic path bindings at mediation points). Finally, a number of interesting researches on autonomic networks, such as [14, 33, 34] are promoting communication architectures and models that federate localised and situated network infrastructures and thus promote network convergence.

In the GP model the notion of contextualisation of functionality in compartments in the GP model, has been extending concepts from the ANA architecture [14], and likewise the metaphor of knobs and dials is inspired by [35].



Document:	FP7-ICT-2007-1-216041-4WARD/D-5.3		
Date:	June 14, 2010	Security:	Public
Status:	Final	Version:	1.0

3 Mobility

3.1 Mobility Use Cases

Mobile terminals have seen a tremendous evolution during the past 10 years. High resolution colour displays combined with multi-mega pixel cameras and gigabyte storage are now common. Together with this fascinating evolution the number of network interfaces also increased. A common mobile phone today can easily have three network interfaces; GSM/UMTS¹, WiFi, and Bluetooth. With the advent of LTE and other network technologies such as cdma2000, TD-SCDMA, and WiMAX, the number of network interfaces increase even more. Mobility requires intrinsic knowledge of the network technology used, which is why all solutions for mobility differ, even though similarities can be found.

What "mobility" is all about can be described as maintaining the service of a user (or a group of users) as: the user (or the group) moves in a geographical area; the user transfers his service between different devices; the connectivity environment changes in a way that services need to be moved to different interfaces of the same device.

Many different use cases or scenarios can be imagined when evaluating mobility solutions. We have come up with three scenarios to address mobility in the GP framework, which we believe are most representative of what current and future applications require in terms of mobility.

Terminal mobility In this scenario a user moves together with the terminal from one base station to another. As the signal from the connected base station deteriorates, the signal from the approaching base station becomes stronger. At one point in time, it is necessary to change the point of attachment from the old base station to the new one, thus performing a handover. This handover does not necessarily need to be performed by the same interface, inter-interface handovers (e.g. switching from UMTS to WiFi) are also addressed in this scenario. The challenge is to do it without affecting the service of the user.

Session mobility In this scenario a user moves from one terminal to another. The network has to move the communication session and adapt it to the new terminal. One example of this is a user following a sports competition on the small screen of a mobile terminal, arrives at home and switches to the big screen of the TV. The challenge is to discover this new possibility to perform a session handover and also to do it with as little user interaction as possible.

Network mobility In this scenario two or more networks move in relation to each other. One example of this can be a bus with a mobile router on-board. The laptops on board the bus are connected to the mobile router, while it has to maintain its connectivity to networks outside the bus. The challenge for the mobile router is to perform constant handovers without breaking all sessions of its locally connected users.

¹Even though it is two separate radio access techniques, GSM and UMTS interfaces can be regarded as one interface because the dependency and similarities of the two techniques.



3.2 Setup of the GP

Setting up all the GPs needed to get a service running in a mobile scenario is done in more or less the same way, independent of the mobility scenario. There can be different GPs reflecting the different interfaces, but since all interfaces provide similar services, the differences are abstracted away for the GPs "closer" to the service. How to setup a GP is described in [2, Chapter 2.3]. What is needed in particular in the mobile scenarios is the functionality to quickly and seamlessly change from one GP to another, this means that an Entity must be able to quickly find a GP providing the same or better service than the currently used GP. For *terminal mobility* it is easy to see that an Entity providing bit transfer using a WiFi GP changes to a UMTS GP if the WiFi coverage deteriorates below a certain threshold for example. As it turns out, switching GPs providing the services to an Entity is a key functionality in all scenarios.

To be able to switch GPs, or select one between two or more, the Entity needs to know the different services provided by the GP and how they relate to the service that the Entity itself is providing to other GPs. The resource ontology explained in Chapter 5 can be used for this. During start-up, a resource object is created. Because this object is a MP it can be queried by the Entity, which can then use this information to select a suitable GP to implement the service.

The selection process described earlier could also be done on a packet-by-packet basis, effectively creating a setup using multi-path routing over the different interfaces. To this end, the Entity needs to have knowledge about the possible routes provided by the GPs in use. It is also possible to do the routing decisions based on the properties of the service. Even though this is not an issue directly related to mobility, it shows that the fundamental building blocks of the GP architecture are reused in many different scenarios and applications, which in turn shows the inherent flexibility of the architecture.

3.3 Three Ways of Managing Mobility

In this section we present three distinct proposals that provide mobility concepts within the GP architecture framework [2]. These solutions take advantage of the object oriented properties of End Points (EPs) for the dynamic update of GPs, thereby going beyond current concepts based on centralized mobility servers like in Mobile IP (MIP) based solutions.

These mobility mechanisms may coexist or may be combined dynamically according to the mobility context.

3.3.1 Dynamic Mobility Anchoring

The Dynamic Mobility Anchoring (DMA) approach provides a distributed, flat architecture able to support mobility in the heterogeneous network, see [2]. This is driven by the expectation that distributed control and data plane architectures achieve better performance and scalability than centralized architectures where central entities introduce much more delays, bottlenecks and constraints [36]. In DMA, mobility support is inherently sustained by Access Nodes (ANs) that act as traffic flow anchors, whereas the rest of the network remains unaware of mobility. When a multi-interface terminal moves in a heterogeneous network, it establishes a single or multiple network connectivity associations with one or more ANs. When set up, each flow is implicitly anchored on the initial serving AN, which then provides the necessary indirections through ANs to which the Terminal is moving. ANs actively cooperate to guarantee the delivery of the traffic flow within the access network by constantly considering the most appropriate traffic flows' mapping over available connectivity associations, adapting traffic flow indirections. Hence, the terminal can take benefit of any connectivity technology available depending on radio resources and Quality of Service fluctuations.

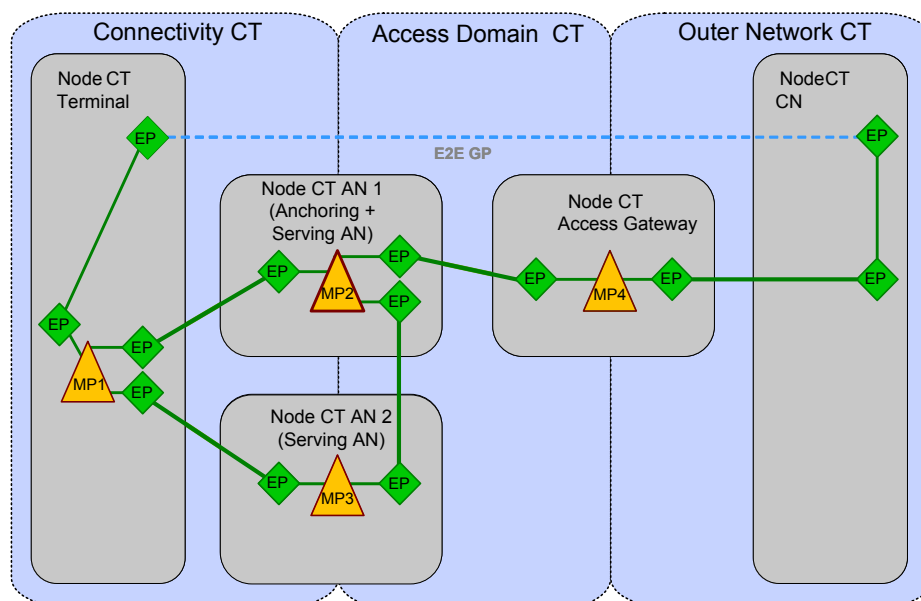


Figure 3.1: The Dynamic Mobility Anchoring (DMA) scheme

Figure 3.1 illustrates an example where a multi-interface Terminal uses a multi-path routing GP to transfer the traffic flow over its two active network attachments, its serving ANs (see AN1 and AN2 in the figure). AN1 acts also as the anchor for the considered end-to-end GP. It aggregates the traffic flow from/to the current serving ANs and the Access Network, allowing flow delivery over either AN1 or AN2 access interfaces. A MP is utilised to split or aggregate the traffic flow within the terminal or the anchoring AN. When the terminal moves out from AN1, its current GP's anchor remains located in AN1 until the GP is terminated. However, any new GP is then anchored on a serving AN, which takes benefit of DMA being dynamic to select the most appropriate anchor at each GP setup. Compared to existing mobility schemes, new dynamic and distributed properties introduced with DMA allow flexible and optimised mobility support. Traffic indirections is activated only when needed with local support in ANs, whereas more traditional scheme rely on systematic indirection between centralised (and eventually hierarchical) "static" mobility anchors (e.g. the Home Agent (HA) in MIP) in core networks and the terminal or its AN.

3.3.2 Anchorless Mobility Design

We now shortly depict the Anchorless Mobility (AM) concept and embed it into the GP architecture shortly delineated in section 2.1.1. For more details on the AM concept, please refer to [2].

The AM concept is based on the use of dynamic Bindings between EPs, see figure 3.2. Having no specific mobility anchor (HA in MIP, [37], [38]), this concept solves the main mobility pain-point today, namely of scalability. Furthermore, the AM concept allows a local break-out, which is not easy to accomplish by using a tunnelling solution like MIP. Despite there are possibilities depending on the mobility protocol used, the problem is that trombone routing can occur readily for signaling and data paths if no precautions are taken.

The proposed concept fits for all types of mobility: session, terminal and network mobility. Conceptually, in the GP architecture there is no relevant difference between inter and intra-technology handover, as modifications and reconfigurations of the GP are performed via a common API. Moreover, the AM concept uses different addresses for locations and for host identifiers in the network. By making use of a proper addressing scheme [39] we also overcome the locator/identifier split [40] problem of today's Internet.

According to the object oriented nature of the GP architecture [41], different classes of EPs are derived

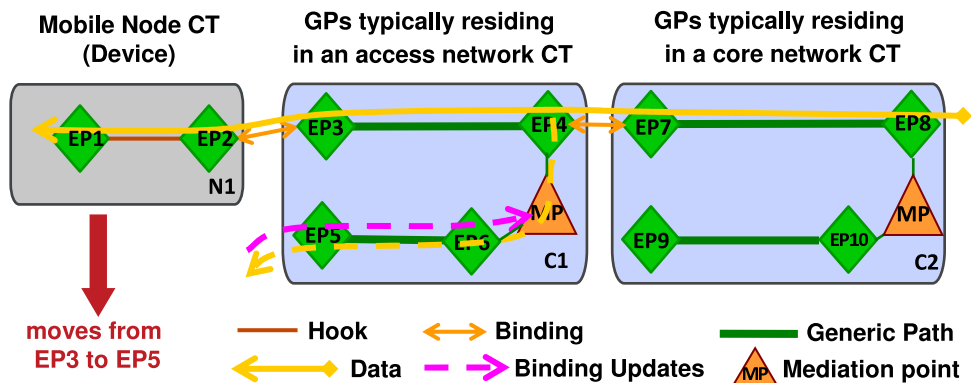


Figure 3.2: The Anchorless Mobility (AM) scheme

from the EP base class. A mobile device for example contains two different kinds of EPs, see EP1 and EP2 in figure 3.2. A logical EP at the Information Object (application) is wired to an EP denoting the device connectivity. This latter EP is connected via a Binding to an EP in the access network, denoting the current location of the device. Such an EP is visible from outside its surrounding CT and also internally connected to other EPs at the edge of this CT via GPs. Thereby end-to-end connectivity within the CT and thus throughout the network is ensured.

The mobile device is represented by a node CT. The access network represented by another CT constitutes the name space and as said before the internal connectivity of Compartments (CTs) is realized by GPs. The main component that facilitates mobility is the dynamic Binding function performed between the EPs. If for example the mobile device is moving, a Binding to another EP is established, before destroying the Binding to the former EP (softhandover). Note that the Binding always connects EPs of different CTs, while the connectivity inside a node CT is realized by Hooks, cf. figure 3.2. Upstream packets from the mobile device easily travel on the new path through the CT immediately after a re-binding to the new EP. Downstream packets from the core network towards the mobile device must be redirected at a suitable point in the network, e.g. controlled by a MP. Usually this point where the former and the new path reconvene lies on the former path and as close to the mobile device as possible.

In case of session mobility, the application is instantiated at a new device, and the state of the Entity (representing the running application) is transferred from the old device to the new device. The mobility of entire access networks, e.g. of a LAN in a train, is accomplished in a very similar manner to the mobility of a mobile device. This is because the involved node CTs and network CTs have the same functionality and Application Programming Interfaces (APIs) and are thus treated in a similar manner.

3.3.3 Multi-Homed End-to-End Mobility

The design of a future multi-homing protocol for multi-interface mobile terminals requires the use of multiple paths between the communication terminals. These paths can be combined to provide composite services as one unique higher level end-to-end path [2]. Multihomed End-to-End Mobility (MEEM) is a mobility management mechanism defined for the GP architecture in which mobility is handled by the multihomed terminals.

As illustrated in Figure 3.3, users of multi-interface mobile terminals can connect to different networks simultaneously through several interfaces. A Multihomed End-to-End GP (MEE-GP) is thus composed of several end-to-end sub-GPs. Mobility related to these interfaces can be handed in an end-to-end manner taking advantage of multihoming. When mobility occurs over an interface, seamless handover is much

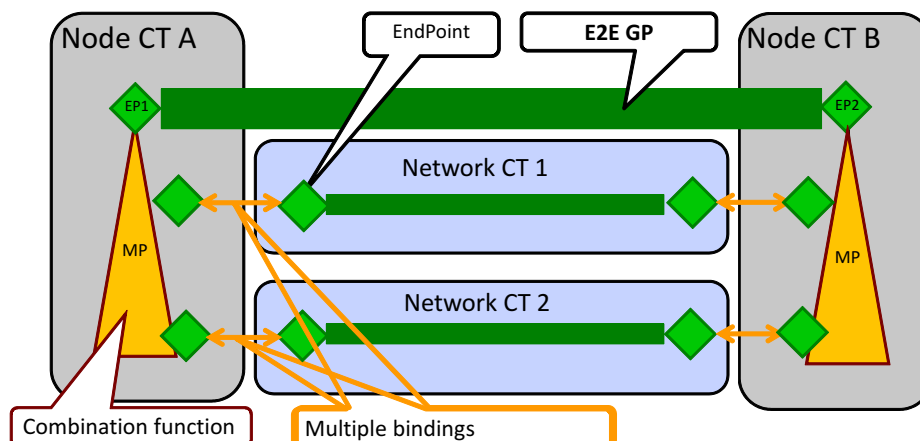


Figure 3.3: Multihomed End-to-End GP.

easier achieved by switching traffic into a secondary interface before the handover and back to the initial interface afterwards. This approach is especially suitable for the case in which the handoff decision is taken by the terminals.

This proposal introduces two functions for mobility management which are implemented within the MP, combination function and multiple binding.

- Combination function estimates performance of the combined end-to-end GP in order to decide the set of paths that should be used simultaneously and the scheduling algorithm that should be used over these paths to provide the best performance.
- Multiple binding sets up and maintains the bindings between the EP of the end-to-end GP and the EPs of the end-to-end sub-GPs. This binding is dynamic. When a node's locator is changed due to mobility, multiple binding is responsible to update the bindings with the new locator.

The advantage of end-to-end mobility management is that no new network entity needs to be defined. However, this approach should not be used in case of large number of end-to-end-path connected to a terminal (e.g. one of the terminals is a big server which has to maintain hundreds or thousands of connections from clients). End-to-end mobility should be used for applications such as video or audio streaming, video or audio telephony or other applications in which data transmission is carried out between two or several multi-technology multi-homed mobile terminals. End-to-end mobility management can be found in SCTP [42] and SHIM6 [43]. The MEEM principle can be applied to other existing protocols such as TCP or UDP to obtain multipath TCP or multipath UDP with end-to-end mobility support. It can be also used as a principle to design new protocols integrating multi-homing and end-to-end mobility management.

3.4 Triggers and Handover Decision

Three steps can be differentiated for supporting mobility management, and more precisely, handover management while maintaining ongoing communications for a moving End-Point (see figure 3.4):

1. **the handover initiation** results from information or event gathering (link monitoring, new incoming call, etc.), leading to a handover decision-making process
2. **the handover decision** has a direct impact on the network resource management, it selects a new connectivity for a dedicated communication taking into account user preferences, operator policies, access link QoS, etc.

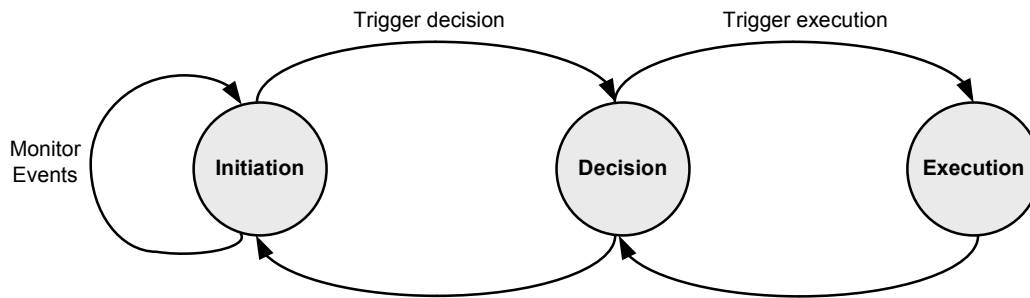


Figure 3.4: Handover steps

3. **the handover execution** may include a possible handover preparation (pre-attachment, QoS context or service renegotiation, context transfer, etc.) before the execution itself consisting of the End-Point connectivity switch and corresponding location update for the data forwarding adaptation.

Current wireless networks technologies (3G, WLAN, LTE, ...) apply different approaches for distributing such events, information and control functions among terminals and networking entities, at various protocol layers. In this context, the GP mobility framework needs to be highly flexible to self-adapt to various heterogeneous connectivity technologies on one hand and to provide efficient multi-level (terminal, session, network) mobility on the other hand. The realization of handover initiation, decision and execution functions in the three ways of managing mobility can be hence distributed and coordinated between hosts and one or several network entities:

- in DMA scheme, handover steps are coordinated in both terminal and access nodes entities, either being controlled at the terminal or the network side;
- in the AM approach, different network entities can take part in the handover process, depending on the considered EP location (access network, core network, hosts);
- in the MEEM, the different handover steps are under host control.

Radio access technologies specificities, such as 3G or WLAN for example, become constrained to the wireless link CT only. Our three mobility schemes apply a global approach able to take benefit of the capabilities of various access technologies.

Beyond the pure handover execution performance, handover steps analysis outlines that an adequate handover triggering and decision framework is key for the global efficiency of mobility management schemes. Several frameworks to facilitate such triggering exist. In the Ambient Networks project a mobility toolbox as a component of the Ambient network architecture was proposed [44]. Also [45] enables to handle mobility mechanisms based on the modular extensible Ambient control space. [46] defines a framework that supports the event collection and processing and triggering from hundreds of different sources. Lastly, the MIH model, see [47], support distribution of event and control information relative to handover initiation and decision steps. It allows for terminal's, network's or shared handover decision model as shown in [48].

All these frameworks concentrate on context information and triggering and do not refer to the mobility algorithms applied. We herein focus more on mobility execution functions, considering their use with such existing frameworks. Thus, in the following, we evaluate and compare more deeply our three mobility solutions.

3.5 Mobility Evaluation Framework

Prior to comparing the different mobility schemes with each other, we provide some initial and specific evaluation experiments. For DMA our simulation results validate the model and allow performance comparison with more traditional mobility schemes. In the case of MEEM, both simulations and prototype validate and assess performance of the proposed approach. For the AM scheme a further performance evaluation was neither planned nor conducted. However when we started the AM considerations, a proof of concept implementation using OMNeT++ was performed.

3.5.1 Simulations for DMA

In order to evaluate the benefit of the DMA scheme with regards to more traditional centralized mobility management, we modelled it using standard IPv6 protocols under the OPNET Modeler tool [49]. Our simulations aim at evaluating TCP and UDP based traffic efficiency obtained on a wireless campus network. As an alternative to DMA, we modelled the MIP+ scheme, an optimized centralized mobility approach providing results in accordance with state of the art's centralized schemes. The MIP+ model is based on the Mobile IPv6 protocol enhanced with cross layer handover optimization. The same optimization between radio and network layers are integrated in both DMA and MIP+ models (radio scanning, handover triggering, IP address configuration, etc.) related contexts in AN when necessary.

The 45 Mobile Nodes (MNs) are moving randomly at a pedestrian speed of 1 m/s within a network consisting of 13 access points during one hour. Each AN is connected to a network gateway to which are connected application servers and HA as well. In the MIP+ scenario, the link delay between the gateway and the HA is 20 ms, which models a typical nationwide network with a central HA. Two typical applications are provided by dedicated servers reachable through the gateway: video streaming session and file transfer. The video stream lasts 360 seconds at constant bit rate of 250 kbit/s delivered over UDP; the file transfer application uses FTP, it connects to the server, downloads a 500 kbyte file and releases the connection. When the application ends, it is automatically re-launched with inter-repetition time of 10 seconds, leading to generation of a new flow. TCP version in use is NewReno with Selective ACK and receive buffers of 8 kbyte in MN and 32 kbyte in server. Other TCP parameters are tuned to be as close as possible to existing mobile terminals' implementations. Lastly, one third of the 45 MNs use the streaming application only, one third perform file transfers only and one third launch simultaneously streaming and file transfer applications.

In one hour lasting simulation scenario, 1849 handovers occurred, 225 streaming sessions and 990 file transfers were completed. In Table 3.1, we compare collected handover duration statistics: better performances are achieved for DMA than for MIP+, especially for the maximum and average handover values. Indeed, minimum handover duration occurs in perfect conditions where the only additional delay in MIP+ scheme corresponds to the link delay between the gateway and HA. When handover conditions worsen, the achieved performance is better in DMA scheme comparing to MIP+. Centralized schemes are more sensible than distributed ones when network conditions deteriorate due to packets loss, delays and collisions. The cumulative duration of 1849 handovers occurring in each scheme outlines that during the one hour simulation, MN are not reachable for about 13 minutes in DMA, 20 minutes in MIP+ with local HA, and 21,5 minutes in MIP+.

The following figures compare several performance metrics for DMA and MIP+ schemes:

- Figure 3.5(a) depicts the tunnelling traffic during the whole simulation. The MIP+ scheme generates more encapsulated traffic than DMA where tunnels are not used when traffic flows are anchored to MN's current AN. Centralized tunnelling in MIP leads to extra transfer and computing traffic delays, especially in the core network where all the encapsulated packets are aggregated and computed by the centralized HA. Indeed, even if DMA makes use of tunnels when the MN moves, their scope

Table 3.1: Handover duration

	DMA	MIP+
min.	0,004	0,024
max.	5,604	9,403
average	0,420	0,607
standard dev.	0,635	1,095

remains local to the flat access network and tunnel endpoints distribution between ANs ensures high scalability compared to centralization towards a single Mobility Manager (e.g. HA);

- Figure 3.5(b) compares the cumulative streaming traffic received by the all MNs. DMA scheme slightly outperforms MIP+ scheme, which is a direct consequence of shorter handover duration leading to less packet loss;
- The average transfer delays for UDP packets, as well as for TCP segments are depicted in Figure 3.6(a) and (b), respectively, confirming that better performance is achieved with the use of DMA. Indeed, traffic flows routing avoids redirections and tunnelling (when the MN is attached to the flow's anchor); or the induced tunnelling delay is far less important than for the MIP+ scheme where RTT between the gateway and the HA is at least 40 msec;
- Cumulative TCP retransmission count as shown in Figure 3.7 is far better for DMA scheme than for MIP+. This is also a direct consequence of packet losses induced by the handovers.

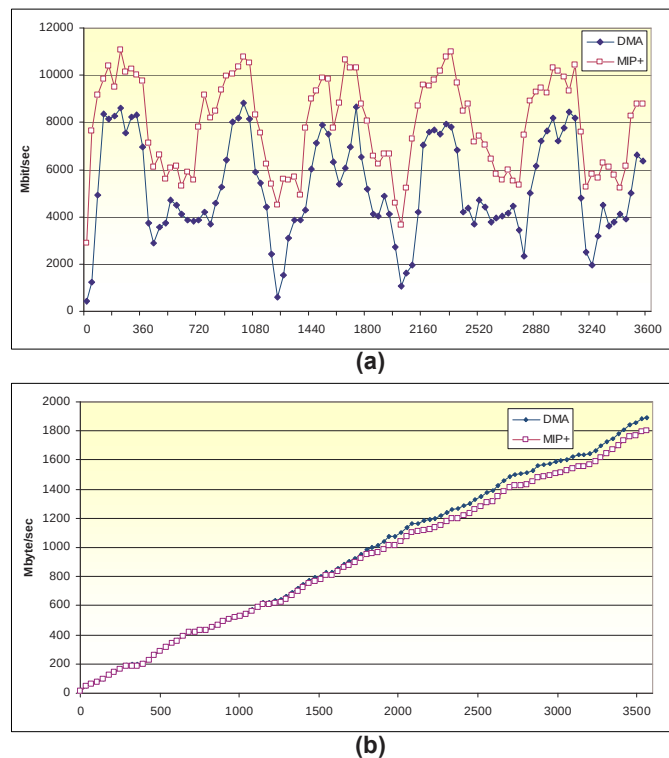


Figure 3.5: (a) Tunnelled traffic sent and (b) Streaming traffic received

Globally, the obtained results outline that, compared to traditional centralized mobility schemes, the dis-

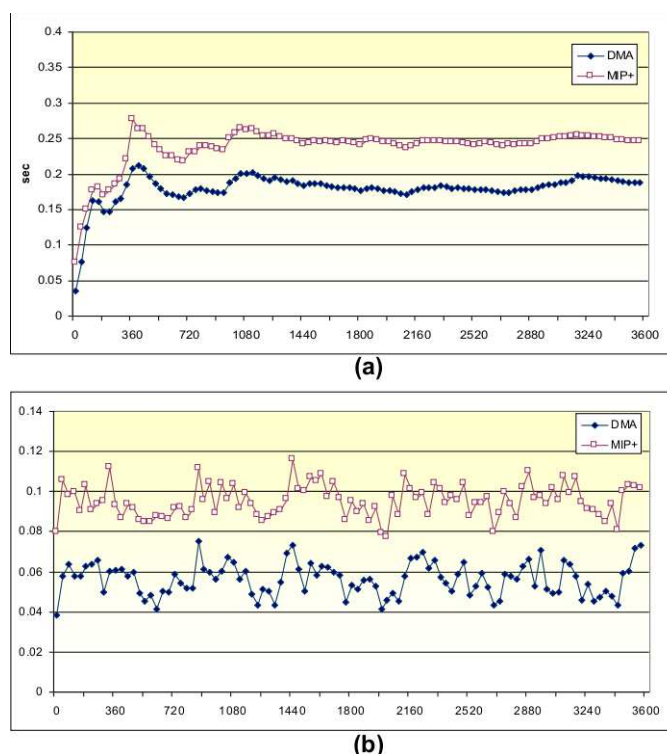


Figure 3.6: Average end-to-end delay (in sec.): (a)UDP packets (b)TCP segments

tribution of mobility anchors allows better handover performance, traffic delays and applications efficiency. Beyond pure performance comparison, the most important outcome is the fact that we can reach better scalability by distributing the mobility management between access nodes without degradation of the global efficiency. Such scalability related properties are obviously of great importance for the future Internet and the wireless networks in general, where the properties of our distributed approach is relevant for the optimization of mobility in various heterogeneous contexts.

3.5.2 Simulation and prototype for MEEM

In order to evaluate the efficiency of MEEM and the benefits of MEE-GP, both simulation and prototype are realized. The simulations based on NS-2 demonstrate the efficiency of our scheduling algorithm which distribute data over available sub-GPs taking into account the path conditions using cross-layer information. The prototype is the proof-of-concept of MEEM which shows the benefit of using the shortest path to realize the handover of another path and measures the handover delay in different scenarios.

The two functions presented in Section 3.3.3 are member functions of MP which interconnects the MEE-GP and the end-to-end sub-GPs. In order to connect the EP of the MEE-GP to an EP of an end-to-end sub-GP within a Node CT it is used a binding realized by a hook.

The combination function needs to be aware of the available interfaces (i.e. interface detection). A multihoming protocol should at first be aware of the available network accesses (Ethernet interface, WiFi interface, 3G interface, etc.). This function detects the lowest level GPs (the physical interfaces) which can be used as candidate sub-GPs. The result of this function is an active interface table 3.2.

The combination function detects all the candidate end-to-end sub-GPs which can be used to constitute an MEE-GP between the two EPs. The end-to-end multihoming protocol should be able to detect available end-to-end paths between two EPs (e.g. for two terminals having two interfaces, each can have 4, 3 or 2

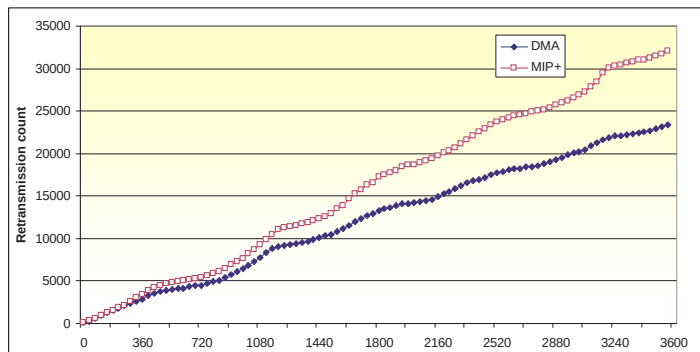


Figure 3.7: TCP retransmissions

Interface name / technology	Addressing scheme / Endpoint ID
wlan0 / wi-fi	MAC / 00-24-2C-39-4C-64
eth0 / ethernet	MAC / 00-13-77-B7-7C-57

Table 3.2: Active interface table

available end-to-end paths depending on network topology). The result of this function is a table in the following form:

Sub-GP name / Compartment / Addressing scheme	Local sub-GP Endpoint ID	Peer sub-GP Endpoint ID
Sub-GP1 / TCP / (port number, IP unicast address)	2222, 192.168.1.10	3333, 192.168.1.11
Sub-GP2 / TCP / (port number, IP unicast address)	2222, 192.168.2.100	3333, 192.168.2.101

Table 3.3: Sub-GP table

Based on the sub-GP table and performance estimation methods, the combination function can provide estimated characteristic of a combined set of sub-GPs. Together with the QoS requirements of the application flow, the combination function can result in the set of sub-GPs and the scheduling algorithm to be used.

For performance estimation, several methods and algorithms exist. The simplest way to provide estimated bandwidth of an interface is to use its theoretical bandwidth. If two available sub-GPs are associated with two interfaces WiFi and 3G, the available bandwidth is 11 Mbps and 384 Kbps respectively. More efficient performance estimation methods can use RTT measurements, packet-pair methods [50] or even cross-layer information [51].

For scheduling algorithm, several algorithms exist. The simplest scheduling algorithm is Round-Robin which sends data equally over the set of sub-GPs without taking into account the paths conditions. We also proposed a scheduling algorithm called crosslayer-CMT [51] which minimizes the packet reordering ratio at the receiver. This algorithm schedules the packets over the fastest path according to the path condition estimation using cross-layer information, the RTT at the transport level and the FER at the MAC level of WLAN. The results obtained by NS-2 simulation are presented in Figure 3.8a and 3.8b.

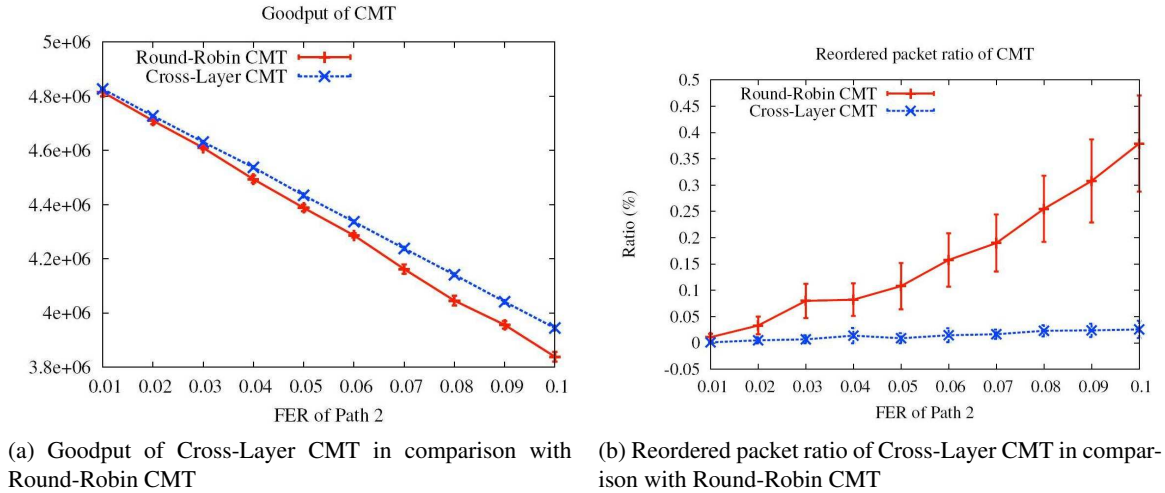


Figure 3.8: Cross-Layer CMT simulation results

In this simulation, we varied FER on WLAN link to observe the performance of the two protocols Round-Robin CMT [52] and Cross-Layer CMT. Figure 3.8a shows that Cross-Layer CMT can obtain better goodput than Round-Robin CMT when FER increases. The performance of Cross-Layer CMT can be seen better when looking at the reordered packet ratio shown in Figure 3.8b. Without taking the path conditions into account, there are many packets received out-of-order at the receiver of Round-robin CMT. Cross-layer CMT can considerably reduce the packet reordering ratio because the protocol monitors FER and can avoid important delay difference between paths caused by the quality of the WLAN link.

The multiple binding function is responsible for assuring the local and peer sub-GP EPs are always updated. If a handover happens leading to a sub-GP EP change, this function has to be aware of this change and update the corresponding EP identifier accordingly in the sub-GP table. When the identifier of an end-to-end sub-GP changes due to handover, a multihoming protocol should send update messages to the peer to update the identifier of the EPs of this sub-GP accordingly. This is the principle of MEEM presented in Section 3.3.3. Mobility management can benefit from multi-homing feature in this case. Data can be sent over alternative path to reduce data loss during the handover. Identifier updating messages can be sent over alternative available authenticated path simplifying the security aspect in mobility update. Shorter delay path (e.g. WiFi) can be used to reduce the handover delay over a long delay path (e.g. cellular end-to-end path).

In order to measure the handover delay, we use a test-bed. The MP and the member functions related to MEEM are implemented in C++. The sub-GPs can be TCP sub-GP, UDP sub-GP, SCTP sub-GP, IP sub-GP and even Ethernet sub-GP if it is available. This implementation shows the flexibility of the GP concept. A MEE-GP can be a combination of sub-GPs belonging to different levels, a UDP sub-GP and an Ethernet sub-GP for example. Inside the Node CT, the hooks can interconnect different GPs belonging to different CTs at different levels using different identifier schemes to build a new type of GP in a new CT which is the MEE-GP. The scenarios under test are presented in Figure 3.9.

Three scenarios are defined corresponding to three test-bed architectures. The first scenario (Figure 3.9 (a)) corresponds to the communication between two laptops within a local network. The second scenario (Figure 3.9 (b)) corresponds to the communication between two laptops, one inside a local network and the other has a long delay path (i.e. the 3G link). The third scenario (Figure 3.9 (c)) takes cell phone, a device with limited resources, with a long delay path (i.e. the GPRS link) into account. For each scenario, IP address change is used as handover trigger to measure the handover delay provided by MEEM. Figures 3.10a, 3.10b, 3.10c, and 3.10d show the handover delays in different scenarios.

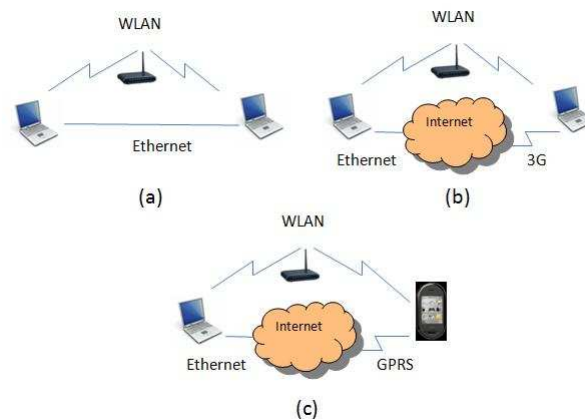


Figure 3.9: Test-bed architecture and scenarios for MEEM

When there is a handover over an interface related to a sub-GP, the MEE-GP EP chooses the fastest sub-GP among the remainder available sub-GPs to send update messages in order to minimize the handover delay. Figure 3.10a shows the handover delay measured in the first scenario when update messages are sent over the Ethernet sub-GP to realize a handover for the WiFi sub-GP. Figure 3.10b shows the handover delay measured in the second scenario when update messages are sent over the 802.11g sub-GP to realize a handover for the UMTS sub-GP. Figure 3.10c shows the handover delay measured in the second scenario when update messages are sent over the UMTS sub-GP to realize a handover for the WiFi sub-GP. This delay has a high variation depending on time of day. Normally, address update messages sent over the WiFi interface after getting a new IP address may reduce the handover delay in comparison with sending update messages over the UMTS sub-GP. However, in the case of double-jump when the two WiFi endpoints move simultaneously, they lose the contact of each other and sending update messages over the UMTS sub-GP is the only solution. Figure 3.10d shows handover delays measured in the third scenario when update messages are sent over the GPRS sub-GP.

3.6 Mobility Comparison and Combination

In this section we compare the three mobility approaches depicted in Section 3.3 and illustrate some results derived by simulation or by analytical analysis [53].

For a first rough comparison we provide a tabular overview in Table 3.4. In general, the following properties are valid for all three concepts. Firstly, all proposed schemes allow for seamless mobility. Secondly, while CT can be treated as a secure domain and we do not want to dive into specific security protocols for adding EPs to a CT or for rearranging GPs in a trusted way, we at least must provide a certain level of security to the signalling messages involved used by mobility management. More precisely, it means that all mobility signalling messages must be authenticated in a similar manner like Binding Updates (BUs) in MIP. Location management can easily be applied for all three schemes if that is foreseen by the operator. If the IP protocol is prevalent the location management can be done by use of the DNS, [54], [55] and/or the SIP [56].

Next we depict the three mobility solutions at hand of a GP architectural mobility example. Figure 3.11 sets the scene and presents the example at which we want to show the differences and similarities of the three mobility proposals suggested in Section 3.3. We select an example of a MN (CT A) that contains a WLAN interface as well as a 3G interface. This Node CT A has established a voice connection to Node CT F. The voice E2E GP between EP1 and EP15 is initially realized by the sequence EP1, EP2, EP3, EP7, EP6,



Table 3.4: Mobility comparison table

	DMA	AM	MEEM
Proposal	Dynamic Mobility Anchoring	Anchorless Mobility	Multihomed End-to-End Mobility
Targeted Problem Domain	Distribution of mobility function in the access NW	Access and Core networks	Terminal Mobility and terminal multi-homing
Terminal Mobility	yes	yes	yes
Session Mobility	no	yes	no
Network Mobility	yes if the access router of the NW uses DMA	yes	no
Handover performance (high/low)	Better than traditional centralised (MIP based) schemes	Slightly better than with MIP as the path to the branching point is usually shorter than to the HA	Handover delay depends on the distance between the two terminals. Seamless handover and high performance handover can be achieved by the use of a secondary path.
Interoperability	fully interoperable with IPv6	not targeted (clean slate), needs additions	Interoperable with multipath transport (SCTP, mTCP...)
BU informing the CN needed		Only if path properties change	Yes, it's end-to-end mobility management
Assumes symmetric routing	yes, could be changed	yes, could be changed	no
Critique	Can lead to inefficient routing	Trades in routing issues for being anchorless	Can lead to important delay for mobility management because of end-to-end address update. Problem with NAT.
Simulator	OPNET Simulator on top of IP	Proof-of-concept in OM-NeT++	NS-2 simulation for flow distribution algorithm
Prototype	not planned	not planned	MEEM prototype in C++ for handover delay measurement
Scope of application area	Application can afford longer delay	If short delay is needed	Mobile devices having multiple access possibilities.
Comment	Actual mobility anchor is taken and kept for a new flow. Distribution of mobility functions in the access NW. Avoids typical MM centralisation/encapsulation in the core network	Uses dynamic bindings in endpoints and new shortest path to existing path or CN	MEEM takes the end-to-end mobility management principle from SCTP and SHIM6 and can be used with any transport layer protocol for mobility support

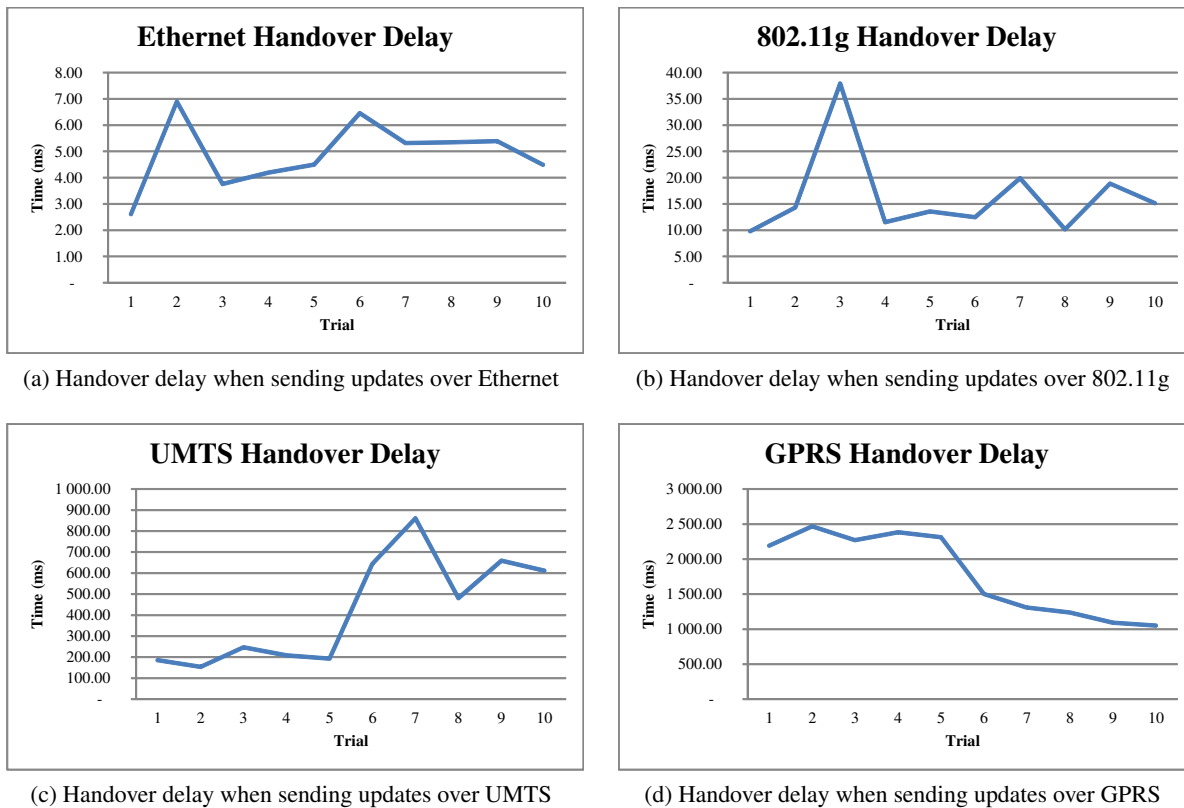


Figure 3.10: Handover delay in the different scenarios

EP12, EP14, EP17, EP16, EP15. Thus the GP uses two hooks inside Node CT A and Node CT F and one hook inside Node CT B and Node CT E. The MN A is attached to the network via WLAN Access Network 1. Thus initially GPs inside the WLAN Access Network CT 1, the Access Network CT 4 and also IP GP 1 is used.

Due to movement of Node A at a certain point in time the 3G Access Network is chosen to provide the connectivity to the network. The exact course of events now depends on the kind of mobility mechanism applied.

- In case of DMA the EP10 will establish a connection to the current mobility anchor EP6, which then also keeps the existing connection IP GP 1 towards EP15 via EP12.
- In case of AM the EP10 initiates a lookup for the shortest path towards EP15 and thus establishes IP GP 2 towards EP12.
- In case of MEEM with several simultaneously existing connections for example via EP7 and EP11, some or more of those connections can be added (e.g. via EP9) or removed (e.g. via EP7) due to mobility.

3.6.1 Dynamic Mobility Combinations

Various mobility solutions support within the given GP framework allows to self-adapt to different mobility situations. More precisely, it means that by combining the existing mobility proposals into a single dynamic mobility stack proposal [53], we are able to dynamically change the stack and adapt to all kind of mobility scenarios.

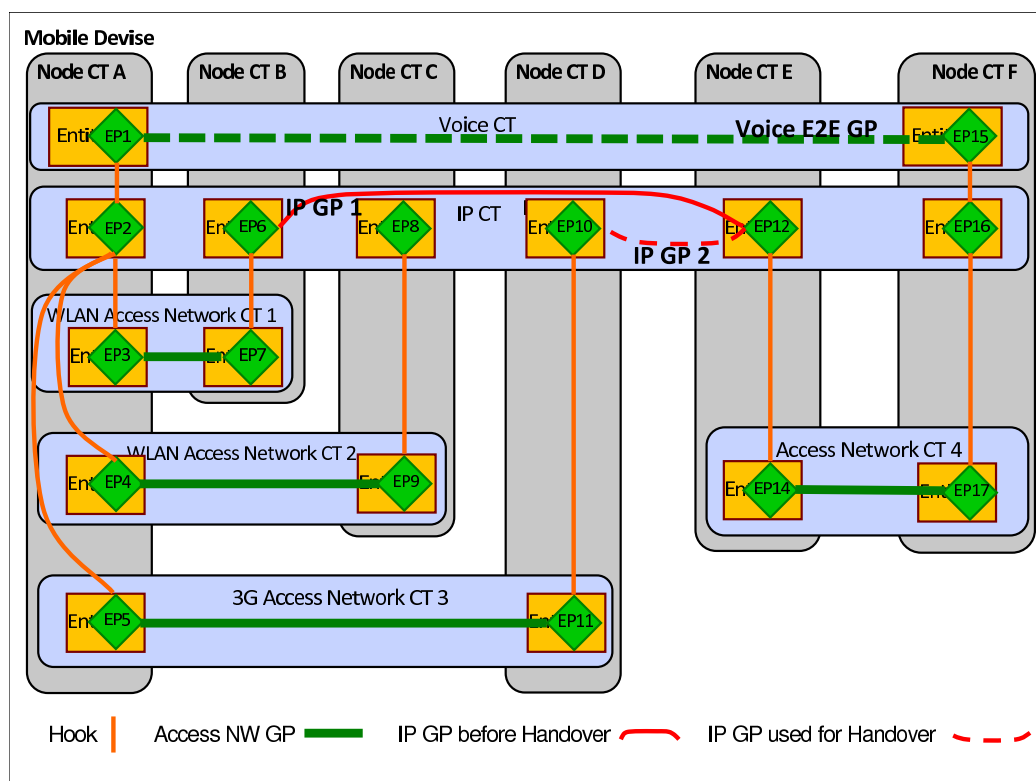


Figure 3.11: GP Comparison Picture.

The GP architecture facilitates this dynamic change of the protocol stacks in each node during run time. Moreover, this GP architecture property can be used to combine the capabilities and mechanisms of the different mobility concepts discussed above. Thereby, a very flexible realization of a mobility management concept superior over a wide range of different mobility use cases is possible. The combined concept can make use of the advantages of each protocol solution while at the same time it omits the disadvantages of each concept.

We can make beneficial use of DMA, AM and MEEM mobility concepts as follows. If the user location moves frequently and the running application can afford a longer delay, then the dynamic mobility protocol selection should switch to DMA. In case of a short delay requirement, AM has its advantage, but if the user changes the cell frequently there is a certain overhead in signalling and processing involved. If however the user needs a big pipe and is willing to pay more by making use of multiple access possibilities, then MEEM mechanism can be applied. Of course the latter mechanism will be more costly in terms of transport and also signalling, especially if applied to a highly moving user. Also, for real time applications a solution like DMA that is less costly in signalling and more efficient in terms of handover performance is recommended. For slow moving mobile users with high, but download only requirements, AM or MEEM is advisable.

Thus dynamical protocol switching realizes the optimal mobility protocol but needs certain information to control the protocol selection. To facilitate this we can apply context patterns and predict geographical access patterns, time of day patterns, social network patterns or other enhanced context information and deliver it to the mobility stack.



3.6.2 Mobility Comparison Conclusion

In DMA, the anchoring functions are activated only when needed, i.e. when the host moves with an ongoing GP that needs to be maintained. Such a dynamic property is new and has not yet been evaluated before. In AM binding and bi-casting functions are used in a more dynamic way than in traditional IP schemes. Thereby the AM approach benefits from the distribution of different EP objects among several CTs in the GP network. MEEM has its advantages when the mobile device is a multi-interface multi-technology terminal. Then each interface may support one or more connections and a user can connect to different networks simultaneously through multiple interfaces. In this situation MEEM solution enables mobility and thus takes advantage of multihoming, thereby improving the user experience and increasing the utilization efficiency of the network resources.

By combination of the proposed concepts we get some advantages. First, the combined mobility approach works for all kinds of mobility (session, terminal or network mobility). Second, it works across a variety of situations, be it slow or fast moving users in small hotspots or big macro cell environments. Unlike today's tunnelling solutions, the combined mobility concept allows local routing, flexible anchoring and multihoming. Thereby it leads to decreased transmission delays and lower bandwidth utilization and improved user experience.

3.7 Mobility Conclusion

In this chapter, we described and compared the properties of the three mobility approaches concerning flexibility, complexity, operability and applicability inside the GP framework. By making use of the GP concepts, the combination of those mobility solutions enable a flexible treatment of various mobility use cases (terminal, session and network mobility). Unlike a 3G/4G mobile network with its fixed functionalities distributed in various specialized network elements, the proposed solutions better adapt to changing topologies and structures. Each GP node can carry out the needed functionalities like forwarding, branching, mediating or binding. With support of the GP architecture these mobility solutions help to solve the issue of decentralization of the architectural elements. Thereby they facilitate multi-technology, multi-homed and multipath handover. Due to the object oriented approach even the introduction of new mobility properties or mobility protocols for future evolution of today's Internet architecture is straightforward. The sub-structured GP approach enables the design of a wide variety of different mobility concepts and types of mobility mechanisms.

4 Routing evaluations

This chapter considers routing-related work developed under the GP architecture scope, supported by corresponding evaluation results. It concentrates on two main concepts: one on a mechanism for interconnecting GP-domain islands with IP-based domains via inter-domain routing, and the other describing robust GP routing mechanisms.

4.1 Inter-domain routing

4.1.1 Introduction

This section describes the simulation experiments done in the area of inter-domain routing based on the framework that is able to interconnect next generation networks based on GPs over traditional Internet Protocol (IP) based infrastructures and thus can serve as a migration tool towards the Future Internet.

Figure 4.1 shows an example of how two GP islands are interconnected using a new routing CT called GP-Border Gateway Protocol (BGP) that is implemented as an extension of the standard BGP (IP-BGP). The evaluation work of the presented approach is divided into two separate simulation experiments; i) the proof of concept simulation to study the feasibility of the GP extension, and ii) the simulation experiment to study what kind of benefits the inter-CT routing could potentially provide in error cases.

The proof of concept simulation is a protocol level simulation that is developed to study how robust and stable Forwarding Information Bases (FIBs) and Routing Information Bases (RIBs) are with the GP support. This support is implemented as a new multiprotocol extension. One additional goal is to analyze and study what kind of additional BGP signalling is required. This simulation model and the results are introduced in Section 4.1.2.

The second part of this evaluation work consists of a higher level simulation to study how inter-CT routing between GP islands could enhance connectivity between Autonomous System (AS) pairs when some of the paths interconnected pair members will go down. This simulation is further explained in Section 4.1.3

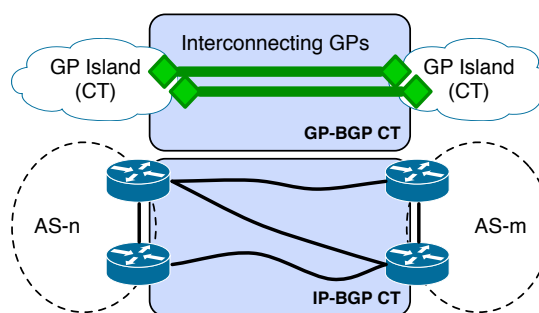


Figure 4.1: An example of interconnecting GP islands using GP framework concepts

4.1.2 GP-BGP evaluation

One way to interconnect different GP islands over traditional IP based infrastructures is to use IP tunnel [57] having BGP routable addresses as end points. This tunnel is terminated at border gateways located at GP islands. Inter-GP islands routing is established by defining the GP Network Layer protocol that defines the so-called GP-BGP, a new routing hierarchy that enables GPs islands routes exchange over the traditional BGP.

To enable BGP to support routing for multiple Network Layer protocols, Multiprotocol Extensions for BGP [58] adds to BGP the ability to associate a particular Network Layer protocol (e.g., IPv6, IPX, L3VPN, etc.) with the next hop information and Network Layer Reachable Information (NLRI). GP-BGP uses the Multiprotocol Extensions for BGP [58] capabilities to exchange GP islands routes and achieve, together with IP tunneling, the inter-GP islands routing.

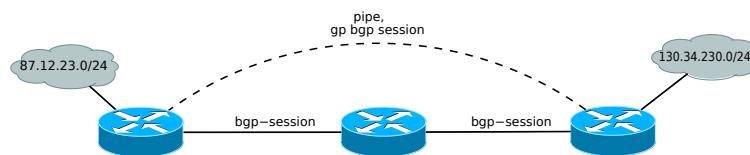


Figure 4.2: GP-BGP Basic Setup

In order to provide a proof of concept and a first evaluation of the applicability of the proposed solution, the Generic Path-BGP extensions have been implemented for a proof of concept on the JavaSim (JSIM) [59]. Figure 4.2 shows a basic simulation environment with three nodes, the two in the edges running a GP-BGP session over an IP tunnel. Figure 4.3 shows the decomposed logical functionality of each of the edges, where the GP-BGP logical function interacts with the TCP logical function for forwarding GP-BGP packets, and with the routing table to include the paths exchanged with other nodes via these GP-BGP packets.

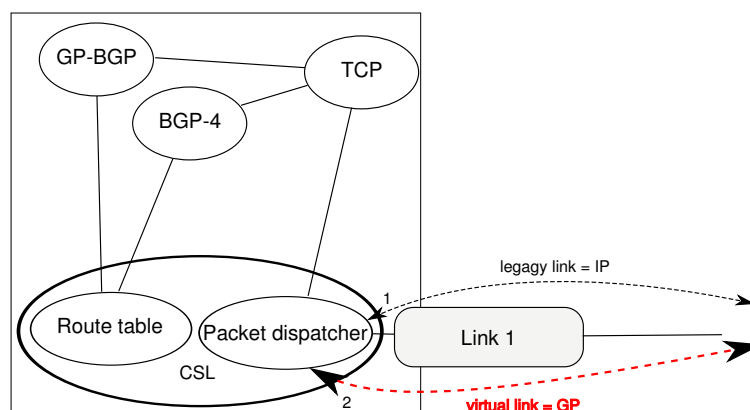


Figure 4.3: Node structure in the simulation environment

4.1.2.1 Fallback scenario

There is however a situation where there is no loss of connectivity when a specific link or router failure happens. Figure 4.4 shows the simulation environment in that case. It shows a topology where the border gateways of the routing CTs (simulated by prefixes 1.0.0.0/8, 2.0.0.0/8, 6.0.0.0/8 and 7.0.0.0/8) run a full mesh of GP-BGP sessions. Dashed lines show GP-BGP sessions, while continuous lines show IP-BGP sessions. As mentioned before and in order to obtain early results, routing compartments were simulated as IPv4 prefixes.

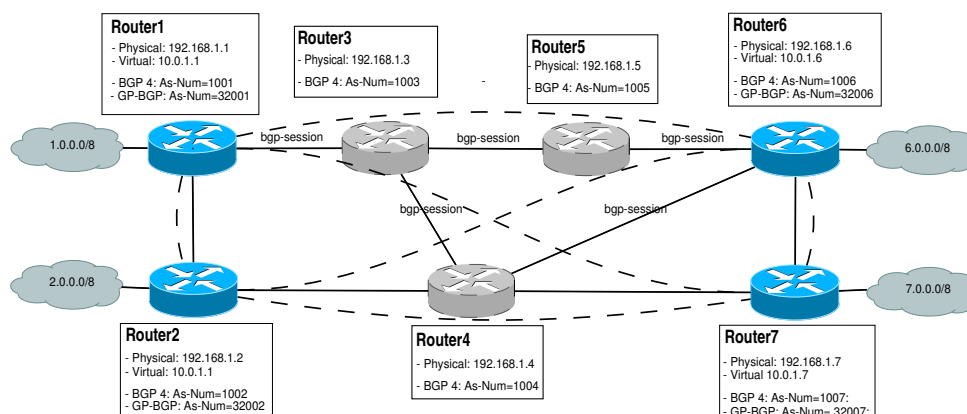


Figure 4.4: Connection of AS Compartments

The following listing shows the routing status for Router1 after both IP and GP routes are exchanged:

```
192.168.1.2 -> direct link
192.168.1.3 -> direct link
192.168.1.6 -> 192.168.1.3 (1003 1004 1006)
192.168.1.7 -> 192.168.1.3 (1003 1004 1007)
10.0.1.2 -> virtual connection (192.168.1.2)
10.0.1.6 -> virtual connection (192.168.1.6)
10.0.1.7 -> virtual connection (192.168.1.7)
```

where the virtual connection refers to the IP tunnel to be used for reaching the other CT routes. The IP tunnels generated are shown in Figure 4.5a.

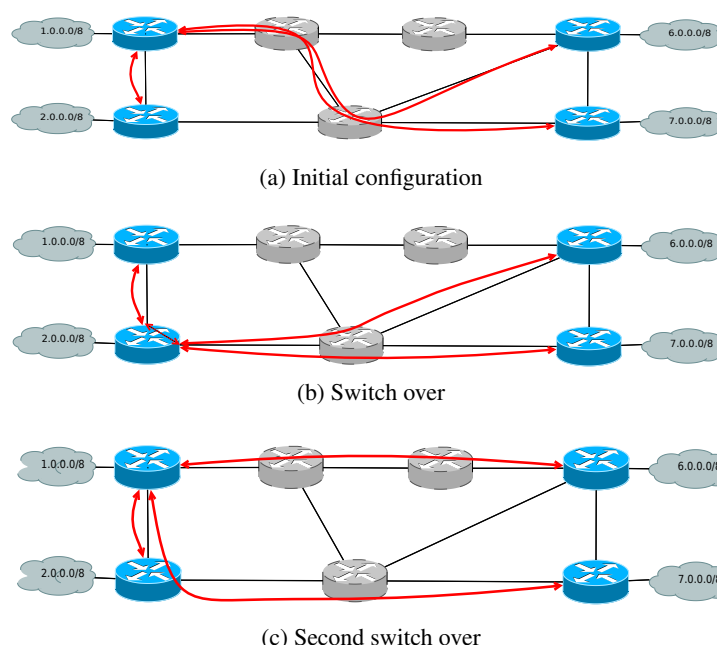


Figure 4.5: Evolution of tunnels in Router1

At instant $t=1000s$, the link between Router3 and Router4 fails so, when Router3 stops receiving the IP-BGP keep-alive messages, it resets its BGP Finite State Machine with Router4:

```
1084.8517681890637 bgp-id=3232235779, peer=(1004;192.168.1.4/32), ESTABLISHED=(HoldTimerExp)=>IDLE
```

Keep-alive messages period is set to $T=30s$ while the time to tear down a session when not receiving



keep-alives is 3*T for both GP-BGP and IP-BGP. Router3 withdraws the routes previously exchanged with Router4 announcing these withdrawals back to Router1, which modifies its routing table accordingly:

```
1089.8523388557303 192.168.1.7 -> 192.168.1.2 (1002 1004 1007)
1089.8529095223969 192.168.1.6 -> 192.168.1.2 (1002 1004 1006)
1119.8523281890637 192.168.1.6 -> 192.168.1.3 (1003 1005 1006)
```

The route towards 192.168.1.6 and 192.168.1.7 now traverses Router2 instead of Router3. Note that the route towards 192.168.1.6 bounces back to Router3 once Router3 informs Router1 about the new path across AS 1005 instead of AS 1004.

In the meantime, even before the IP-BGP routing reorders its routing, the GP-BGP in Router1 reroutes its traffic through Router2 when it does not receive the appropriate keep-alive messages from Router6 and Router7:

```
1075.3373485086665 bgp-id=167772417, peer=(32006;10.0.1.6/32), ESTABLISHED=(HoldTimerExp)=>IDLE
1089.0377309912103 bgp-id=167772417, peer=(32007;10.0.1.7/32), ESTABLISHED=(HoldTimerExp)=>IDLE

1075.3373485086665 6.0.0.0/8 -> 10.0.1.2 (32002 32006)
1089.0377309912103 7.0.0.0/8 -> 10.0.1.2 (32002 32007)
```

With this routing change on the GP-BGP level, traffic towards 6.0.0.0/8 and 7.0.0.0/8 from 1.0.0.0/8 will be sent towards Router2 using the previously setup tunnel for encapsulating traffic towards 2.0.0.0/8 from Router1. Router2 then decapsulates the traffic and encapsulates it again using as well the previously setup tunnels for encapsulating traffic towards 6.0.0.0/8 and 7.0.0.0/8 from Router2. This behaviour is shown in Figure 4.5b.

It is important to note that traffic is forwarded to 6.0.0.0/8 and 7.0.0.0/8 via Router2 using two consecutive tunnels before IP-BGP routing reacts from the link failure and updates its routes:

```
time routing recovers towards 6.0.0.0/8 is setup again via GP-BGP = 1075.3373485086665 s
time routing recovers towards 7.0.0.0/8 is setup again via GP-BGP = 1089.0377309912103 s
time routing recovers towards 6.0.0.0/8 and 7.0.0.0/8 via IP-BGP = 1089.8523388557303 s
```

Once Router1 re-establishes the GP-BGP session with Router6 and Router7,

```
1099.0435123245438 bgp-id=167772417,peer=(32007;10.0.1.7/32),OPENCONFIRM=(RecvKeepAlive)=>ESTABLISHED
1106.342276508667 bgp-id=167772417,peer=(32006;10.0.1.6/32),OPENCONFIRM=(RecvKeepAlive)=>ESTABLISHED
```

and the routes towards 6.0.0.0/8 and 7.0.0.0/8 are announced again to Router1, Router1 selects these paths:

```
1104.0466803245442 7.0.0.0/8 -> 10.0.1.7 (32007)
1141.3457858420006 6.0.0.0/8 -> 10.0.1.6 (32006)
```

falling back to the previous situation where direct and unique tunnels were used to forward traffic between the different CT routes. This behaviour is shown in Figure 4.5c.

It is important to note that even the tunnels are the same as in the initial situation, they traverse different ASs as specified by the underlying IP-BGP routing protocol.

Finally, as the link from Router3 and Router4 recovers at t=2000s the IP-BGP converges back to the initial situation of Figure 4.5a. This recovery is totally transparent to GP-BGP protocol as no sessions states are changed and no routing announcements are done: the only changes are on the ASs the tunnels traverse but not on any of the GP-BGP signalling.

4.1.2.2 Simulation results

In order to check the figures shown in the above scenario of Figure 4.4 1000 simulations were performed to see how the unpredictable conditions such as time delays, packet losses, etc. affect these results. Figure 4.6 compares the density functions of IP traffic recovery time in the IP-BGP case versus the GP-BGP case.

It is important to note that IP traffic recovery time in the GP-BGP case is always slightly better than IP-BGP case. This is the case because the IP traffic for the GP-BGP routes can be recovered in two ways; the first one when GP-BGP decides to re-route the traffic through the new path, and the second one when BGP

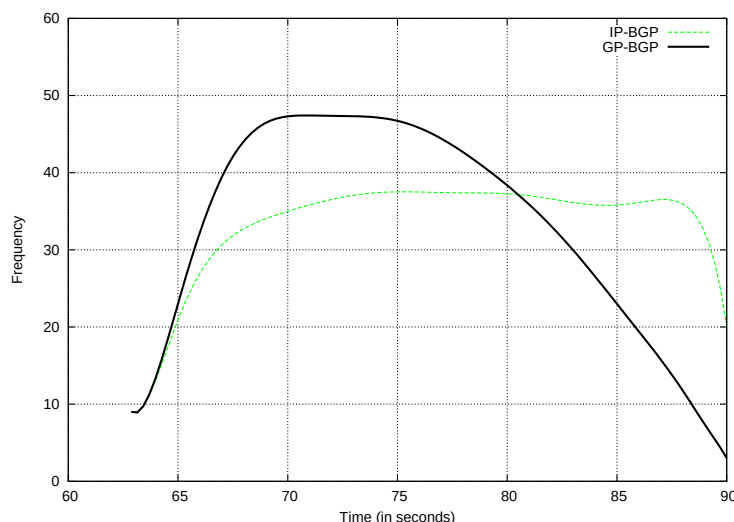


Figure 4.6: Density function of traffic recovery time

detects the failure and chooses a new alternative route to forward the traffic. IP traffic for the GP-BGP case take advantage from these two detection and recovery algorithms: its recovery time is the minimum between the two. Also, this gain can be improved by decreasing the GP-BGP time between keep-alive messages, so failure detections can be detected earlier; however, there is a tradeoff with the extra signalling burden it brings, therefore extra care is needed when setting this parameter.

4.1.3 AS pairing evaluation

This section describes the simulation experiment used to study of what kind of potential benefits AS pairing could result in case of routing incidents based on random error patterns. AS pairing means that a set of ASs with GP awareness are willing to forward and route GP transit traffic from/to their pair members. For non-GP traffic, ASs should follow the BGP routing practices meaning for instance that they do not allow to use their peering links for transit traffic. Such set of ASs can be seen as an GP island in the AS topology representing a core network's partition where GP traffic could receive special processing such as specialized routing and traffic engineering. Section 4.1.3.1 shows the simulation model and other used settings in the simulations. Section 4.1.3.2 presents the simulation results and their analysis.

4.1.3.1 Simulation model

The simulation model uses both the policy compliant routing and the shortest path routing based on the real AS topology used in Internet. This topology data includes all different relationships between ASs in order to support the standard BGP policy compliant routing. The data set is publically available in the Cooperative Association for Internet Data Analysis (CAIDA) website [60]. The data set used for the simulations consists of 32218 ASs and is a snapshot from the AS topology that was in use on October 2009. Since then, new ASs has been naturally added into it, but the used data set is still large enough and represents "Internet like" topologies.

For a single simulation round, the following parameters are randomized based on the uniform distribution;

- AS number of the error origin where a routing incident is going to start to spread, and
- Two AS number pairs representing GP islands for which interconnecting routes are computed using the policy compliant and the shortest path routing.

While randomizing AS number pairs, the policy compliant routing is used to compute their distances and the minimum distance requirement was 2 AS hops, i.e., the shortest acceptable distance between AS pairs was 3. Figure 4.7 illustrates different types of paths that are computed during a simulation and that are also part of the presented results.

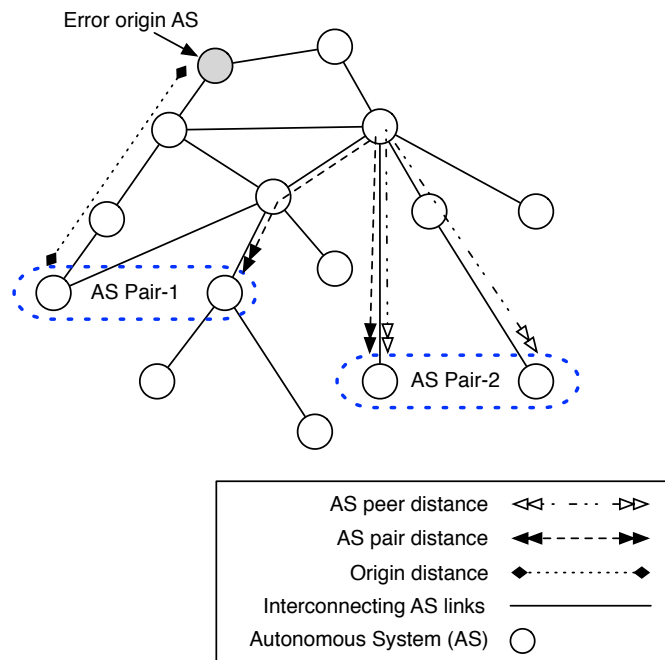


Figure 4.7: AS topology and measured AS distances

Algorithm 1 shows a high level algorithm that is used in the simulations. In the beginning, an AS topology data is read from the file and after this, an error pattern is created for each randomized error origin AS and based on these patterns, routes between randomized AS pairs are computed. After each step of path computation, the computed paths are stored for the analysis phase. The routes are computed using the standard BGP policy compliant routing and the shortest path routing algorithms.

Figure 4.8 illustrates how error patterns are generated based on the randomized error origin AS. In this example, the error origin is AS-A meaning that during error cycle 1, routes going via it are not usable anymore. For error cycle 2, all adjacent ASs of AS-A are affected, i.e., AS-B and C. And in the same way, routes via these two ASs are not usable anymore during error cycle 2, and so on. There is one exception in this, if for instance AS-A and AS-B has a peering relationship, then AS-A cannot "infect" AS-B since peering links are not used for transit traffic. So in order to "infect" an adjacent AS, the relationship with that AS must be something else than peering.

The simulation experiment consisted of two different simulation runs using the same AS topology data. Only difference between these runs was that they were using different seed value for random number generator, which resulted in the different AS numbers that were used to run a simulation. For both runs, the following parameters were used;

- Number of randomized error origins was 80,
- Number of randomized AS pairs per error origin was 100, and
- Error cycle was 4.

```

Input: Number of error origins, Number of AS pairs, Error cycles
Output: Error status
read AS topology data from file;
foreach Numberoferrororigins do
    Randomly select error origin AS j;
    Generate error pattern(j, Errorcycles);
    Build AS graphs;
    foreach NumberofASpairs do
        Randomly select AS pairs;
        Compute policy compliant path between AS peers;
        Compute shortest path between AS pairs;
        Compute policy compliant path between AS pairs;
        Compute shortest path between AS pairs and the error origin AS;
        Compute policy compliant path between AS pairs and the error origin AS;
        Store computed paths;
        foreach Errorcycles do
            Compute policy compliant path between AS pairs;
            Store computed paths;
        end
    end
end

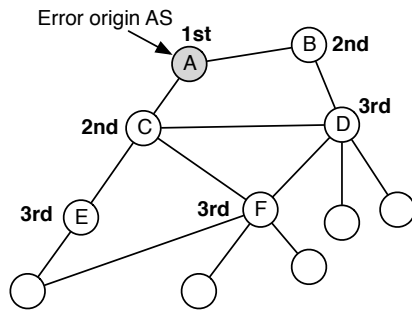
```

Algorithm 1: Simulation algorithm for path computation

4.1.3.2 Simulation results

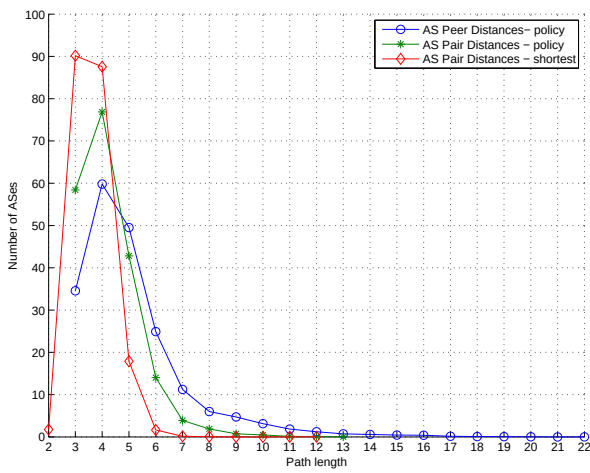
Figure 4.9 shows the measured path lengths (mean values) between AS peers forming AS pairs and between AS pairs for both simulation runs. AS peer distances (curve with circle marker type) are measured by using the standard BGP policy compliant routing and AS pair distances are measured by using both policy compliant (curve with asterisk marker type) and shortest path routing (curve with diamond marker type). The results in Figure 4.9a and Figure 4.9b have only minor changes due to randomization of the used AS numbers. As the subfigures illustrate, most of the AS peers have their distance between 3 and 5 AS hop count, but in some rare cases, this distance was even over 20 AS hop count, because while randomly assigning ASs, only the minimum distance was limited by 2. Another interesting finding is most of the shortest paths between AS pairs were 3 or 4 AS hop counts whereas the respective policy compliant paths were more evenly distributed resulting also a longer distances compared to the shortest ones available. This same trend can be seen in both subfigures. Additionally, it should be noted that because the policy compliant routing was used during the AS pair randomization to verify that the pairs are topologically far away enough from each other, in some cases, they were still 2 hops away from each other when interconnected paths were computed using the shortest path routing (curve with diamond marker type).

Figure 4.10 presents the measured distances (mean values) between the AS pairs and the error source AS from where a routing incident was originated. These distances were almost identical for both AS pairs due to the network graph characteristics, like an average path length between any two ASs/nodes in “Internet like” topology. The same trend that was visible in Figure 4.9 is also present here, i.e., an average path length when moving from the shortest path routing to the policy compliant routing is only slightly increased. Naturally, there are some exceptions as can be seen from the figure, i.e., when comparing the maximum path lengths between both routing schemes. In addition, the results of both simulation experiments illustrated in subfigures have only minor differences. The curves of AS peer distances in Figure 4.9 (with circle marker type) are almost identical with the policy curves in Figure 4.10 (with asterisk and circle marker types),

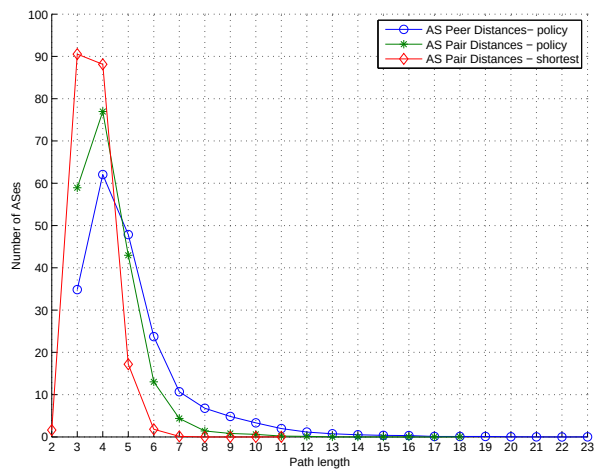


Error Cycle	Faulty ASes	Via ASes
1	A	-
2	B, C	A
3	D, E, F	B, C

Figure 4.8: An example of error pattern generation scheme

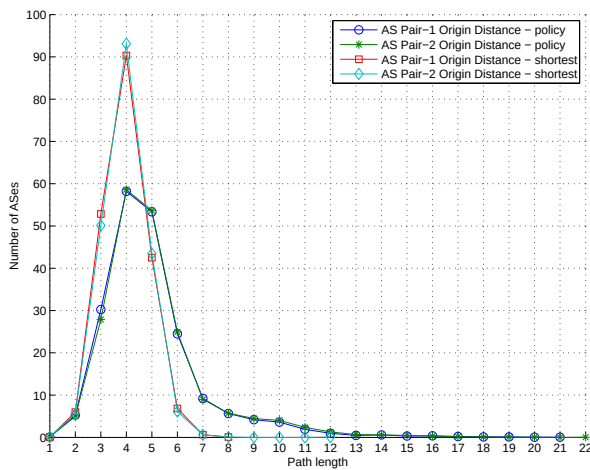


(a) Simulation run-1

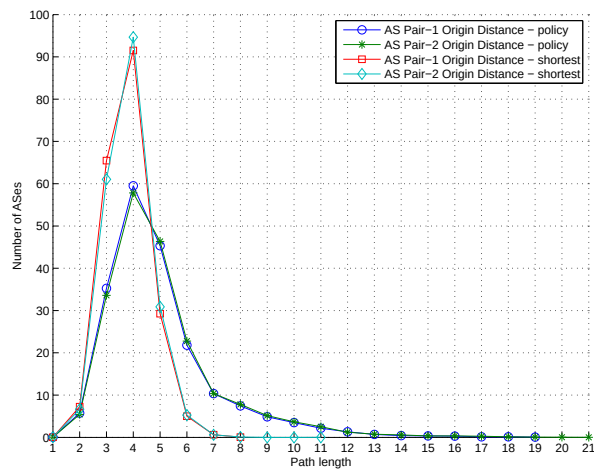


(b) Simulation run-2

Figure 4.9: Shortest average distances of AS peers and pairs



(a) Simulation run-1



(b) Simulation run-2

Figure 4.10: Shortest average distances to the error source AS

which was expected, since all these curves represent the case where 2 ASs are randomly selected in the AS topology and the interconnecting path length is measured. The rest 2 curves in Figure 4.10 (with diamond

and asterisk marker types) represent the case where 2 pairs of ASs are randomly selected and then the shortest interconnected path' distance is then measured either based on the shortest path or policy compliant routing.

Figure 4.11 shows how well ASs were able to maintain their interconnectivity during the error cycles. For individual ASs, if less than 4 paths were available, then it was considered that there is no connectivity. For AS pairs, any number of connecting paths was adequate to guarantee interconnectivity, because “peering” was supported between AS pair members. In Figure 4.11a during error cycle 1 and 2, the difference was practically non-existent varying between 100% and 99%. But when error pattern was spreading and more ASs were going down, the difference was getting bigger. For error cycle 3, the connectivity between AS pairs was 97.0125% and the respective value for individual ASs was only 63.275%. Finally for error cycle 4, the connectivity got worse in both cases and was for AS pairs 42.4875% and respectively for individual ASs 9.725%. Respectively, in Figure 4.11b the difference during error cycle 1 and 2 was practically non-existent; less than 1.2%. But when error pattern was spreading and more ASs were going down, the difference was yet again getting bigger. For error cycle 3, the connectivity between AS pairs was 96.85% and the respective value for individual ASs was 60.4625%. And finally for error cycle 4, the connectivity got worse in both cases and was for AS pairs 37.0875% and for individual ASs it was only 5.875%.

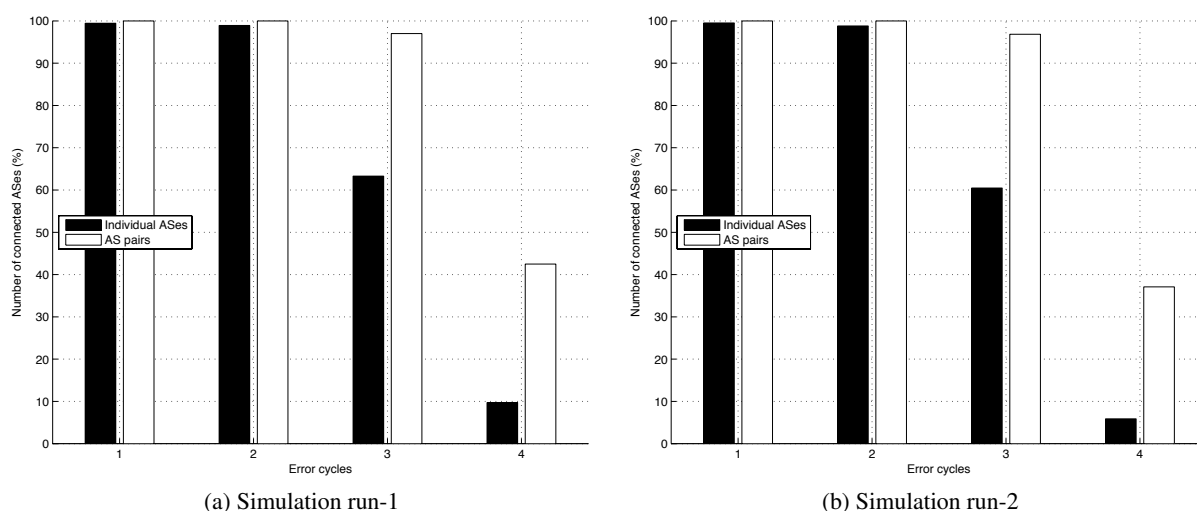


Figure 4.11: Connectivity between ASs and AS pairs

Table 4.1 shows the numerical values based on which Figure 4.11a was created. During error cycles 1 and 2, all 4 paths between ASs were maintained very well, but after that, its share starts to decrease rapidly and 1 and 2 paths start to dominate the interconnectivity between ASs and AS pairs. This also explains why the differences between “individual and pairs” in Figure 4.11 are getting so large. An interesting observation is the rarity of the cases where 3 paths were available, i.e., less than 1%. One explanation is that when an AS loses its connectivity to its remote pair, it tends to lose it to both of the ASs in the remote pair.

Respectively, Table 4.2 shows the numerical values for Figure 4.11b. One of the main differences compared to Table 4.1 is that in this case during error cycle 3 and 4, there is approximately 3-4% lower amount of cases where all 4 paths would have been available between the ASs. Otherwise, the results are consistent, i.e., whenever a value is clearly meaningful like > 10%, then it is visible in both tables. It should be noted that in both tables, there are some minor rounding errors for the columns representing the 1st and 2nd error cycle, i.e., the column sums are not equal to 100%, thus a few tenths of a percent less. This is the result of using mean values to make these calculations. All column sums of the tables represent the “AS Pairs” bars in Figure 4.11.

Paths	Error Cycle 1	Error Cycle 2	Error Cycle 3	Error Cycle 4
1	0	0	4.7125	17.975
2	0	0.525	27.9125	14.2125
3	0.175	0.175	0.7375	0.2
4	99.45	98.925	63.275	9.725

Table 4.1: Connecting paths (%) for Figure 4.11a

Paths	Error Cycle 1	Error Cycle 2	Error Cycle 3	Error Cycle 4
1	0	0	4.825	18.35
2	0	0.7	30.875	12.2625
3	0.1	0.1	0.3	0.2125
4	99.5125	98.8125	60.4625	5.875

Table 4.2: Connecting paths (%) for Figure 4.11b

So as the connectivity results presented in Figure 4.11 clearly show, if a topological area where a routing incident occurs is large enough, then use of AS pairing could potentially improve interconnectivity between ASs. But as Figure 4.9 shows, when using “peering” inside an AS pair to maintain connectivity towards a remote AS, this could mean that the used AS path length is even more than double compared to a “direct” policy compliant route with fault free topology.

4.1.4 Conclusions

In this chapter, a proof of concept for the interconnection of isolated GP islands based on Multi-Protocol BGP over the traditional IP based infrastructures and thus a migration tool towards the Future Internet has been presented.

It has been shown that this GP-BGP protocol behaves at least as good as regular BGP and its performance could even be further elaborated by setting the appropriate GP-BGP versus IP-BGP time to live session ratio. Theoretically GP-BGP time to live session could be fairly smaller than IP-BGP time to live session as, at least in its origins, GP-BGP topologies should be less complex as what we see nowadays on the Internet IP-BGP topologies. Also, it has been shown how fallback mechanisms can be supported by routing traffic between GP islands that may have a peering agreement.

Like the AS pairing simulation results clearly showed, additional AS pairs supporting inter-pair connectivity using GP-BGP routing could potentially provide more stable routing framework. However, the results should be considered to be indicative, as the results are case sensitive, which was visible in the connectivity results showed in Section 4.1.3.2, i.e., the differences are up to 5%, between the presented simulation runs. Another thing to be noted is that if a topological area where a routing incident occurred is relatively small, then AS pairing does not provide any significant benefits over the standard BGP routing since the BGP cope with these situations fairly well. But for a larger area, the gains are up to 30% compared to the connectivity between individual ASs.

4.2 GP routing and robust resource management

4.2.1 Introduction

Robustness, referring to the ability to cope with variations from the nominal operating conditions, is a key property of any engineering system. In this spirit, a robust network should be able to sustain acceptable performance despite foreseeable traffic variations and component failures. A common optimization objective in robust networking is to minimize the worst-case link loads, where worst-case should be understood as over all potential load variations or component failures. Our focus is on demand variations due to statistical fluctuations in an Multi-Path Routing (MPR) GP. This text describes two distinct but cooperative mechanisms under such scope, and refers to the obtained results in recent publications. The first proposal, Robust Multi-Path Routing, is an algorithms for improving guaranteed-performance routing, while the second one, QoS-RRC, is a mechanism integrating QoS-control and Resource Control strategies for dynamically controlling GP session requirements.

4.2.2 Robust multi-path routing

In order to balance load, the operator needs an estimate of the traffic situation in the network ([61], [62], [63]). Such estimates are usually based on a statistical model in combination with measurements and/or some estimation procedure to obtain values of the model parameters. Estimation and monitoring of traffic is studied in the joint task TC45 with WP4. The focus in WP5 is on how routing settings for GP's can be found that are robust to statistical fluctuations in traffic demand between end points. We assume traffic demands follow a Gaussian distribution with mean and Covariance matrix . Such models arise in many contexts and have been verified on many different real-world datasets. This model will serve as a point of departure for our calculations.

When the traffic is normally distributed, it is impossible to find a routing which guarantees that the link loads are below a certain target for all realizations of the traffic. However, we can deal with probabilistic requirements which, for instance, specify that a target link utilization should be respected with high probability.

Our approach for dealing with statistical uncertainty is developed via the concept of confidence regions. For example if the traffic is Gaussian, then the most likely outcomes of form ellipsoidal regions in a higher dimensional space. To optimize performance we cast the problem as a Second Order Conic Program (SOCP) that can be solved in polynomial-time using using, e.g., modern interior-point methods (see, e.g., [64]).

In our experiments we have found that correlations between flows play an important role. For instance, negatively correlated traffic flows can be routed on the same links in order to achieve a lower utilization on links. Similarly, positively correlated flows avoid common links in order to not amplify each other. Even when flows are uncorrelated the optimization gives a performance guarantee. Hence, when traffic varies according to a Gaussian distribution, service requirements within a Compartment can be met with prescribed level of confidence. Details and numerical examples of our findings can be found in [65].

4.2.3 QoS-aware Routing and Resource Control (QoS-RRC)

The success of the GP architecture greatly depends on the availability of efficient mechanisms for mapping physical resources to comply with each session requirements, in order to deliver end-to-end multi-party data transport with guaranteed QoS over the time. QoS-RRC [66] is a routing-enabled QoS control mechanism which consists of bootstrapping a routing table with minimum QoS-aware path information, namely the address of each router along the path and correspondent bottlenecks QoS requirements (per-class available



bandwidth and rates of delay, jitter and loss). For such, each ingress node floods the network, and each visited node initializes per-class over-reservations, adds its address and current QoS conditions to the message, and forwards it downstream. After completion, the egress node signals the ingress one. The flooding cycle provides information about unicast paths, composed by one ingress router, a set of core routers and one egress router. After the flooding cycle, each ingress router implements an algorithm that takes unicast paths to generate all possible QoS-aware multicast GP trees. QoS-RRC then filters the information, keeping only best multicast GP trees for avoiding excessive matchings. Multicast GPs with ingress routers in the middle are discarded for keeping only downstream transport. Afterwards, QoS-RRC signals each path to setup multicast state accordingly.

A connectivity control mechanism is supported by QoS-RRC, allowing the aggregation of multiple GPs in the same tree, thus forming a multicast GP. This aggregation technique aims to reduce state and processing overhead inside the network (as of legacy IP multicast), allowing scalable multi-party support. Such mechanism also improves resilience, as sessions can be-routed seamlessly under network dynamic events. For instance, a served GP can be allocated into another path due to link failure of the previous path, and a session can request the allocation of the GP to the original path when the link comes up again, being such decisions under In Network Management (INM) scope, dependent of capabilities such as anomaly detection. Thus, QoS-RRC allows fast resilience support and intermittent communications.

After all over-reservations and routing table completion, ingress routers wait for session setup requests in order to start control operations. The QoS-RRC controls QoS by associating admission control and dynamic per-class bandwidth over-reservation resizing, allowing QoS-aware routing for session propagation over appropriated GPs, while keeping overall system performance. The over-reservations are resized whenever a class cannot admit a GP request. QoS-RRC resizes a class over-reservation by taking into account statistics about network conditions, resource demands and class-policies. The integration of admission control and per-class over-reservation foresees network performance improvements, where multiple GPs can be installed with minimal use of per-flow signaling.

QoS-RRC signalling behaviour was compared against a relevant per-flow solution (MIRA [67] was chosen due to its improvements against the state of the art, e.g., RSVP [68]). Thus, the over provisioning-centric QoS control strategy of QoS-RRC reduced in more than 5 times the amount of signalling exchanges, with such reduction achieved through bandwidth over-provisioning, which allows admitting multiple sessions without any signalling; therefore QoS-RRC drastically improves overall network performance over per-flow operations.

The bandwidth consumption is used as a measure to testify the influence of QoS-RRC in resource allocations for data delivery. QoS-RRC admission control prevents congestions by not allowing excessive traffic of sessions, and even under congestion periods of some classes, QoS-RRC managed residual bandwidth for sessions admission. QoS-RRC results show that it allows better bandwidth utilization than per flow analysis by avoiding waste of resources even during congestion periods or presence of residual bandwidth.

During GP creation, QoS-RRC is triggered by a GP request carrying information about source and destination, as well as QoS requirements (at least bit-rate, class of service, maximum delay, jitter and packet loss rate, for traffic characterization). Then, QoS-RRC chooses the most appropriate path among all the available ones in the routing table, accepts the request and then triggers the MP to establish the connection (setup the multicast GP). After setting all configurations, the GP factory is fed back to map the referred GP into the path provided by QoS-RRC, to further allow session propagation accordingly. In case the admission control fails in all candidate paths, QoS-RRC attempts to adapt resources of the preferred path by resizing the over-reservations accordingly. In case of no success (cannot resize over-reservations in none of the path candidates), the request is denied.

4.12 shows an exemplifying multicast scenario where both mechanisms are used for robust routing. Susan and Peter are random users (from the multicast group) who requested the same content, e.g. a live streaming

of a World Cup match. ISP1 will therefore recur to QoS-RRC for over-provisioning, aware of the large variation in the traffic matrices (due to the number of users changes and dynamic profiles).

4.2.4 Conclusions

MPR is particularly important for studying the correlation between distinct classes and routing according to such relationships. Furthermore, a robust routing setting allows an operator to accommodate variations in traffic load with a single routing setting. In addition to being beneficial in planning and dimensioning the network, a robust multipath routing is helpful for gaining insight into the properties of the routing problem and what characteristics are important in order to balance uncertain traffic demands; thus, MPR is needed for selecting the bottleneck parameters which will characterize the multicast GPs built up when using QoS-RRC.

This concludes the present section. The following section pertains to GP Resource Management.

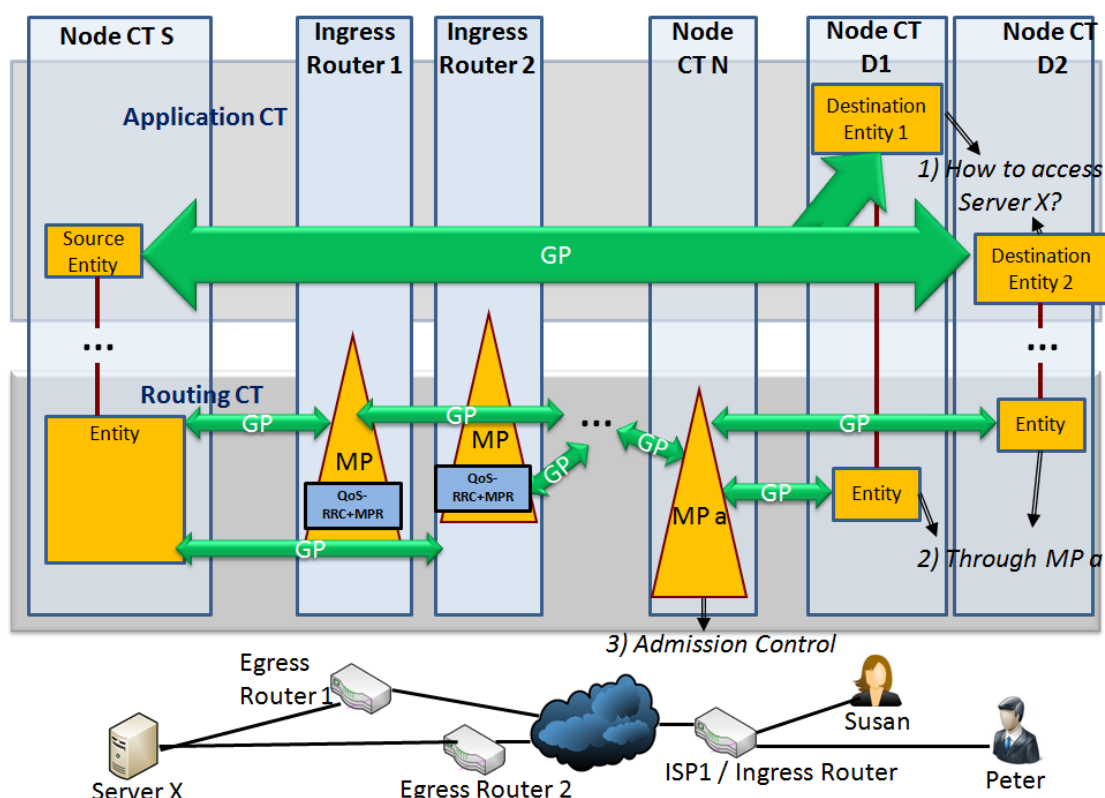


Figure 4.12: Combination of QoS-RRC and Robust Multi-Path Routing.



Document:	FP7-ICT-2007-1-216041-4WARD/D-5.3		
Date:	June 14, 2010	Security:	Public
Status:	Final	Version:	1.0

5 GP Resource Management

In this chapter we discuss how the resource management for the Generic Path is applied in various scenarios, extending the results as reported in D5.2 [2].

The first part deals with Optical Transport Network (OTN) as a use case for the Generic Path (GP) framework in a managed, path-oriented environment. We develop a generic model for optical resources and show how this can be used as a basis to understand how different optical layers work together for providing advanced transport services on user demand. Essential parts of this approach have been realized in a simulation environment, details of which are contained in Appendix D.

The second part of this chapter shows that the GP architecture framework can also be applied in a typical "connection-less", but flow-oriented environment without explicit allocation of resources. We present results of a light-weight management for shared GP resources based on a new scheduling mechanism that can be implemented in a Mediation Point (MP).

5.1 Optical Transport Network (OTN): A use case for the GP framework

5.1.1 Introduction

Core and metro networks are now massively built on optical fibre infrastructures. Optical fibre indeed provides a huge bandwidth potential that has been exploited since the early 1980's for point-to-point links in the Core networks and more recently for optically switched transparent networks. This evolution of the core optical fibre infrastructure towards a more flexible and transparent optical layer has mainly been driven by cost reason and has been enabled by efficient switching functions and to efficient transmission systems with increased reach. This evolution has brought however an important change in the optical layer conception. Historically, fibre optics links were considered by design as reliable and error free fat pipes between equipments: there was no need of any knowledge on physical impairments once the systems were installed, only the need to know the number of available channels. Now in an optically switched network, the easy availability of these fat pipes can no more be guaranteed:

- because of physical reasons: impairments have to be considered when setting up new lightpaths across the network,
- because of incompatibilities between equipments using different modulations formats or codes, making inter-working between vendors almost impossible, and
- because of specific transmission management requirements.

Physical layer awareness at large is thus becoming a requirement for today's and tomorrow's optical core and metro networks as pointed out in [69]. However physical layer awareness in today's multi-technology context is already complex (see e.g. [70], [71], [72]) and may even be more complex with the coming of new transmission technologies (coherent format, optical Orthogonal Frequency Division Multiplex (OFDM) ...) or new optical switching technologies such as optical burst switching.

In this context we have proposed a solution that will enable an operator to manage the particularities of optical technologies and their complexity thanks to an abstract description based on a new resource ontology that is based on the GP architecture framework. OTN can be layered according the multiplexing hierarchy from Time Division Multiplex (TDM) technologies (such as Synchronous Digital Hierarchy (SDH) / Synchronous Optical Network (SONET)) up to the use of entire wavebands, spanning multiple wavelengths. Through complex multiplexing structures, those transport media can be combined and switched in a flexible way, as long as certain compatibility rules are obeyed. The structures of possible switching and transport units are shown schematically in Figure 5.1.

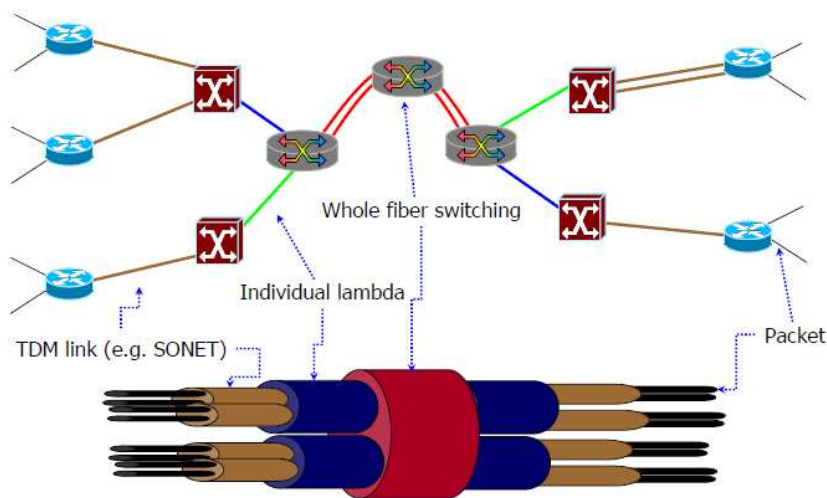


Figure 5.1: Generic switching and transport hierarchies, simplified , source: [73].

This approach to describe the configuration and capabilities of a generic optical transport network has been validated through realization of essential parts in a simulation environment (based on OMNeT++ [74]), which proved the correctness of the solution and can serve as a basis for a full integration into a GP-based network architecture. Details of the simulation framework are contained in Appendix D. In this chapter we are going to explain the underlying modelling work.

The approach comprises the following parts:

- a model of generic OTN entity objects that can be parameterized and instantiated according to their physical properties (Note of course that those objects are restricted by the (in)flexibility of the underlying hardware components) in order to play a role of any existing or future optical component, also with their vendor specific properties);
- an ontology that explains how these equipment components as well as derived "virtual" aggregates and their properties (physical parameters) are related to each other, and mechanisms how to derive and propagate these properties;
- methods that allow combinations of those entities to compound network resources during network configuration and dynamic network management, considering the compatibility rules between them;
- interfaces that allow management of GPs across these OTN resources, i.e. how to use the network resources for actual data transfer.

A comprehensive overview of related work based on the ITU-T OTN architectural framework ([71], [75]) can be found in the thesis [76]. Recent studies also introduced the notion of ontologies in the discussion of the description methodology of OTN topologies and resources ([77], [78], [79]).

5.1.2 The OTN GP Model

5.1.2.1 Resource Object Abstraction for OTN

The OTN model interprets the transport functionalities of the network as GPs set up across connected generic resource entities that play the role of physical resources.

A single generic resource entity is shown in Figure 5.2. The abstract resource object consists of a variable number of external connectors (*PortIn* and *PortOut*), and modules for connecting them internally by switching and forwarding of data, with the (optional) capability of specific control and data processing. The ports are "typed" in a way that allow (by configuration rules) only external connection of ports of the same type. To achieve this condition, internal type conversion is provided by the data processing module.

For instantiation and parameterization of those generic entities, a configuration interface is provided which will be used during the hardware setup and physical network configuration phase. Thus it will be possible to instantiate "node-like" network entities like transponders, amplifiers, switches and add/drop multiplexers as well as "connecting" elements like fibres and cables, provide them with their specific physical parameters and connect them according to the compatibility and configuration rules.

Though typically used in the role of a "node element", the design of the generic resource entity is very much inspired by the internal structure of a GP, as described in Section 1.3 of [2]. By use of this generic entity and its internal structure, it is possible to describe and explain the relation of specific entities (and the whole OTN) by means of an ontology (see 5.1.3.2).

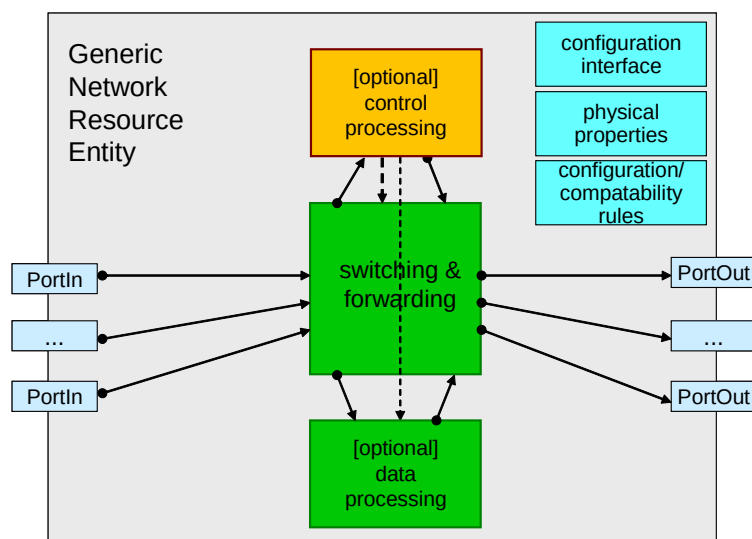


Figure 5.2: Generic Resource Entity.

The physical configuration of the network (which corresponds to the actual deployment of hardware in the field) can be seen as the setup and organization of "node compartments" in the GP architecture framework. After that, we assume that the "deployed" resource elements become active, and during a phase of dynamic self-organization the "horizontal compartment" of a specific operator-owned OTN is built up. The proposed Open Shortest Path First (OSPF)-like mechanisms will also take care of adaptations after modifications of the physical hardware basis. In real systems, this will of course need some support by management systems for principally passive elements like e.g. fibres and cables.

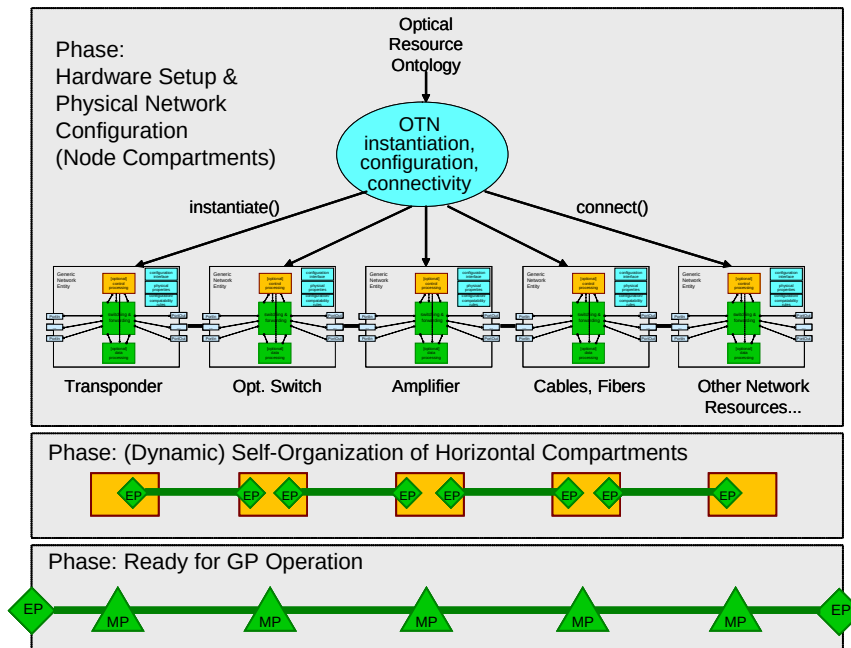


Figure 5.3: Setup, configuration and operation phases for network resources.

5.1.2.2 Network Resource Ontology

The generic representation of the network resource parameters has already been developed in [2]. This standard model provides the definition of the attributes, the associations between them and the way they are measured based on the following main classes:

- *Network Resource Parameter* representing a property of the resource within a specific compartment.
- *Metric* defining how each parameter is assigned with a Value.
- *ConversionFormula* class enabling the transformation from one metric to another.
- *Impact* representing the way the parameter value contributes to the perceived quality
- *Type* specifying the category of the ontology vocabulary, where the parameter belongs to
- *Aggregated* describing the property of aggregation.
- *Relationship* characterizing the way a Parameter is correlated with others.

Figure 5.4 shows an example of a network resource ontology model for arbitrary network resource parameters.

As an example in the case of OTN, the most relevant resource attributes can be given as:

- *type* or *capabilities*: the property that describes the hierarchy level (e.g. SDH level, wavelength, or waveband) in OTN and the characteristics of the (vendor) implementation,
- a nominal *capacity* or bandwidth figure in bit/s,
- a mean *delay* induced by this resource,
- a *congestion status* of the resource (like the Congestion and Pre-Congestion Notification ((P)CN)), as a derived parameter from offered capacity and used capacity.

For the encoding of these properties and parameters, a Generalized Multi-Protocol Label Switching (GM-PLS)-like format can be used [80].

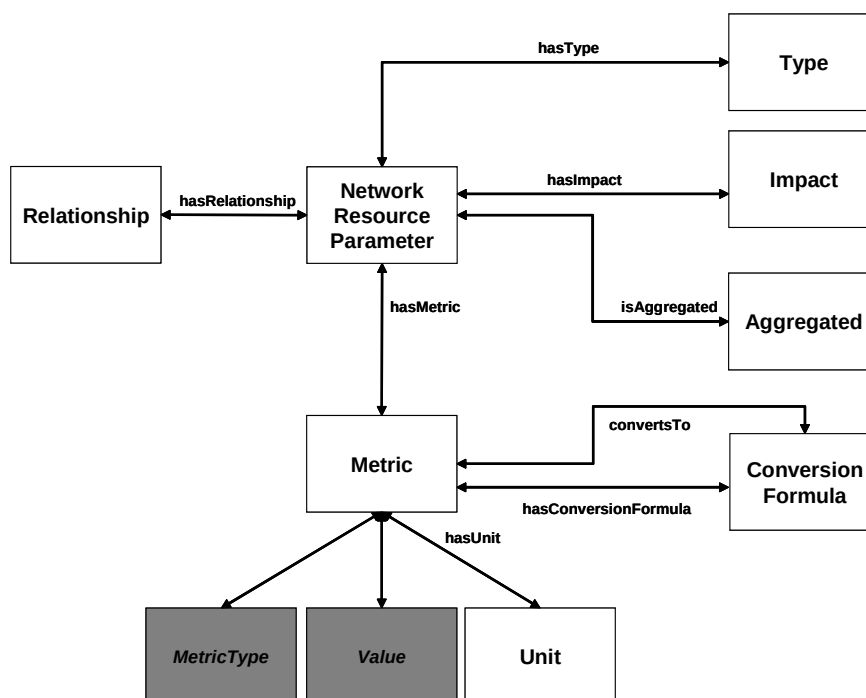


Figure 5.4: Network resource parameter ontology.

If the discovered network resources are accompanied with descriptions of their non-functional properties, automated network resource selection and aggregation is possible. Figure 5.5 sketches exemplarily a network resource ontology representing the relationship between diverse resource types from various physical layers up to Application layers.

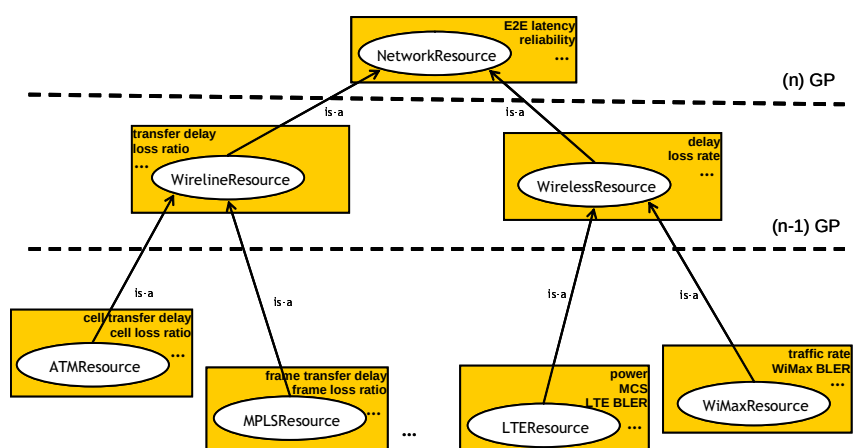


Figure 5.5: Sample of a network resource ontology.

It also shows examples for resource type associated network resource parameters, instantiated as subclasses of the network resource parameter class. The *WirelineResource* and the *WirelessResource* classes are examples for an abstract resource type providing an encapsulation for resource aggregation. Generally, the *ResourceObject* represents physical and abstract resources at a specific GP level. One possible implementation for association of the *GPObj* with the *ResourceObject* is depicted in Figure 5.6. In this example the corresponding *ResourceObject* is realized as a GP attribute at runtime.

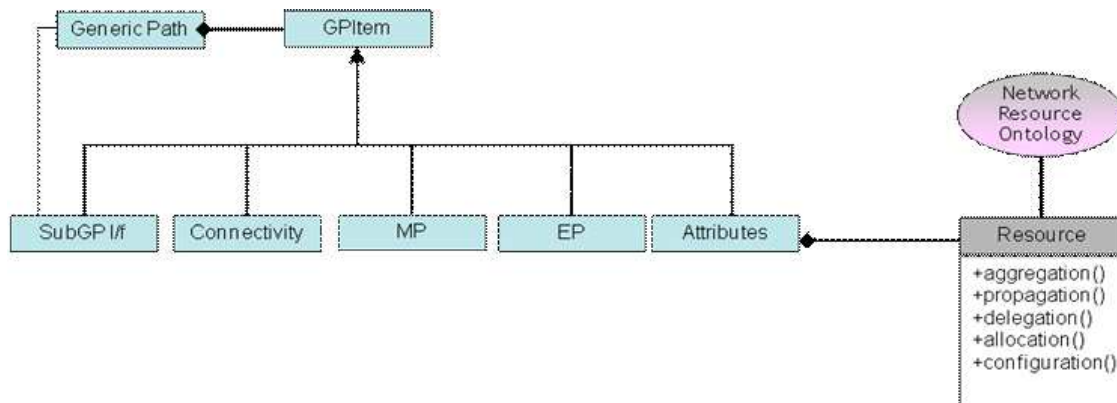


Figure 5.6: Association of *GPObj* and *ResourceObj*.

The interface of the *ResourceObject* should be available as a service, providing methods e.g. for resource requests and resource releases. Each method (operation) corresponds to a message. The message contains the information needed to invoke and perform the method. Figure 5.7 sketches how the resources are represented in a XML-based service description.

```

<UML:Class name="ResourceInterface" xmi.id="EAID_D8E35744_CF04_4200_8CC4_7A37BDD0B3I"
  <UML:ModelElement.taggedValue>
    <UML:Classifier.feature>
      <UML:Operation name="advertise" visibility="private" ownerScope="instance" is
      <UML:Operation name="aggregate" visibility="private" ownerScope="instance" is
      <UML:Operation name="delegate" visibility="private" ownerScope="instance" is
      <UML:Operation name="finalize" visibility="public" ownerScope="instance" is
      <UML:Operation name="requestResourceAllocation" visibility="public" ownerScop
      <UML:Operation name="requestResourceInfo" visibility="public" ownerScope="ir
      <UML:Operation name="requestResourceRelease" visibility="public" ownerScope=
      <UML:Operation name="ResourceInterface" visibility="public" ownerScope="inst
    </UML:Classifier.feature>
  </UML:Class>
  <UML:Comment xmi.id="EAID_FDFEACA6_98C3_4caa_8DE9_1CA2E38FE650" visibility="public"
    <UML:ModelElement.taggedValue>
      <UML:TaggedValue tag="documentation" value="Class libraries, API's and other
      <UML:TaggedValue tag="isSpecification" value="false"/>
      <UML:TaggedValue tag="ea_stype" value="Note"/>
      <UML:TaggedValue tag="ea_ntype" value="0"/>
    </UML:ModelElement.taggedValue>
  </UML:Comment>

```

Figure 5.7: XML-based resource description.

5.1.2.3 Building GPs over OTN

When applying the framework of network resource ontologies to the OTN case, there are mainly two types of compositions that are applied: serial concatenation of GPs and resources and aggregation (sort of parallel composition) in order to form compound objects.

In either case, we are able to describe how the attributes of the compound (concatenated or aggregated) object are built from the involved components. As a constraint, without "type"-changing data conversion, only compositions of the same type and capabilities are possible. The other attributes behave like indicated in Figure 5.8.

In managed networks like in the OTN case, the resource management process between requesting GPs and available network resources can be coordinated by a *ResourceObject* Manager in a node compartment. It is the mediator between the "resource view" and the "GP view" of the network.

We first describe the interface and the interaction between requesting/providing GPs and the resource manager in a 4-way handshake that consists of a "resource discovery" phase and a "resource selection"

Resource Attribute	GP concatenation	GP aggregation
capabilities, type	==	==
capacity, bandwidth	MIN	+
delay	+	MAX
PCN status	OR	AND

Figure 5.8: Attribute abstraction by composition.

phase. This mechanism is basically in line with the process the work package VNet organizes resource provisioning for virtual networks, as described in [81].

In the discovery phase, the requesting party (an "Application" entity or a GP) sends a *resource_discovery_request* to the *ResourceObjectManager*, indicating the source and destination endpoints, as well as additional minimum requirements such as available equipment capabilities and indicated bandwidth. By means of the network resource ontology, the resource manager is able to map this request to available network resources and will return in a *resource_discovery_result* a list of available options i.e. handles (ports, hooks) of underlying GPs that can provide the requested service together with their actual attribute values.

In a second handshake step, the requesting party will select the preferred resource by sending a *resource_select_request* to the resource manager, which will be confirmed by *resource_select_confirm*. By that process, all relevant information between the requesting GP and the providing GPs is exchanged, such that further data transfer can be done directly between the involved GPs possibly via a mediation point.

Concurrent to this interaction with the "requesting" entities, the resource manager interacts with the "providing" resources and GPs. Any underlying resource available will actively advertise its capabilities and resource attributes to the *ResourceManagerObject* to enable it to build a local view of available network resources. This has to be done anytime the internal state changes or a soft state need to be refreshed. In the other direction, the resource manager needs to inform the providing GPs about allocation and release of associations to requesting GPs. This is done primarily to establish and maintain local references, and need not necessarily mean a specific "reservation" of resources.

The principle of this interaction between requesting and providing GPs is illustrated in Figure 5.9.

5.1.2.4 Semantic Resource Management

After having described the local mechanism for resource management, we are going to explain how the relevant information is propagated through the network of connected GPs.

The semantic resource management supports fair resource strategies by combining best effort and strict allocation policies to a hybrid approach.

Arriving packet flows from requesting GPs are classified according to the QoS profile, resulting in a traffic resource request. This request arrives at the top-level (GP) *ResourceObject* (Figure 5.10), representing the current status of the abstracted End-to-End (E2E) resource. Based on the current route path the resource assignment starts and checks if the status of the *ResourceObject* is *UpToDate*. In this case, the process verifies whether the available resources fulfil the requirements. If the required resources are available resource assignment can be performed. Otherwise, a re-routing has to be triggered. In case re-routing does not succeed, the request has to be delayed or rejected. In this way, the GP-using Application gets immediate

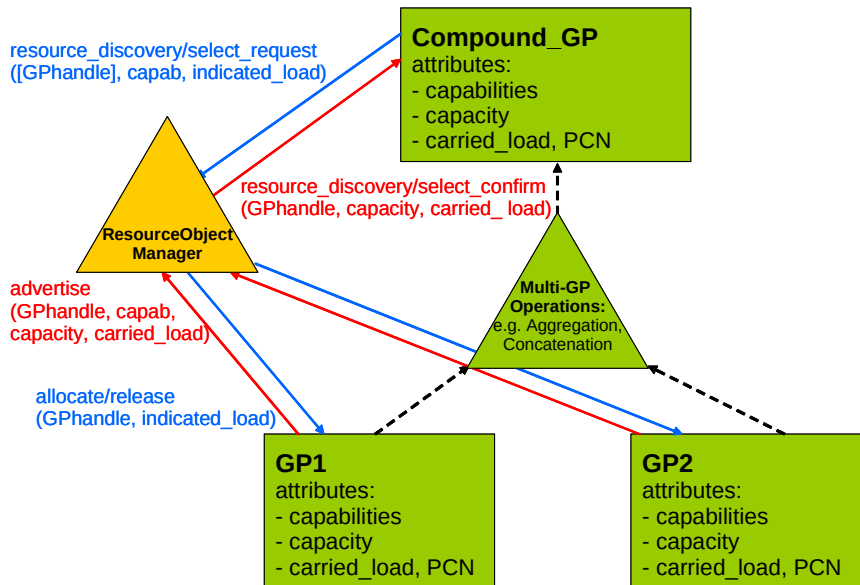


Figure 5.9: Resource management mechanisms and GP associations.

feedback to a dedicated resource request by accessing the top-level (GP) *ResourceObject*.

At the bottom-most GP level *ResourceObjects* represent real physical resources, which are located and distributed directly at the physical resource level. There technology specific resource functionalities are performed and information about the current link state, e.g. link degradation etc, are available. As the physical resource is a highly shared object, it is very beneficial to provide a pro-active behaviour at each layer by iteratively advertising its resource status to all *ResourceObjects* in the compartment (abstraction) layers above. In Figure 5.10 the resource status is periodically advertised starting from the bottom-most *ResourceObject* level to vertical and horizontal adjacencies. The advertising process can be triggered from a resource request in the case when the *ResourceObject* is not *UpToDate*.

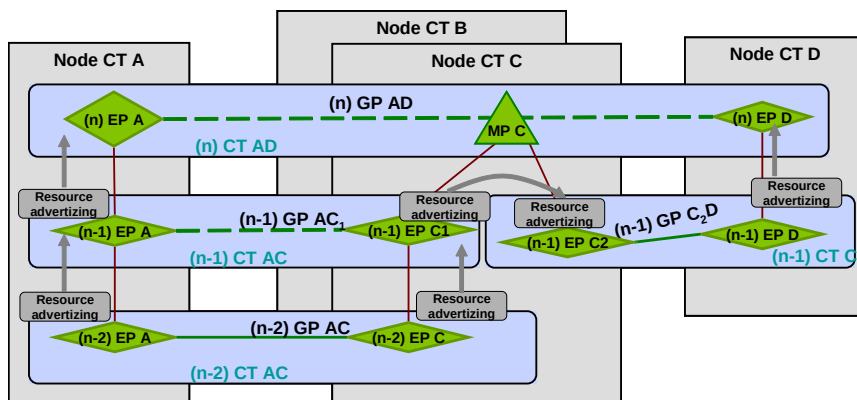


Figure 5.10: Propagation of GP resource state through advertising.

In vertical direction the advertising process leads to an aggregation (concatenation) of different and possibly heterogeneous *ResourceObjects* (level (n-1)) resulting in a more abstract representation of the *ResourceObject* (level (n)). Aggregation and abstraction are supported from the network resource ontology. In particular specific resource parameters are converted to a new metric with the help of the conversion formula depicted in Figure 5.4.

5.1.2.5 ResourceAdvertising Use Case Example

For making the use case of the advertising mechanism more clear, the metric of the resource parameter capacity on the different compartment layers is assumed to have the same type. This simplifies the use case particularly for resource aggregation and abstraction. The complete advertising mechanism will be supported by the network resource ontology by providing dedicated aggregation rules and conversion methods for different metrics. Also the delegation of the initial resource request is simplified by just delegating the request of capacity 40 just through the underlying layers without the need for mapping or conversion of generic metrics into more specific metrics on the different compartment layer as illustrated in Figure 5.11. In order to explain the propagation and advertising mechanism, we label every EP/MP entity with capital letters and the locally available connectivity edges, i.e. Hooks and Ports (both internal and external) by integer numbers.

Let *Path1* denote the path between *ApplicationClient1* and *ApplicationClient2* across the entities (A,B,D,F,I,H,G), and *Path2* across (A,B,C,F,I,H,G). *Path1* and *Path2* are partially disjoint at the link compartment level.

We assume that *ApplicationClient1* produces *flow1* with capacity demand 40 and *ApplicationClient2* produces *flow2* with capacity demand 20.

Each entity crossed by the flow path is maintaining its active *AdjacentList*. The *AdjacentList* contains the *Ids* of active Hooks or Port *Ids*, each related to a particular Endpoint or Mediation Point. *ActiveHooks* are those *Hooks*, which are affected from current GPs by crossing the EP/MP entity. The *AdjacentList* and the *activeAdjacentList* is maintained by the entity itself. As soon as an additional GP arrives at a non-active Hook, the *HookId* is added to the *activeAdjacentList* of the entity.

The use case exemplifies the transition between the current state and the corresponding *AdjacentLists*, where a *flow1* is already active, and the next state and its corresponding *AdjacentLists*, where the second *flow2* becomes established. The arrival of the second *flow2* affects the overall resource situation.

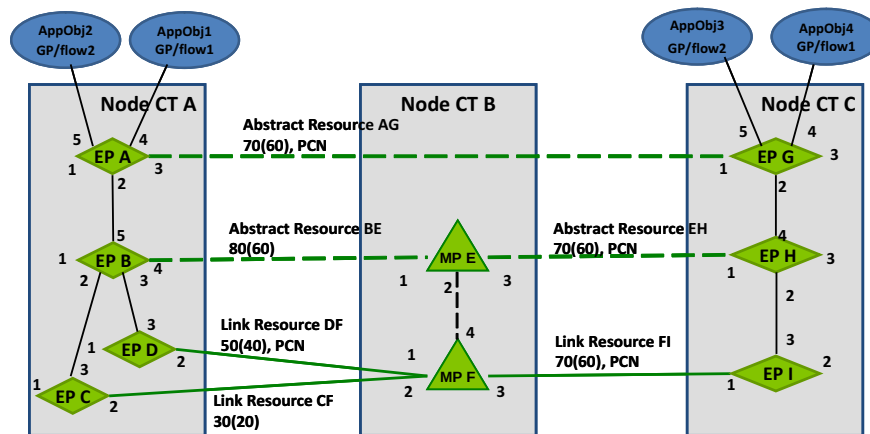


Figure 5.11: Resource Advertising Use Case.

Current State: *ApplicationClient1* has requested a resource for *flow1* (capacity 40) on route *path1* (A,B,D,F,I,H,G) by using *LinkResource DF* and *LinkResource FI* to establish connection between *ApplicationClient1* and *ApplicationClient4*.

LinkResource DF performs bookkeeping of the occupied capacity 40 out of 50. Additionally a PCN flag will be set since the capacity allocation reached 80

LinkResource FI performs bookkeeping of the occupied capacity 40 out of 70.



	current state	next state
EndPoint A:	(2,3,4)	(2,3,4,5)
EndPoint B:	(3,4,5)	(2,3,4,5)
EndPoint C:	()	(2,3)
EndPoint D:	(2,3)	(2,3)
MediationPoint E:	(1,2,3)	(1,2,3)
MediationPoint F:	(1,3,4)	(1,2,3,4)
EndPoint G:	(1,2,4)	(1,2,4,5)
EndPoint H:	(1,2,4)	(1,2,4)
EndPoint I:	(1,3)	(1,3)

Table 5.1: Hook labels in current *activeAdjacentList* and after additional resource request arrival.

The *LinkResources* encapsulate the media specific resource management and export the relevant resource attributes for further advertising. The advertising mechanism in every node is similar (identical), i.e. the entity advertises the available resources to the adjacent entities based on the entries in the maintained *AdjacentList*.

The *LinkResource DF* (50(40), PCN) is advertised to the aggregated *LinkResource BE* (80(40)). The *LinkResource FI* (70(40)) is advertised to the aggregated *LinkResource EH* (70(40)). Finally the *LinkResource BE* (80(40)) and the *LinkResource EH* (70(40)) is aggregated (concatenated) and abstracted to the E2E *LinkResource AG* (70(40)).

Next State: The state transition is initiated by an additional resource request: *ApplicationClient2* requests resources for *flow2(capacity 20)* on route *path2 (A,B,C,F,I,H,G)* by using *LinkResource CF* and *LinkResource FI* to establish a GP between *ApplicationClient2* and *ApplicationClient3*. The initiated resource request of *flow2* adds new entries to the *activeAdjacentList* of the affected EP/MPs.

LinkResource CF performs a *ResourceUpdate* of the occupied capacity 20 out of 30. *LinkResource FI* performs a *ResourceUpdate* of the occupied capacity 60 out of 70. Additionally a PCN flag will be generated since the capacity allocation exceeded 80

Then, the *LinkResource CF* (30(20)) is aggregated with the *LinkResource DF*(50(40), PCN) and advertised to the abstract *LinkResource BE* (80(60)). The *LinkResource FI* (70(60)) is advertised to the abstract *LinkResource EH* (70(60), PCN).

Finally the *LinkResource BE* (80(60)) and the *LinkResource EH* (70(60), PCN) is aggregated (concatenated) and advertised to the E2E *LinkResource AG* (70(60), PCN).

This mechanism ensures that changes in the resource states are propagated only along related GPs, using their *AdjacentList* tables, without the need of flooding the whole network or Comapartment of connected entities. The information propagated needs not to be an exact allocation status of "reserved" resources, it can also be used to distribute statistical or measurement-based performance or load indicators through the network along pathes and routes in use.

5.1.3 Validation of the Approach

5.1.3.1 Object-oriented Representation of OTN Entities

The proposed description is based on a classification of the optical resources. Final objective of the description is to provide a way to mask technology complexity behind an abstract and generic description that is

able to fully describe the resource. For example we want to mask the transmission system design complexity (the client layer does not need to know how to optimize the optical power in the system, or why the reach is limited). We also want that the description encompass present and future technologies still keeping the same standard language to describe them. For this purpose, we have proposed a first basic separation between optical elements that are part of the operator infrastructure (cables, fibres, buildings ...) and the equipments that make use of it. Inside the second category we have made a distinction between element destined to transmission purpose and elements destined to switching purpose. The global description is illustrated in figure 5.12. Each optical element correspond has attributes (names, type of equipment, location ...), parameters and relationship with other elements. For example, transmission system reach depends on transponder type and parameters and on fibre type and parameters.

The complete description of all optical elements is provided in Appendix D.

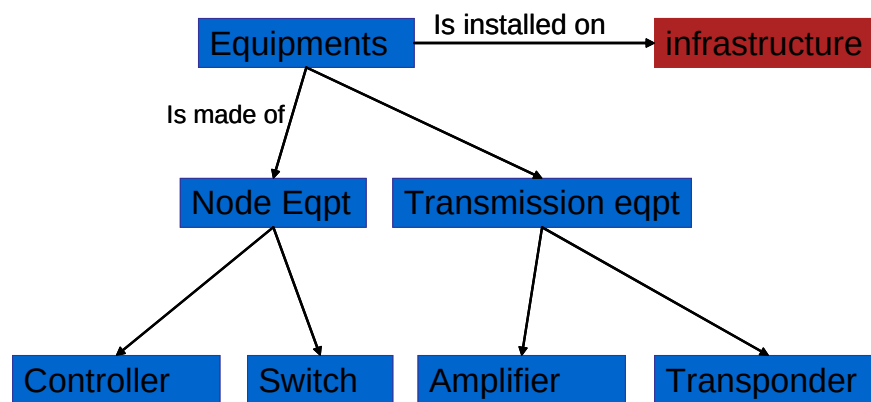


Figure 5.12: Hierarchy of Optical Entities.

5.1.3.2 Automatic Abstract Representation Builder

The real network representation. In order to validate the ontology-based resource description concept, we have developed a simulation framework based on OMNeT++. The framework consists of an optical network model composed of representations of physical elements as they exist in real networks. For the sake of simplicity of the demonstration, not all possible elements have been illustrated in the example, rather the main ones (cables and fibres for the optical infrastructure, amplifiers, optical switches and transponders, for the equipment parts). In the example, the equipment representation corresponds to real commercial solutions that could for example be recorded in the information system of the operator. The simulation first builds the optical network representation based on the developed resource ontology. In our simplified representation, each physical element is represented as the combination of 3 basic modules:

- a module that manages the elements gates and connections
- a module that manages the rules of the elements
- a module that manages its physical parameters or attributes

The three basic modules should describe as closely as possible the behaviour of the elements. For example, one rule for amplifiers consists in amplifying signal with the given working point. As another example a fibre has only one input and one output and any signal going in, exits the fibre with more or less impairments corresponding to the fibre physical parameters.

The abstract representation constructor. In order to establish an abstract representation of the network, we introduce abstract resources for the optical layer that represent real ones. These abstract resources

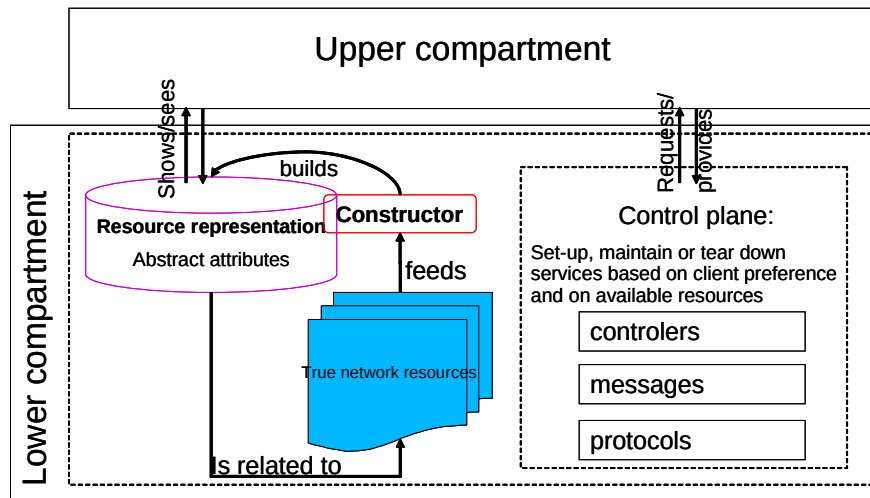


Figure 5.13: OTN Configuration.

are "Nodes", "Links" between nodes and "Channels" that act as interfaces to the upper layer. When combined, they represent the resources that would be used by a signal to be transported from one end point to another end point. In other words: they represent the resources that support a generic path. For simplification or security issues, these abstract resources have attributes that can be interpreted by other network layers and accurately represent the underlying physical layer. In our example, the most interesting attributes would be *delay*, *bandwidth* and *availability*, with *delay* being directly derived from fibres lengths, *bandwidth* from the client interface bit rate of transponders and the number of wavelength involved in the transmission, and *availability* from the combination of equipments and fibres availability and impairments (e.g. Polarization Mode Dispersion (PMD) also induces outage).

We have proposed a distributed approach for building this abstract representation: each node of the network has an "abstract representation constructor" inside its controller that will build the node's own representation thanks to exchange of respective information messages. The messages are similar to control plane messages but are not meant to replace the control plane role: they do not set-up, maintain or tear down generic paths, they only collect and propagate resource information throughout the network and interpret it into an understandable and technology free "language" (figure 5.13), which can be chosen similar to existing approaches to Resource Description Language (RDL) or Network Description Language (NDL).

In the example we have used two types of messages:

- neighbour advertisement messages
- link advertisement messages

They are similar to "Hello" and "Link State Advertisement (LSA)" messages of the Open Shortest Path First - Traffic Engineering Extensions (OSPF-TE) protocol [82], for example, with the following difference: as the message aim to represent optical resources that are free to be used, which also encompasses bypasses of nodes (provided that switches are configured so or that they *could* be configured so) the hello messages are "broadcasted" to output gates towards possible physically adjacent nodes to also advertise indirect possible adjacencies. This implementation requires that physical plane and construction message plane shall be congruent. Hello messages are no more forwarded as soon as the physical feasibility of the physical Link they advertise is no more assured (based on an estimator for example). The abstract resource representation of the network of each node is progressively updated (example figure 5.14) with the message exchanges until no more new links or nodes are discovered.

Possible end-to-end channels with associated capacity are then deduced based on the transponders present

in Links end points and on their compatibility.

The upper layer can request an abstract view of the underlying resources for its own purposes, the advantage being that it does not have to deal with technological complexity of the underlying network to make its routing or reservation decisions. The method can also provide a secure way to present resources, but still leaves the upper layer the decision.

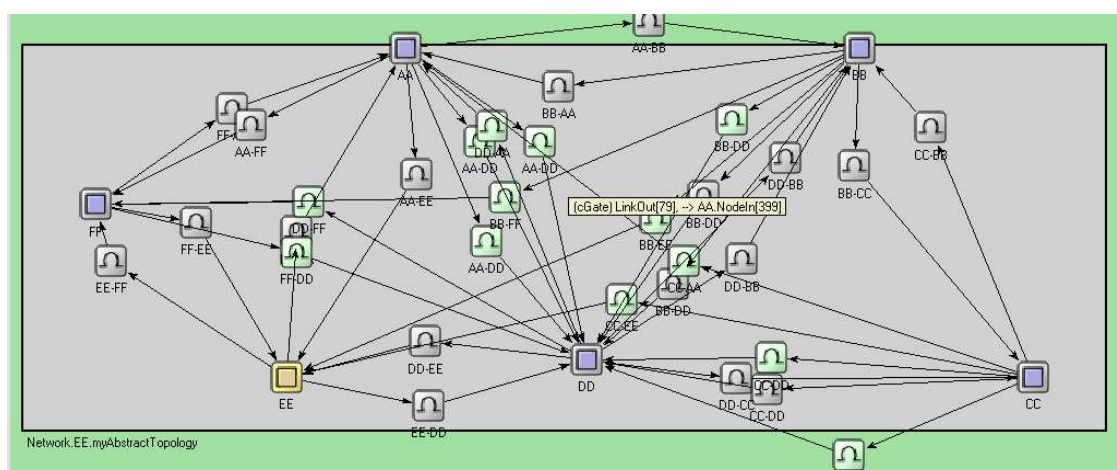


Figure 5.14: OTN Topology Detection.

The details of the proof-of-concept realization based on OMNeT++ simulation environment are provided in Appendix D.

5.2 GP light-weight resource management: Evaluation results

This section presents performance results (analysis and experimentation) for the flow-aware packet scheduler called “shortest queue first” (SQF) that aims at protecting streaming and conversational traffic from the congestion induced by data traffic. The benefit of a flow-aware approach for QoS handling has been discussed in [2] and SQF was introduced therein, as part of the GP light-weight resource management framework.

Unlike standard solutions (e.g. Diffserv as reservation-based QoS mechanism), SQF achieves implicit service differentiation, i.e., services to be handled with priority are not known in advance. It is rather the packet arrival characteristics of audio and video Applications (regulated by the source codec) that are used to detect their delay sensitivity. The SQF scheduler doesn’t require any prior configuration of the network and, as such, adapts to the fast evolution of traffic and usage. The performance of the proposed solution is demonstrated using both analysis and experiments on a testbed. We also discuss the implementation of SQF in the GP architectural framework.

5.2.1 SQF description

The SQF packet scheduler is “flow-aware”: each packet is dropped or served depending on the flow it belongs to. We refer to a *flow* as the set of packets that belong to the same instance of Application (e.g. a VoIP call, a gaming session, a file upload). In the current context of IP networks, a flow is identified either through the 5-tuple of the IP header, or through some pre-defined subset. The service discipline consists in serving the packet of the shortest queue (A logic queue per active flow) along with dropping packets when necessary from the flow with the longest queue. A flow is said to be *active* if it has at least one packet in



the buffer. This discipline ensures low latency to packets of streaming and conversational Applications as they typically exhibit a regular packet arrival pattern that contrasts with that of bursty Applications (e.g. file transfers). Thanks to the Longest Queue Drop (LQD) buffer management policy, regulated flows (with little backlog) suffer no packet loss. The total buffer is simply sized so as to ensure an efficient utilization by TCP-like congestion control protocols; typical values range from 50 to 200 MTU sized packets, see [83].

Compared to the well known per-flow Fair Queuing (FQ), the applicability of SQF is not limited to metro/core networks where the peak rate of audio and video flows represent a small fraction of the link capacity. Unlike FQ, SQF is *not* based on the throughput of active flows but only on the number of packets they have in the buffer. In particular, SQF is able to prioritize audio and video traffic of high rates, close to the link capacity. The efficiency of SQF was shown in [84] for CBR traffic, using both analysis and simulation, and demonstrated in [85] on the Livebox, an ADSL router available to customers of Orange's Broadband services¹. We present in the sequel both analytical and experimental results comparing the performance of SQF and FQ in the more realistic context of VBR traffic.

5.2.2 Analysis

We assess the robustness of SQF to the rate fluctuations of one or several VBR video flows in the presence of data traffic.

A single video stream. We first consider a single video stream that competes for bandwidth with n TCP flows. We assume that these TCP flows saturate the buffer and have approximately the same number of packets in it. For simplicity, we also assume that all video packets have the same size, taken as the data unit.

Let x be the number of video packets in the buffer. Denoting by B the buffer size, the volume available for each TCP flow is given by:

$$\frac{B - x}{n}.$$

The video stream has priority as long as it has a lower data volume in the buffer than each TCP flow, that is if:

$$x < \frac{B}{n + 1}. \quad (5.1)$$

Let C be the link capacity in packet/s. If the video stream were purely CBR at rate $\lambda < C$, there would be at most one packet in the buffer and the video stream would always be prioritized. More generally, if the video stream is VBR but has a strictly enforced peak rate $\lambda < C$, there is at most one packet in the buffer and the video stream is always prioritized. Now assume that the peak rate λ is not strictly enforced. Specifically, we assume that packets arrive according to a Poisson process of intensity λ . Thus, while the *theoretical* peak rate is equal to λ , the *actual* peak rate is infinite (there is no lower bound on the packet inter-arrival time).

Note that this traffic model with transmission at the theoretical peak rate λ is in fact much more pessimistic than any more realistic, long-range dependent traffic model that accounts for the rate fluctuations between 0 and λ due to the structure of the video [86].

We seek to estimate the probability $p(T)$ that the video is deprioritized, in the sense that condition (5.1) is violated, over some time period of length T . In view of the above assumptions, this reduces to the analysis of the transient behaviour of an $M/D/1$ queue.

¹http://en.wikipedia.org/wiki/Orange_Livebox

In order to get simple and explicit results, we first calculate the probability $q(t)$ that the number of arrivals exceeds the service capacity plus the buffering capacity (5.1) over any time interval of length t :

$$q(t) = \Pr \left(N(t) \geq Ct + \frac{B}{n+1} \right)$$

where $N(t)$ denotes a Poisson random variable of mean λt . We then divide the time period T into k intervals of length T/k and calculate the probability q_k that the number of arrivals exceeds the service capacity plus the buffering capacity (5.1) on at least one of these intervals:

$$q_k = 1 - \left(1 - q\left(\frac{T}{k}\right) \right)^k.$$

The probability of depriorization $p(T)$ is then estimated as the maximum of the probabilities q_k over all k :

$$p(T) = \max_{k \geq 1} q_k. \quad (5.2)$$

Note that the maximum is reached for some finite k since q_k tends to 0 as k grows to infinity.

Figure 5.15 gives the probability of depriorization as a function of the video peak rate λ for a 10-min video on a 10Mbit/s link with different numbers n of competing TCP flows. The buffer size is equal to 200 packets, with a packet size of 1250B. For $n = 20$ competing flows for instance, the probability of depriorization is close to 0 as long as the video peak rate is less than 6 Mbit/s.

The figure also shows the probability of depriorization under FQ, which is equal to 0 if $\lambda < C/(n+1)$ (since the video peak rate is less than the fair share) and to 1 otherwise. For $n = 20$ competing flows for instance, the video peak rate cannot exceed 480 Kbit/s.

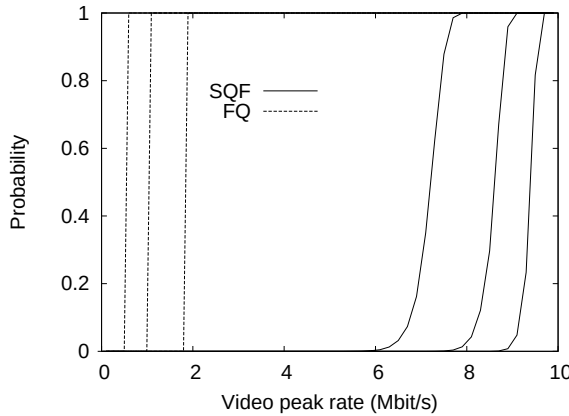


Figure 5.15: Probability of depriorization for a 10 min video on a 10Mbit/s link with $n = 5, 10, 20$ competing TCP flows (from right to left).

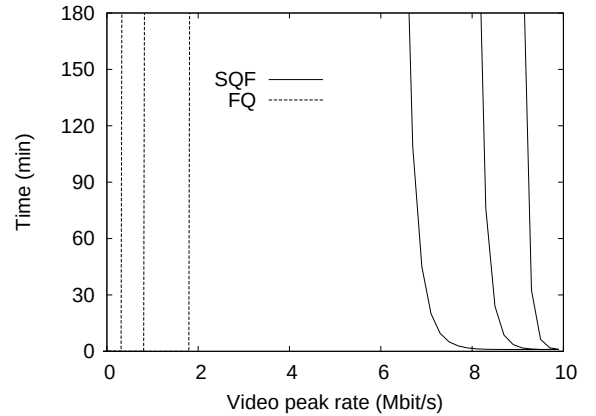


Figure 5.16: Average time of prioritization of a video on a 10Mbit/s link with $n = 5, 10, 20$ competing TCP flows (from right to left).

Another way of evaluating the performance of SQF is to estimate the average time T during which the video is prioritized. Note that the probability that the time of prioritization is larger than t is equal to $1 - p(t)$, where $p(t)$ is given by (5.2). We deduce:

$$T = \int_0^\infty (1 - p(t)) dt.$$

Figure 5.16 gives the average time of prioritization T as a function of the video peak rate λ on a 10Mbit/s link with different numbers n of competing TCP flows. For $n = 20$ competing flows for instance, this average time is larger than 180 min as long as the video peak rate is less than 6 Mbit/s. Under FQ, the video peak rate cannot exceed 480 Kbit/s in this case.

Multiple video streams. The results readily extend to an arbitrary number m of video streams. Let x_i be the number of packets of video stream i in the buffer. Let $x = \sum_{i=1}^m x_i$. All video streams have priority as long as each has a lower data volume in the buffer than each TCP flow, that is if:

$$\forall i, \quad x_i < \frac{B - x}{n}. \quad (5.3)$$

Clearly, the condition (5.1) imposed on the total number of video packets is sufficient. This is equivalent to consider the m video streams of rate λ as a single video stream of rate $m\lambda$. Thus for a 10 Mbit/s link with $n = 20$ competing TCP flows, SQF gives priority to $m = 3$ video streams of peak rate 2 Mbit/s (since the total video stream does not exceed 6 Mbit/s).

5.2.3 Experiments

This section presents some experimental results on the performance of FIFO, FQ and SQF.

Test-bed setup. The testbed consists of four Linux PCs, that emulate a router, two servers and a client, as depicted by Fig.5.17. One server is an Apache Web server [87], the other one is a video streaming server using VLC [88]. The Linux router is equipped with three network Ethernet cards so as to connect the two servers and the client in a star topology, using standard Linux tools [89]. All line cards are used at 100 Mbit/s except for the link of the client that is used at 10Mbit/s and plays the role of the user's access line.

The corresponding output line card of the router has a buffer size of 50 MTU sized packets (with a MTU of 1500B) and embeds one egress queueing discipline, chosen among FIFO (without buffer management), FQ (Deficit Round Robin algorithm with Longest Queue Drop), and SQF (as described in Sec. 5.2.1). Flows are identified through the 5-tuple of the IP header.

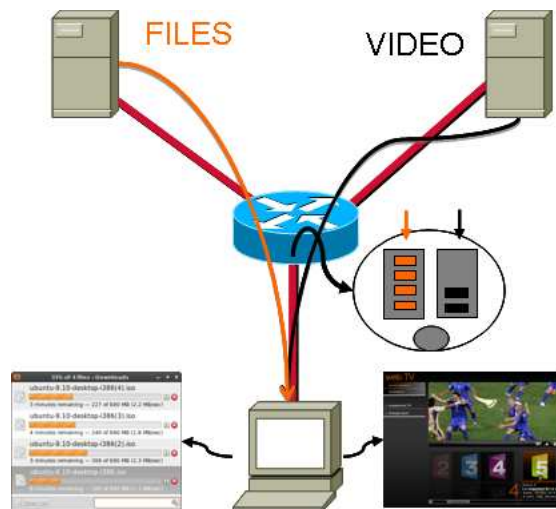


Figure 5.17: Testbed set-up made of two servers and one client connected in a star topology.

Traffic scenario. The client downloads several files from the Web server while watching a video channel streamed on RTP over UDP by the VLC server. The video is composed of two flows, one for the video and one for the audio, on different UDP ports. The codec is MPEG2 for both video and audio. The video stream is a VBR flow with an average rate of 5Mbit/s while the audio stream is a VBR flow with an average rate of 64kbit/s.

Quality of the video stream. The quality of the video stream is estimated through the loss rate and jitter experienced by packets of the audio and video flows on a 10-min sequence. These performance metrics are calculated from tcpdump records [90], available at the client input interface. The loss rate is deduced from the stream sequence numbers while jitter is estimated through the inter-packet delay variation, as defined in [91].

Figure 5.18 shows the packet loss rate of the audio and video flows with respect to the number n of competing file downloads, varying from 1 to 20. We observe that the loss rate is negligible under SQF independently of n , as expected in view of the analysis of Sec. 5.2.2, whereas it is much too high under FIFO and FQ as soon as $n > 1$. Under FIFO, the packet loss rate of the audio flow grows linearly from 1% to more than 25% as n grows from 1 to 20; it is slightly less for the video flow. Under FQ, the behaviours of the audio and video flows differ significantly: the audio flow has an average rate of 64 kbit/s that is less than the fair share, which explains the resulting low loss rate; the video flow, on the other hand, has an average rate of 5Mbit/s that is larger than the fair share as soon as $n > 1$, which explains the resulting high loss rate. These results confirm the poor performance of FQ under high data traffic load.

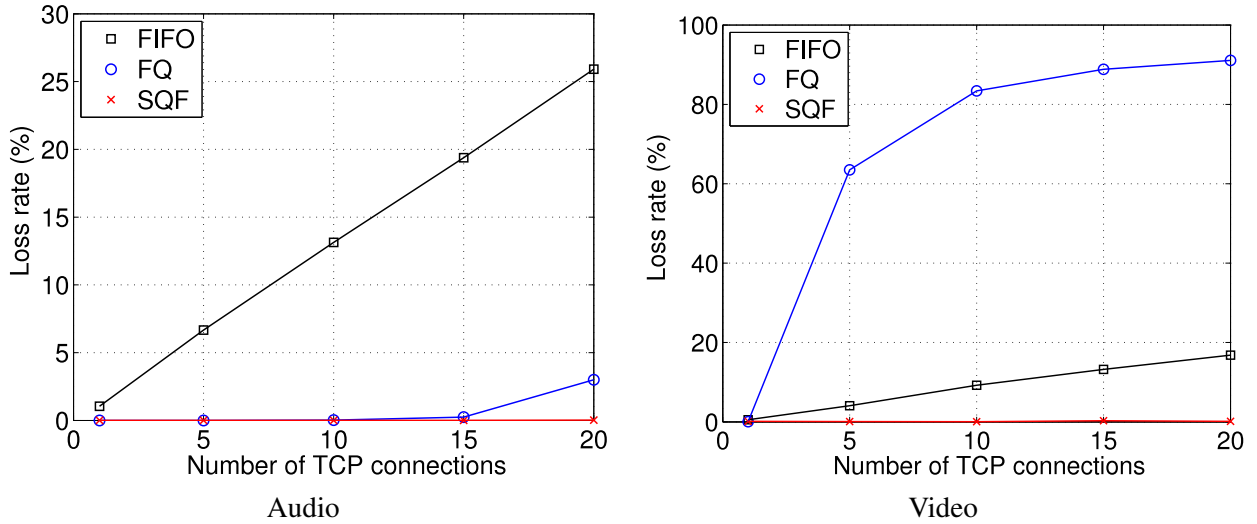


Figure 5.18: Packet loss rate of the audio and video flows with respect to data traffic load.

Results relative to jitter (not included) show that under FQ, the jitter increases with the number of flows for both the audio flow and the video flow, due to the round robin scheduling time. SQF turns out to be beneficial for both audio and video, independently of the number of competing file downloads.

Data traffic. In Tab.5.2 we have reported some performance measures for data traffic. The data bandwidth share corresponds to the total throughput of data traffic while Jain's index indicates how this bandwidth is distributed among the n data transfers. Specifically, Jain's index is defined by:

$$J = \frac{\sum_{i=1}^n y_i}{n \sum_{i=1}^n y_i^2}$$

where y_i denotes the normalized throughput achieved by the i -th data download at the end of the experiment. Note that J varies from $1/n$ for a totally unfair sharing (a single flow gets all the resource) to 1 for a perfectly fair sharing (all throughputs are equal).

As expected, the data bandwidth share increases with n under both FIFO and FQ whereas it remains constant under SQF, due to the self-priorization of the video stream. Under FQ, the bandwidth available for the video stream is not sufficient as soon as $n > 1$, which is in agreement with the above mentioned results



n	1	5	10	15	20
FIFO	4.1/0.95	4.4/0.91	4.8/0.95	5.1/0.95	5.6/0.99
FQ	4.1/0.99	7.5/0.99	8.5/0.95	8.9/0.99	9.1/0.99
SQF	4.1/0.95	4.1/0.90	4.1/0.91	4.1/0.95	4.1/0.95

Table 5.2: Data bandwidth share (in Mbit/s) and Jain’s index fairness as a function of the number n of data downloads.

of . Note that in all cases, the overall utilization does not reach 100% due to the transmission of duplicate packets by TCP. Under SQF for instance, the data bandwidth share is equal to 4.1Mbit/s so that the overall utilization, including the 5Mbit/s video flow and the 64kbit/s audio flow, is approximately equal to 92%. Finally, fairness as measured by Jain’s index is excellent for all schedulers.

5.2.4 SQF implementation in the context of the GP framework

GP identification. The main issue regarding SQF’s integration into the GP framework concerns flow/GP identification. SQF assumes it is possible to identify a flow (e.g. a voice conversation, a video streaming) or a flow bundle (e.g. at the subscriber level) to which a packet belongs on the fly, by inspection of header fields. This identification should be facilitated by the GP architecture thanks to the concept of *GP identifier*. On-the-fly identification based on protocol header attributes (e.g. TCP/IP in the context of present networks) exhibits several limitations, namely the need for router to inspect TCP/IP headers, possibly encrypted, and the fact that an Application instance may correspond to a bundle of TCP connexions. A more satisfactory solution (though not implemented) consists in combining the IPv6 flow label with either the source address and/or the destination address.

In the context of the GP architecture, a flow would be mapped to a GP. As a consequence, flow identification boils down to defining the GP identifier. Similarly to what have been suggested in the context of IPv6, we propose that GPs be identified using a *GP label* and a *set of flags* indicating which other packet header attributes should be considered for GP identification purpose. The use of flags allows for a more flexible GP identification approach, which is necessary given the tight coupling that may exist between GPs and Applications. The way a GP is identified depends notably on the underlying connectivity paradigm (i.e., point-to-point, multipoint to point or point-to-multipoint). An example of GP identification in the context of a multi-point to point connection is exemplified in the sequel.

The SQF scheduler in the GP architecture. SQF, as a scheduling mechanism, naturally belongs to the data plane. The SQF mechanism being GP-aware, requires a packet classification on a per GP basis. Both the scheduler and the GP identification module could be part of the forwarding functional bloc of a Mediation Point. SQF schedules packets of GPs that do not terminate at the Mediation Point.

Example. The example, described hereafter, details GP identification in the context of a *multipoint-to-point connectivity*. Consider the case of a Mediation Point that merges two or more GPs pertaining to the same P2P content transfer on the downstream path towards the end user into a single GP (cf. Figure5.19). The GP associated to this P2P file download could be simply identified using the GP label along with the destination address of the end user requesting the content. In this example, the incoming sub-GPs correspond to one or more chunks of the requested file sent by a given peer. All sub-GPs involved in this P2P file download will have a common GP label, originally set by the requester and inherited from his request for the file of interest. As a result, at any Mediation Point along the downstream path, SQF schedules packets relative to this P2P file download as belonging to one same GP. A logic queue is instantiated per *active GP*,

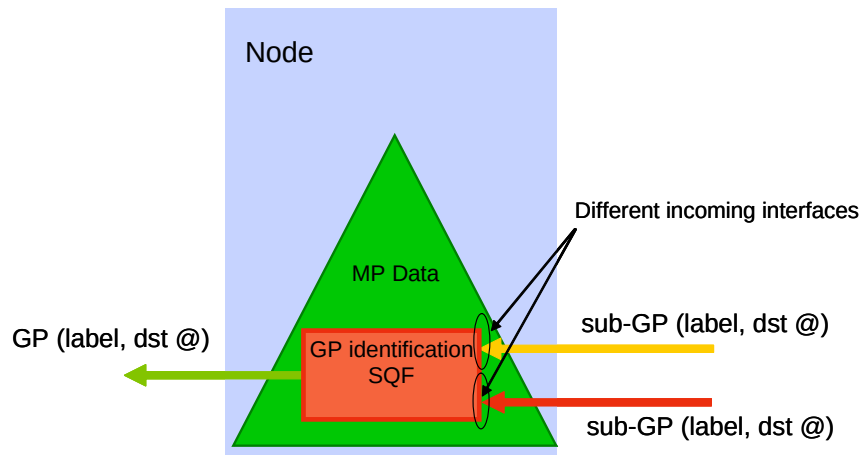


Figure 5.19: GP identification and GP-aware scheduling in a Mediation Point.

i.e., a GP that have at least one packet in the mediation point buffer. Note that SQF, like FQ, does not raise a scalability issue regarding the number of active GP to maintain, which will typically be in the order of hundreds and, independently of the link capacity [92].



Document:	FP7-ICT-2007-1-216041-4WARD/D-5.3		
Date:	June 14, 2010	Security:	Public
Status:	Final	Version:	1.0

6 WMN: A design exercise for the GP Model

6.1 Wireless Mesh Networks, the wireless Internet paradigm

We consider a research problem of engineering a mechanism for optimal management of radio resources in multi-radio Wireless Mesh Network (WMN), aiming to reveal several limitations of the current Internet architecture and highlight the engineering innovations and usability of the GP model, for introducing new functionality in existing networks.

WMN lay on the evolutionary path of wireless networks, by extending the single-hop access wireless paradigm to multi-hop ad-hoc (backhaul) networks that combine heterogeneous radio access technologies. Mesh Access Point (MAP) offer wireless infrastructure connectivity to end-users and form among themselves a self-organised multi-hop wireless backhaul network that provides gateway access to the Internet, as illustrated in Figure 6.1a). MAP are resourceful and have reduced mobility, and thus able to support a robust topology. However, before WMN solutions become the architectural extension of the wireless Internet, there are a number of obstacles that must be overcome.

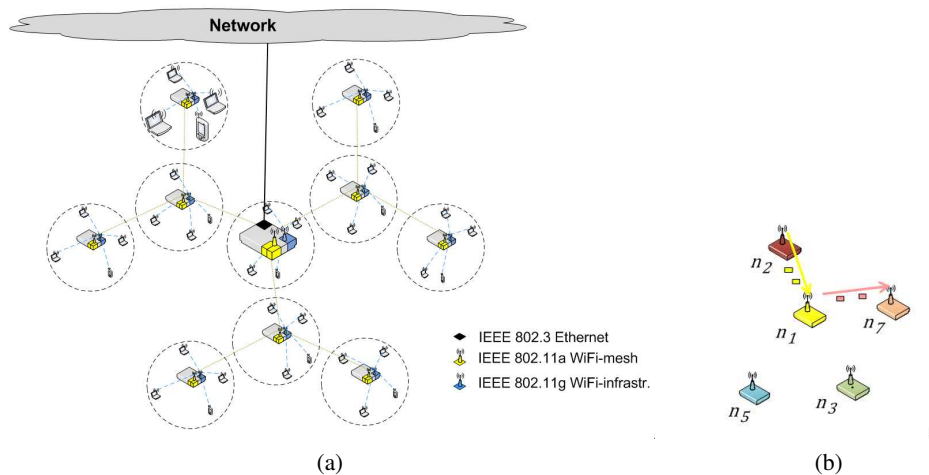


Figure 6.1: (a) Heterogeneous multi-radio WMN topology. (b) Mesh node operation with 2 radios of 1 wireless system

The conventional layered-protocol Internet architecture does not allow optimal performance, presenting several motivations for cross-layer design in WMNs [9] in order to address a number of challenging problems such as heterogeneous QoS constraints, multi-hop wireless communications, and variable link capacity. The underlying incentive is that to integrate effectively the diverse wireless network technologies (IEEE 802.11 [93], 802.16 [94], 802.15 [95], Long Term Evolution (LTE) [96]) and radio communication solutions (multi-radio/multi-channel nodes, directional antennas, etc), there is strong need for tight coupling and cooperative optimisation between the link, routing and transport layers.

Another key challenge in WMN regards the distributed management and coordination of wireless and other resources, which will allow the network to self-configure (“plug-and-play” fashion), organise and op-



timise its servicing capacity, and incorporate self-healing capabilities in case of failures. In this direction, one innovative framework is presented in [97], which allows the network to take advantage of the local resources and characteristics of nodes (communication capabilities, surrounding environment, mobility pattern, persistence, energy, computation capacity) in an opportunistic fashion. Any node may assume any network role in the mesh (resource management, network gateway, forwarding, address assignment), thus contributing in the collective optimisation of the network.

The general conclusion from an analysis of the state-of-the-art on resource management in WMN is that no single prevalent management scheme can be employed. Instead, multiple resource management schemes can be derived depending on the way the complex task of joint optimisation of aspects such as channel allocation, power control, transmission rate, load balancing and routing, is decomposed. This motivates the research undertaken in selected topics such as channel assignment, routing, power allocation and scheduling.

6.2 Research scope: Wireless connectivity management

A typical example of the need for distributed resource management in WMN regards the effective and integrated management of interfering radio resources in order to improve mesh node communication in a dynamic network topology and the exploitation of multi-path capabilities for more efficient use of the network. The use of a shared broadcast communication medium, presents an important trade-off between maximising connectivity and minimising interference. A fixed wireless channel assignment among mesh nodes results in enforced topologies, analogous to the wired ones, however, often with variable quality or episodic connectivity (due to the susceptibility of the medium to noise and interference). Hybrid and dynamic solutions (spontaneous and self-organised WMNs), on the other hand, increase inter-connectivity at the cost of increased management complexity since they face special challenges, when compared to networks statically deployed and controlled by an administrated entity. The heterogeneity of nodes in such networks should be fully exploited in order to increase, as much as possible, network availability and usability.

A promising path lies in the use of mesh nodes, equipped with several radio interfaces that may work simultaneously on multiple radio channels and a channel management strategy that effectively assigns channels/carriers to radio interfaces, in order to maximise channel utilisation and minimise interference. In the proposed framework, Inspired by the Hybrid Multi-Channel protocol [98], every mesh node is equipped with N_R channel interfaces (so called *radios*), enabling it to operate simultaneously on N_R different carrier channels. These are divided in two groups: (a) N_{St} ; the set of *stable* radios, each identifying a persistent channel during a certain period of time; and (b) N_{Dy} ; the set of *dynamic* radios, which are dynamically switchable between any of the remaining available carrier channels. A node advertises its N_{St} stable channel(s) as its receiving channel(s). For example in Figure 6.1b, if node n_2 wants to forward data to neighbouring node n_1 , the dynamic radio of n_2 is switched to the stable channel of n_1 . At the same time, node n_1 can use its dynamic radio to switch to the stable channel of n_7 , or any other neighbour node, and forward packets to them, respectively. This addresses the trade-off between maximum connectivity (the dynamic radio that enables communication with any neighbouring node) and minimum interference (strategic allocation of stable-channels).

A set of data structures leverage the operation of the algorithm:

- the *RoutingTable* indicates the next neighbour node to which each packet must be forwarded,
- the *NeighbourTable* holds the receiving channel of each neighbouring node,
- the *ChannelQueuesList* associates packet queues to entries of the *NeighbourTable*, and
- the *ChannelUsageTable* holds channel quality information about an extended neighbourhood of nodes for the periodic decision of re-assignment of stable channels.

Information from the *ChannelUsageTable* is periodically transmitted to the nodes in the extended neigh-



bourhood allowing them to update their *ChannelUsageTable* and *NeighbourTable* and decide locally the allocation of stable channels. The allocation of stable channels takes into account the channel usage, and local estimates of received signal strength and interference.

A full presentation on the algorithms for the allocation of stable channels and the switching between the dynamic radios is given in [15]. It should suffice to mention that computation of metrics for channel selection relies on an interference model that groups interacting wireless nodes in clusters and abstracts their dynamics in virtual nodes competing over the wireless medium. The scheme is designed to provide stable interconnection, avoidance of a multi-channel hidden terminal problem and protection of ongoing transmissions from interference, by means of technology-agnostic *logical channel interfaces*, which abstract into equivalence classes the per-technology available channel sets (that compete for the same radio resources)¹.

6.3 Object-oriented engineering in the GP model

In order to exploit multiple wireless network interfaces simultaneously, we eventually have to address the heterogeneity of current and emerging wireless technologies. Radio resources must be partitioned in the physical dimensions of time, frequency, space and/or code in a way that allows multiple logically independent transfers to take place simultaneously. On the other hand, the WMN – when seen as a black box by its users – must meet requirements for flexibility, fairness, scalability and measurability. This raises the challenge of orchestration and exchange of internal information, which in the current Internet model, needs to overcome a number of cross-layer constraints.

Using the GP model and object-oriented design foundations, we aim to tackle the following engineering challenges:

- Isolate and modularise the management of individual wireless radio technologies and resources through the process of *compartmentisation*.
- Abstract and virtualize access to different wireless radio technologies and resources through generic and polymorphic interfaces that allow coherent and yet dynamic coupling of different resources without compromising modularity.
- Develop orthogonal resource management schemes (encapsulated in different *GP classes*) depending on the way the complex task of joint optimisation of components (such as channel allocation, power control, transmission rate, load balancing and routing) is decomposed.
- Facilitate MP control interfaces at identified policy enforcement points of the mesh topology to enable access to a number of operational tasks, such as: adjusting granularity of routing paths according to mesh network policy; congestion control feedback between internal traffic handling functions and the functions in use on the mesh client side; admission control for load balancing of client flows in the mesh network, and other.

6.3.1 Wireless resource abstraction

Addressing the first two challenges and overcoming the barrier of technological heterogeneity will enable effective pooling and generic distribution of wireless radio resources. The separation of functionality for the different link-connectivity (Wireless Link Layer Control (W-LLC)) wireless radio technologies can be provided in different compartments and implemented in respective GP classes. Within each wireless technology oriented compartment, a W-LLC GP class encodes the technology-specific attributes and provides generic

¹Conceptually, this is close to a transposition of Redundant Array of Inexpensive Disks (RAID) disk technologies to network resources.

access to the management of respective wireless radio interface. Objects of each class thus correspond to the wireless links (services) established using this wireless technology.

While W-LLC classes provide access to technology oriented management functionality, a second GP class type provides generic technology independent access to a logical radio channel link resource, allowing Radio Medium Access Control (R-MAC). Temporal multiplexing of R-MAC class objects (that correspond to logical channel interfaces) over a W-LLC class object (physical wireless radio interface) allows virtualisation of a wireless radio interface, to provide multiple single-channel interfaces. Each W-LLC class is a singleton that R-MAC objects can control by means of software radio control and using internal multiplexing strategies. At the same time, R-MAC class objects provide a generic channel-oriented interface abstraction across wireless technology compartments, allowing generic management and the development of technology-independent operations and services (routing, network coding, etc).

Figure 6.2 schematically exemplifies the attribute list exposed by a technology-specific alliance to support IEEE 802.11 WLAN standards family (WiFi) GP class and the subset of attributes exposed in the generic R-MAC interface.

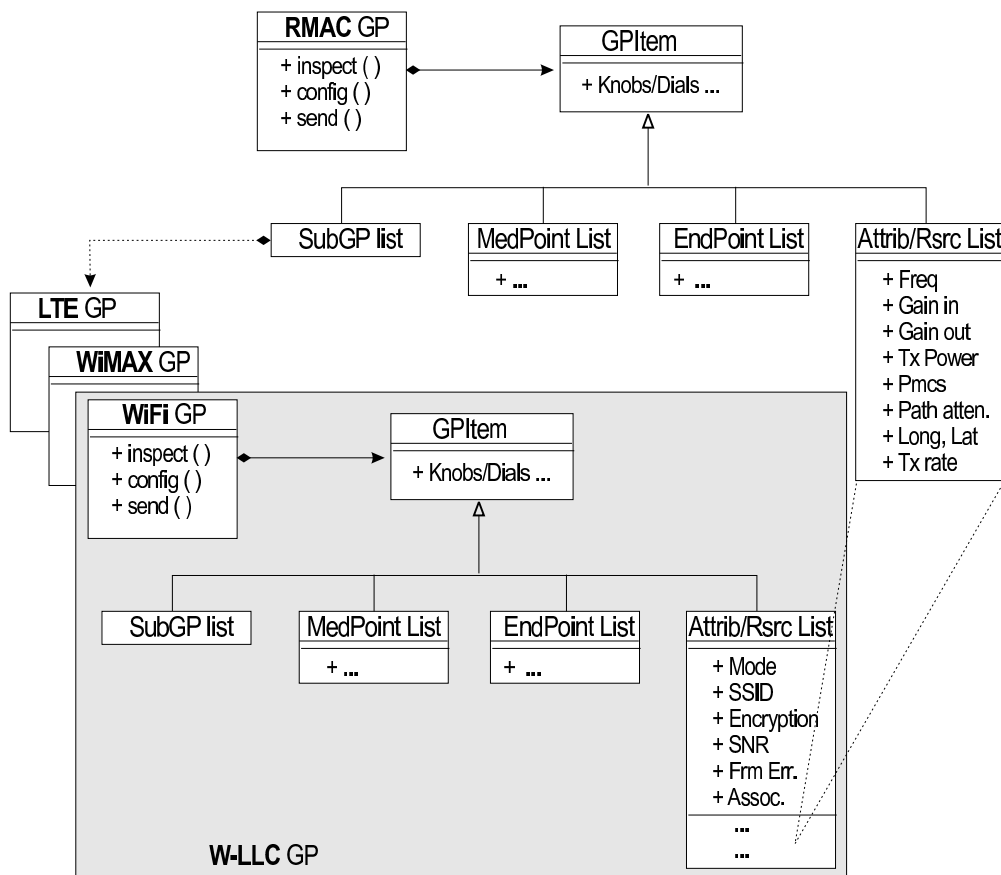


Figure 6.2: A R-MAC object providing virtual access to multiple W-LLC objects

6.3.2 Channel assignment strategy

Having isolated and localised technology-specific aspects in wireless technology compartments and abstracted them as single-channel resources (R-MAC objects) that hide different radio interfaces, we proceed to provide technology agnostic management mechanisms (for addressing the 3rd challenge). Although we concentrate on one channel allocation strategy for the establishment of temporally dynamic WMN topolo-



gies, other such technology agnostic mechanisms may include routing schemes, coding frameworks, and other services.

Every mesh node participating in the WMN topology is identifiable in the context of a WMN service compartment, and is responsible for maintaining up-to-date information about other WMN service nodes in its neighbourhood. This includes the distributed data structures presented earlier (routing and neighbour tables, channel queue lists and channel usage table) as well as information about the node locality and physical environment (e.g. geo-position, cluster size and conflict sets, power decay and shadowing, gaussian noise, and other).

The active topology is established through a set of instantiated WMN GP class service objects that hold the dynamic interconnection state in the cloud. Such dynamic information includes estimates of the average channel activity and utilisation/contention, queue associations, received power/noise estimates from competing wireless clusters, hop-distances to gateways and respective path attenuation, queue occupancy, and other. Most of this information being estimated relies heavily on information acquired by the the wireless channel interface, (i.e. R-MAC objects). By contrast to the current Internet layer model whereby this information would need to be accessed directly at the physical interface through ad-hoc cross-layer APIs, in the GP design it is collected from the W-LLC objects, abstracted and exposed by the R-MAC objects and used by the WMN objects (and always using the same generic interface).

6.4 Discussion

In the factorisation of functionality across different compartments and GP classes, so far, there are a number of interesting aspect revealed regarding the effectiveness the GP model.

Within a wireless technology compartment, an R-MAC object and the respective encapsulated W-LLC object provide access to the same wireless link service, enabling joint customisation. This compositional arrangement in two objects emphasizes on the separation of functional and operational concerns, and allows polymorphic management of a wireless link service. On one hand, the W-LLC object provides exclusive technology-oriented management of the entire wireless radio interface and through it to the link service it provides. On the other hand the R-MAC objects permit multiplexing of multiple single radio channel interfaces over a wireless radio interface (permitting its customisation on a per-channel basis) and the temporal virtualisation any radio channel interface across different wireless radio interfaces. Such a level of interaction and polymorphic access is not easily (if at all) attainable across layers in the current Internet model.

Data transportation is controlled at the WMN service class level which dynamically associates packet queues to R-MAC channel interface objects. The actual data transportation primitive is however, provided by the per wireless technology protocol implementation which is implemented in the respective W-LLC object, and virtually accessed by an R-MAC object. Notice how natural and implicit the factorisation and placement of functional roles is in such an object-oriented model, and how the dynamic interaction of the different components delivers the overall service, allowing any of them to be replaced subject to the dynamism of the channel allocation algorithm. This is of key importance for effective channel management as there is need for fast frequency switching at the physical radio interfaces and tight rate control of the packets fed to the lower layers. In the current layer model, such dynamism as involved in the channel allocation abstraction requires special cross-layer design, violating layer opacity by relinquishing external control to other layers.

Another interesting point regards the re-usability of the different service class objects. Through the inherent modularity of an object-oriented model and the generic interface template implemented by all GP service classes, it is easy to facilitate dynamic service composition by introducing, replacing or removing the WMN compartment services, as needs be. Network layer functionality (as provided by IP) may use

directly (as a link layer) the W-LLC and R-MAC objects or the WMN objects. The interface remains the same and the functionality is optionally/dynamically extensible. Similarly, it should be possible to integrate within the WMN layer optimised (for the WMN environment) routing functionality that can lead to dynamic deprecation of an overlaying IP GP class by the transport level GPs. Simplified as it may sound, the GP model thus provides a powerful facility for optimisation decomposition [10].

6.5 Proof of concept validation

A proof of concept validation of the aforementioned design was performed with OPNET Modeler [49] simulation environment. For it, multi-radio MAPs were developed, which implement all the details of the distributed channel management mechanism detailed in [15].

In the simulated scenario, MAPs build a self-organized WMN. Each one of the MAPs that build the WMN network is switched-on in a random instant within simulation interval $[0, 2]$ s and self-configures its resources randomly. It then announces its initial configuration to its one-hop neighbourhood broadcasting a MAP state packet $P_{MAPstate}$ in all channels. Each MAP collects information from its neighbours from the received (P_{nstate}) packets. Every T_{Hello} seconds the MAP decides individually if self-optimization of its radio resources is needed, based on the information collected from its neighbourhood. If there are needed changes in his configuration, it broadcasts them using a $P_{MAPstate}$ packet. Connectivity with neighbouring nodes is guaranteed by the channel management strategy (for more details see Section 2 and deliverable D-5.2 [15], where the strategy is detailed).

First, a chain WMN topology is studied for different number of MAPs. Figure 6.3 depicts a chain WMN of 4 MAPs, being the gateway 3 mesh hops away from the depicted end-user client. MAPs form a backhaul WMN network compartment. Equidistant MAPs lie in communication range of their 1-hop neighbours, and in interference range of one- and two-hop neighbours. Each MAP is equipped with 3 radio interfaces (W-LLC objects): one IEEE 802.11g for providing connectivity to end-user clients, and two IEEE 802.11a for backhaul interconnection with other MAPs, all operating at a physical data rate of 54 Mbps. The two IEEE 802.11a radios on every MAP are under control of the channel management mechanism (using R-MAC objects – virtual channel interfaces).

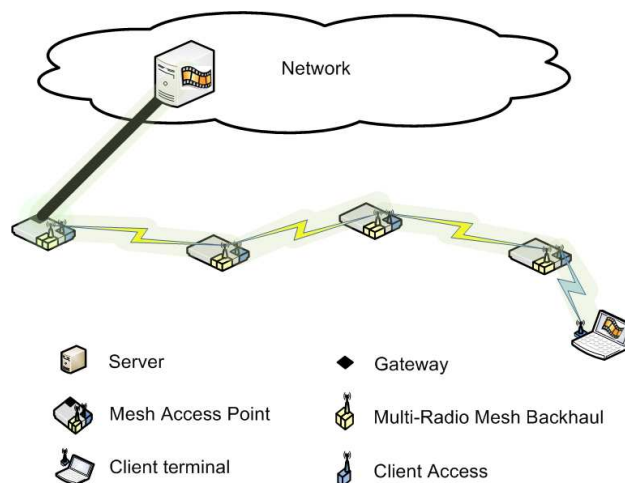


Figure 6.3: Example of a 3 hop chain topology WMN with a server reachable through the gateway providing a streaming application to a client connected to a MAP.

A server in the fixed network provides a streaming application, consisting in a stream of 1500 byte packets, delivered over User Datagram Protocol (UDP) with a given inter-arrival time. This server is reachable



via the MAP with gateway access to the fixed network. In the present scenario, a client at the opposite extreme of the MAP gateway is the destination of this stream. When a packet is received at a MAP, coming from a gateway to the fixed network or from a client in infrastructure network the MAP provides connectivity to, needing to be forwarded to a specific destination, a path must be known. If it is not available in the routing table, it is discovered by the reactive Ad hoc On-Demand Vector (AODV) routing algorithm.

The theoretical maximum application rate of a single hop is calculated. An application data packet that shall be transmitted in the wireless medium, is encapsulated recursively in UDP, IP, Medium Access Control (MAC) and Physical Layer (PHY) packets with associated headers, increasing the size of the packet. Besides this packet overhead, a time overhead is introduced in the transmission of the packet by the Carrier Sense Multiple Access/Collision Detection (CSMA/CD) contention mechanism to the wireless channel, recurring to a back-off and sending of RTS, CTS and ACK frames plus the corresponding inter-frame spaces to protect the transmission of the packet. The total overhead can be easily estimated, being detailed in [99] its calculation. Considering this overhead, a physical data rate of 54 Mbps only enables to transmit 1500 byte packets at a maximum application rate of 26.6 Mbps.

The maximum rate for the streaming application is then evaluated through simulation for a multi-radio chain WMN scenario, varying the following parameters:

- Number of equidistant multi-radio MAPs in the chain WMN, increasing the number of hops between the gateway and the client;
- Number of orthogonal channels used. The use of orthogonal channels guarantees no interference of links within interference range.

For each scenario, the evaluation of the maximum supported application rate is done in consecutive simulations, where the intensity of sent packets flow is increased (by decreasing the average packet inter-arrival time, exponentially distributed). Is considered acceptable a packet loss up to 2%.

For comparison of results is simulated a scenario with a similar topology but with MAPs with only one IEEE 802.11a mesh radio interfaces (one mesh W-LLC), all fixed to a common channel (i.e one R-MAC object per W-LLC object).

The results of the simulations are presented in Figure 6.4. From the simulation of a single-hop chain WMN (with only two MAPs), was achieved a maximum application rate of 24.5 Mbps, 92% of the maximum theoretical bound of 26.6 Mbps. This slight mismatch occurs due to the realistic simulated environment. In fact, to obtain the maximum throughput we have to simulate in a threshold region where the high intensity of packets may result in some collisions, requesting retransmissions at the MAC layer (up to 7 retransmissions are allowed), increasing the contention window and thus average back-off. Even so, no packet losses were measured.

When simulating chain WMN scenarios of MAPs with one mesh W-LLC, there is a strong degradation with the increasing number of hops as expected, as all nodes use the same channel and thus interfering with each other, strongly decreasing the throughput.

For chain WMN scenarios of multi-radio MAPs with 2 mesh W-LLCs, is observed, from Figure 6.4, that the achieved maximum application rate has no degradation with increasing number of hops, as long as the number of channels (R-MACs) is similar or greater than the number of hops. In these conditions, no packet losses are observed. This result is expected, as no link is interfered, making disappear typical hidden, exposed and deaf node problems of multi-hop networks [100] that result in collisions and packet losses. Using the proposed distributed channel management strategy with 5 channels, an optimum channel assignment guarantees an application rate of 24.5 Mbps, without hop degradation, for any size of chain WMN. This is guaranteed by the fact that all co-channel links will occur beyond the interference range (2 hops). Nevertheless, as may be expected, the bigger the chain, the larger the average packet delay, due to the overhead introduced in each of the links, as discussed previously.

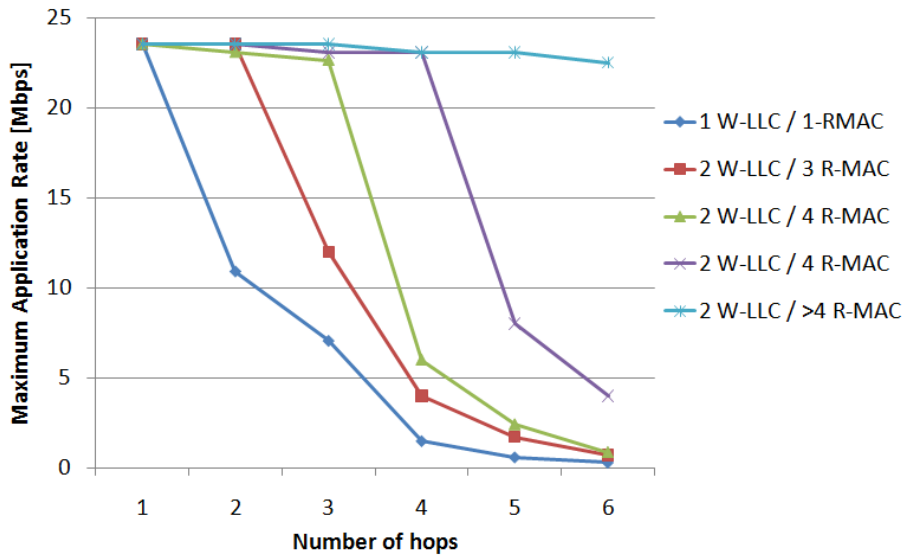


Figure 6.4: Maximum application rate in WMN chain topologies of different sizes, varying the number radio interfaces for mesh backhaul (W-LLCs) and channels (R-MACs)

The global performance results presented in Figure 6.4 show that an optimum channel assignment is reached for each MAP node. This is achieved by the distributed channel assignment algorithm that, for each node, converges in less than $3T_{Hello}$ seconds (T_{Hello} is a configurable parameter, set to 1 second in the present study), a relatively small transitory period to achieve a chain WMN with optimized radio resources in a distributed way.

Secondly, was simulated the topology illustrated in Figure 6.1a, where 10 MAPs form a backhaul WMN network compartment, and each MAPs lies in the communication range of three others; the MAP in the centre provides gateway connectivity to the Internet. MAPs are configured as previous, with two mesh W-LLCs and one W-LLC for end user connectivity. Each MAP provides connectivity to a single end-user client.

Similarly to the chain topology scenario, a server in the fixed network establishes one streaming application session with each of the end-user clients, sending to each packets of 1500 bytes size with an average inter-arrival time (T_p), exponentially distributed, delivered over UDP. The gateway MAP establishes Down Link (DL) WMN GPs to each MAP, for sending the streaming packet flows. The MAP gateway has to forward nine flows to the three first-ring neighbouring nodes, and each of those first-ring nodes will forward 2 flows to the two second-ring nodes. In this sense, the MAP gateway will be the bottleneck of the WMN.

Considering only 3 channels (R-MAC interfaces) available, flows of 1.2 Mbps encounter no packet loss at the receiving R-MAC interface of any of the MAPs in the scenario. When increased to 2.4 Mbps, packet losses are around 0.5%. Increasing the number of available channels allows better performance, as interference is reduced as discussed previously: using 4 channels, 2.4 Mbps flows present 0.2% of losses, and using 5 channels leads to no packet losses. The border configuration is for 5 channels, where there is 0.02% packet loss for 3 Mbps flows. Increasing the flow throughput beyond this value results in high delays up to a situation that packets are dropped due to limitation of the physical channel capacity (and not due to interference in the wireless medium). For comparison, using a similar topology of MAPs with only one W-LLC, we get 10% packet loss for 1,2 Mbps streaming flows, increasing to 34% for 2.4 Mbps. This is due to the referred interference problems (hidden, deaf and exposed nodes), evidencing the the poor performance of



single radio WMNs.

Making some theoretical considerations, in this classical WMN scenario traffic concentrates towards the gateway. The gateway will be the bottleneck of the scenario, limiting the maximum throughput, since it has to forward nine flows. Was observed that the maximum theoretical throughput for sending packets of 1500 byte without errors is 26.6 Mbps. In simulation, the gateway MAP reaches to forward 21,6 Mbps, 81% of the maximum throughput, which is relatively high result. In fact, one shall consider that, besides the classical problems already referred, there is a back-off introduced after switching into a new channel to avoid the multi-radio hidden node problem (to avoid collisions with ongoing receptions of neighbouring nodes in the same channel). These simulations show the effectiveness of the proposed strategy in reducing interference and increasing throughput in WMN with multi-radio MAPs.



Document:	FP7-ICT-2007-1-216041-4WARD/D-5.3		
Date:	June 14, 2010	Security:	Public
Status:	Final	Version:	1.0

7 Conclusions

In this chapter we present conclusions regarding the *lessons learned* and discuss some of the problems solved, but also remaining open issues or trade-offs. The document presents the results of prototyping and evaluation phase of WP5. The results show a convergence of understanding of core aspects of a new networking architecture that extends the traditional network stack (as in TCP/IP or OSI) by a number of main features:

- A recursive layering/composition structure with a common API
- A method of querying/inspecting paths and retrieving their internal state
- A common way of description and negotiation of resources and transport services

As with any architectural framework that tries to be *generic*, a number of options had to be defined in order to not restrict development. This regards different flavours and interpretations of the best way of transferring cross-layer information. Another – related – problem is that of so-called unified control planes like GMPLS vs. the easy introduction of new functionality in the network. It is natural that discussions that have been going on for more than forty years now in the networking community can not be closed within a research project, but the identification of options that support either direction in a single framework hopefully contributes to a better understanding of the general problems of networking. Two prototype implementations of the framework have been developed within WP5 that provide more insight into the feasibility and scalability of some of the proposed solutions. Other activities were concentrating on solving specific problems (like mobility, WMN, or OTN) with the new tools at hand.

7.1 Prototyping and performance evaluation activities

The two most advanced prototyping activities within WP5 (Section A and C of the Appendix) are suited to give an outlook on how the GP can actually be introduced in the network. The combined simulator and prototype, mainly developed at University of Paderborn, consists of an OMNet++ network simulator that contains a wrapping function for POSIX communication primitives. This means that the code developed and tested in the simulator can easily be transferred into Unix-based environments like Linux PCs. The whole node compartment can be installed on a virtual machine and can serve to gradually introduce the new API in real networks. The prototype is available under an Open Source license and is documented in the Appendix A as one stand-alone chapter. Another evaluation and prototyping activity is presented in chapter C. The evaluation concentrates on the performance of the filters that realize the service composition inside a MP. The prototype starts from a different option and implements a MP. Both prototypes use similar APIs with differences in the implementation when it comes to appropriately supporting the respective option (with or without MP).



7.2 Solved problems

7.2.1 Session mobility and terminal mobility

The key to an efficient management of mobility is the separation of paths in different CTs, in other words, the maintenance of a binding table between identifiers for the calling Entity (Identifier) and the serving Entity (Locator). The GP architecture repeats this Loc/Id split recursively and transparent to the calling Entity, making all mobility equal to a change in binding. This by itself does not say anything about the performance of a mobility management system, as this depends on the implementation of the name resolution system and other factors like the position of anchors. The recursive nature of the framework allows however the design of specific *handover support* CTs that have a limited number of Entities and a high-performance name resolver and to insert this CT at any desired place in the stack. An elegant way of moving an entire session is explained in section A.2.4.3 of the Appendix.

7.2.2 Introduction of innovation in the network stack

One of the major objectives of the ForMux Work Package (WP) within 4WARD was to ease the introduction of new functionality in the stack of routers, switches, but also end nodes. The concept of CTs and the recursive layering allows inserting *special purpose* overlay (or better, *shim*) networks. The Cooperation and Coding FrameWork (CCFW) that was shown in the previous deliverable D-5.2 [2] and the chapter 6 on WMN are examples for such shim networks.

7.2.3 Investigation of internal state of a path

Whereas a path in the Internet is typically just the sequence of routers that a packet traverses we added a method to it that inspects its inner parameters. Since the EP of the path is its local state keeper, it is the EP that is actually being queried. Examples for the benefit of this can be found in section B.2 where individual sub-paths can be investigated and lead to potentially different routing decision.

7.2.4 Flexible resource description and composition

The functional composition of GPs relies on a typically pre-established control graph (or service graph). This composition takes place *in the control plane* and creates bindings between initial Endpoints of Entities that actually can mutually provide connectivity. The control graph inside the node CT is then used to discover available resources, followed by neighbour discovery (reaching out to remote Entities in the respective CTs and subsequently the exchange of routing information. While this control graph today is hard-wired and rather sparse (allowing little routing decisions in the *stack*), chapter C of the Appendix evaluates the performance of a dynamic composition. This dynamic composition requires a generic description of resources and services across CT boundaries. Optimizations of cross-layer information transfer that can be found in literature [101, 102, 103] are specific to the interface between certain layers (typically between data link and transport) and therefore hinder any dynamic composition. A generic resource description based on ontologies of network services now enables Entities to negotiate a set of parameters that a) are *allowed* to be seen by another CT and b) are *of interest* to the respective other Entity. The set of parameters that is the result of the negotiation/ontology is henceforth carried in route advertisements and `create_GP()` calls. We demonstrated in chapter 5 one heavy-weight and one light-weight interface offered by different Entities. A powerful ontology of resources will be able to match the required and offered services even between such rather opposite transport services. Within this project we could only go the first steps towards creating such an ontology, but for a dynamic service composition this is essential.



At the same time, we observe that the concept of "resource" and "service" collapse. The service offered by an Entity is the management of a resource, the Entity "controls" the resource. This resource can be arbitrary, it may well be a video stream or a web page, computing cycles of a server, it may also be a certain bandwidth that is to be shared, or even more abstract the attention of a number of people. The combination of Entities that characterize the resources under their control and a powerful match-making ontology that creates the (control path) bindings between the Entities creates a potentially truly universal framework. It also shows that the difficulty and complexity would be shifted to the ontology. Architecturally again, an MP would host the ontology (as well as the control graph), whereas both could be distributed inside a node CT as well, creating higher flexibility in adding/dropping new CTs at the price of an increased communication cost within the node.

7.3 Open issues

7.3.1 Unified control plane vs. multiple compartments

A unified control plane in our terminology is formed by Entities of a CT that reaches across all nodes. These Entities control Endpoints of very different nature, optical transponders, wireless transmitters, video sources, and could therefore mediate between different GPs. One can easily see that the optional MP is the incorporation of this principle in our architectural framework. The apparent opposite is a set of more or less opaque (and very "thin") overlay CT. The two extremes reflect the old discussion of peer vs. overlay networks [104]. Regarding the architecture this means that there will not be a single solution, but rather that there has to be a way to group or compose a number of CTs into one CT. The Entities of this compound CT may then well be MPs that however export the GP API as if they were "simple" Entities.

7.3.2 Light-weight vs. heavy-weight resource description and control

The two example implementations of resource descriptions in 5 illustrate that there is no single answer to the question how much information a CT should provide to its clients. The light-weight resource management evaluation results indicate however that few parameters could be sufficient to provide a fair sharing of the bandwidth in today's connection-less networks. Other results from the project [105] that energy savings for reasonably dimensioned IP-over-WDM networks are rather modest when comparing full knowledge of optical channel information in the IP layer to an opaque underlay. However, as there were several assumptions regarding, e.g., known traffic patterns, we conclude that further investigation is required. The architectural framework of the GP allows for both options (and any in between).

7.3.3 Performance gains?

The work package aimed at new functional architecture for describing today's and designing tomorrow's networks. A gain in performance (measurable in bit/s) was therefore not the primary target. As mentioned above, several problems of today's Internet architecture disappear with the new one, and as consequence there will also be gains in performance, but these will rather be of boolean type (did not work – works now, like, e.g., multi-path routing or the Cooperation and Coding Framework (CCFW)). The signalling itself that comes along with establishing GPs will be the same as today, as there may be connection-less (implicitly established) and connection-oriented (explicitly established) GPs. The actual exploration of the inner state of a GP may lead to an increase of management traffic. Some attempts to limit this traffic have been considered in WP4 and the joint deliverable [3].



7.4 Perspective

As the GP concept is disruptive in nature, direct applicability of the GP concept to standardization should not be expected, as the essence of the work is a fundamental change in the way the protocol stack would be composed. Only recently, however, a commercial path has opened for future Internet hardware: OpenFlow, an initiative to create an open and flexible interface between the control and the forwarding path in modern switches started at Stanford University in 2008. The aim of the OpenFlow consortium is to open the interface between a switch and its controller in order to lower the entry barrier for innovation on both sides of the interface. As it turns out, the GP API is orthogonal to the current OpenFlow API, as the latter controls communication between a controller (the Entity) and a forwarding engine, while the former is situated between the Entity and an Endpoint that demarks the layer borders or edge devices; something that was considered out of scope so far in OpenFlow. In fact, the forwarding of data is one of the key properties of a Generic Path, but the GP includes in addition data processing (header add/drop) at borders of compartments. The OpenFlow interface would be one between the controller of an MP and its data backplane (we used yellow and green triangles to indicate this split). It can also be considered a part of the GP API if forwarding is exclusively attributed to Endpoints (the non-MP case). Multi-layer problems have not yet been considered within OpenFlow, so the GP interface will be suggested to the OpenFlow consortium with the intention to create a set of interfaces for processing of data.

There are other standardization activities like the Forwarding /Control Element Separation (ForCES) [106] working group inside Internet Engineering Task Force (IETF) and the ongoing GMPLS standardisation that came up with the Path Computation Element (PCE) in order to speed up the path setup. All these activities deal with "split architectures", i.e. separate forwarding from control. The contribution of our work in WP5 was to add to these two networking functions a third one, the Endpoint, as the point where layers are crossed and composed. In addition to this functional split there is also a split between phases of service discovery/composition, control and forwarding. These typically take place on different time scales and should follow a decreasing complexity (forwarding being the least complex and hardware-near function). The increasing dynamics of networking equipment (through mobility and power saving) will increase the pressure to find automated ways of composing network services. This will foster the work on ontologies for network services.

A GP Prototype Design and Implementation

A.1 Introduction & Fundamental Concepts

Within this chapter we will firstly present, why the ForMux/GP approach is interesting for today's networks and we will secondly describe the fundamental building blocks of the ForMux/GP architecture.

Remark: All topics are based on our own point of view, which tries to comprise both the architecture described by 4WARD WP5¹ and the limitations of a real world native implementation. Some points may seem to be a little tricky but they are necessary for the native implementation.

A.1.1 Motivation

Nowadays the network protocol landscape seems rather patchy and without well-defined interfaces. Many well known and well implemented protocols (e.g., TCP/IP, Ethernet, HTTP, ...) exist, as well as many more small and unfamiliar protocols. All of them are glued together using custom-tailored concepts. One example is the interaction of TCP, IP and ARP in many operating systems. Adding new protocols, implementing new functionality, also is (at least) problematic because of the ISO/OSI model.

In a future with ForMux/GP there will still be many different network protocols, but they will be glued together by a generic architecture. This allows to arbitrarily combine protocols to form a transport which completely fits the users needs.

A.1.2 Basic structures and their relationship

The essential structures and how they are related will be explained in this section. Always keep in mind, this is only a very brief introduction into our concepts. For an in-detail explanation of all design decisions, see Chapter A.2.

A.1.2.1 Entities & Compartments

Entities are the basic building blocks of the ForMux/GP architecture. An Entity is the container for any kind of programming logic. There might be an Entity which provides TCP/IP functionality or an Entity for Ethernet encapsulation as well. Additionally, there is one special type of Entity, which is unique for each physical node called *Core*. It is responsible for the management of Entities and the communication between them.

All Entities and the Core together on one single physical node form a *Node Compartment*. Furthermore, Entities are grouped in *Compartments* when they support the same protocol (e.g., a TCP/IP Compartment for Entities supporting this protocol), whereas each Entity could be member of several Compartments (but only member of one *Node Compartment*). Hence, Entities in a certain Compartment do understand their *names*, e.g., IP addresses. Of course, an Entity could have multiple names (e.g., several multicast names).

¹<http://www.4ward-project.eu>

To make it clear which Entities are allowed to communicate with each other, we restrict communication to those Entities that are part of the same Compartment. Therefore, the introduction of a Node Compartment enables Entities, supporting different protocols, to communicate. That's necessary, for example, to get data from a TCP/IP Entity on the cable, i.e., to an Ethernet Entity.

The decision to be part of one Compartment is made by the Entity itself. There might be for example a TCPIP Compartment or an Ethernet Compartment. Two points are important about Compartments and the names of Entities:

1. Every Entity needs to have a name in the Compartment, which can but does not need to be unique (e.g., all Entities may have a non-unique multicast name and a unique unicast name). One exception is the Node Compartment, in which all names must be unique.
2. Every Entity in one Compartment must be able to understand all names in the Compartment. As a consequence, syntax and semantics of names are not limited in any way. One exception is the Node Compartment where the names must be normal strings.

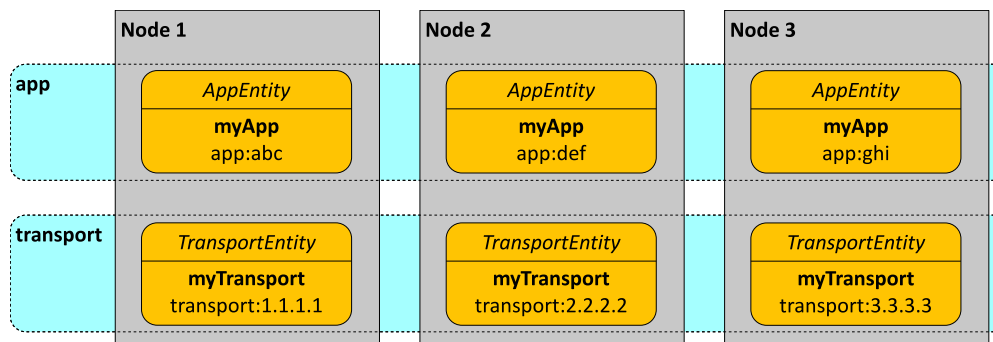


Figure A.1: A simple Entity

Figure A.1.2.1 shows a simple example of three physical nodes with two Entities each. The class (or protocol/application type) of the Entity is written in italic face, the Node Compartment in bold face and all other names in normal face in the form “Compartment name:Entity name”.

Entities interact with each other over Hooks & Ports or Generic Paths & End Points, which will be described in Section A.1.2.4. The interaction between Entities is a kind of service usage and providence.

A.1.2.2 Capabilities

The *Capability* concept introduces a generic *service provider and requester* model to the world of GP. More precisely, Capabilities are formulated using a certain kind of description language. It is motivated by the variety of different needs of each protocol type. For example, some protocols may need a channel that by itself provides error correction since the protocol does not want to deal with this issue by itself. Another protocol may need a reliable unicast connection, whereas a third protocol could ask for an ordered delivery of the passed packets. Since GPs are established by using/establishing “underlying” GPs, we need a way of selecting the right underlying GPs, satisfying our needs as exemplified above. Capabilities are the intended way to express these needs in a structured and processable way.

A.1.2.3 Resolutions

Inside Compartments a generic resolution functionality is defined. This allows Entities which own a name inside some Compartment to resolve names from this Compartment into another Compartment. These

results of the resolution can be used to contact the resolved name, e.g., a DNS name can be resolved into an IP address over which it can be contacted.

The way how resolution works is implemented by the Entities but the interface to the outside is the same for all Compartments. Hence, Entities can be combined arbitrarily.

Some possible resolution schemes are multicast or broadcast resolution, DNS-like resolution by central resolvers, or resolutions based on querying many other Entities in the same Compartment as is implemented by many peer-to-peer overlay networks. A resolution can also be restricted to specific Capabilities, e.g., to only get resolutions which provide a specific property.

For more details on the name resolution mechanisms implemented in this prototype, please refer to the Chapter “Naming and addressing” of the 4WARD Book.

A.1.2.4 Hooks & Ports

As mentioned in Section A.1.2.1 and A.1.2.2, Entities are providing services (you might think of them as protocol *handlers*). Furthermore, Entities are able to use services of other Entities inside the same Node Compartment throughout a defined interface as well. This interface is comprised of Ports and Hooks.

A *Hook* is the connection between exactly two *Ports*. You can think of it as a cable connecting two sockets. Every Port is owned by a single Entity and this Entity can send data over this Port, which will then be received from the Entity on the Port at the other side of the Hook. Hooks are bidirectional and guarantee that data is received without errors and in order.

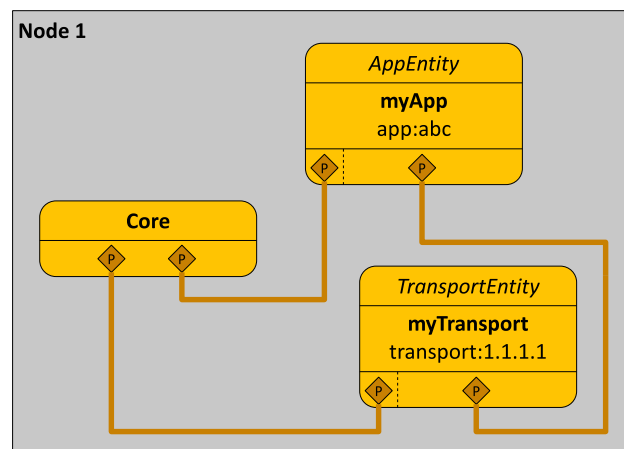


Figure A.2: Figure showing how Ports and Entities are drawn in our documentation.

Figure A.1.2.4 shows how we will draw Entities and Ports, as well as their interconnection.

A.1.2.5 Generic Paths & End Points

A *Generic Path (GP)* is, from the conceptual point of view, the instantiation of a protocol and optionally the state of the protocol, too. *End Points (EPs)* are, just like what Ports are for Hooks, your entry point to a Generic Path. They implement the Generic Paths communication paradigm and logic. The linkage between Ports and End Points, as well as Hooks and Generic Paths, will be explained in a minute.

On the conceptual side, Hooks are specialized Generic Paths, living only within a single Node Compartment. This is possible because Hooks are (simply speaking) an end-to-end connection between exactly two Entities, providing certain Capabilities, hence less powerful than the general Generic Path concept.

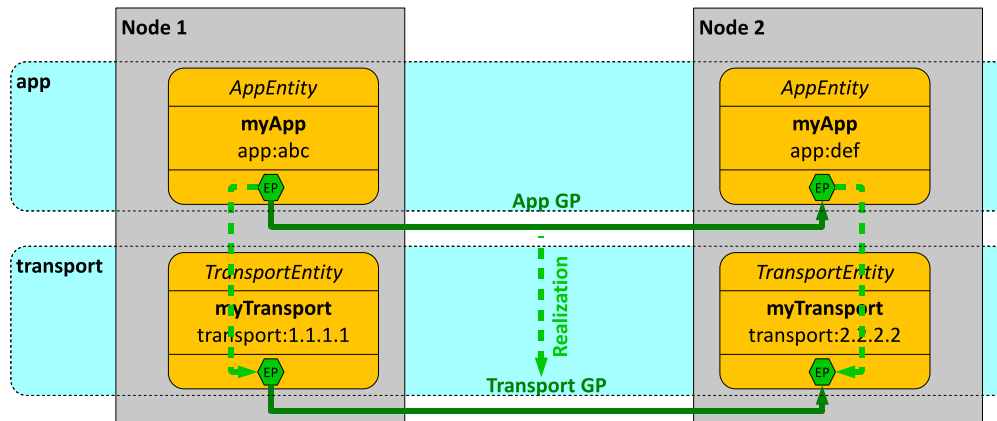


Figure A.3: Generic Paths

About the relation between Hooks and Generic Paths

Just to differentiate between the Node Compartment view and the more general Compartment term, we will talk about Ports and Hooks within Node Compartments, and in contrast about End Points and Generic Paths in all other cases.

Ports and End Points can and will be tied together within Entities, i.e., if linked, a packet dropping out of Port P_a will be put into End Point E_a and vice versa. That is a simplification which is only true if the Generic Paths are bidirectional but some Generic Path types are only unidirectional by design.

A.2 Technical concepts

One central part of our work was to determine how the ForMux concepts could be implemented in an efficient and reliable way. In this chapter we present how a parallel implementation for both simulation in OMNeT and native operation on POSIX systems is possible.

A.2.1 Overall structure

There are two different views on our architecture: A view on inheritance and code reuse as described in Section A.2.1.1 and a view on the architecture on one node as shown in Section A.2.1.2.

A.2.1.1 Class Layout

To achieve a high degree of code reuse, we developed a class hierarchy as depicted in Figure A.4 for the Entities (and analogously for the Core). All code which is generic for both POSIX and OMNeT (or any other platform) is implemented in `AbstractEntity`. All platform specific code resides either in `POSIXEntity` or `OMNetEntity`. Furthermore, `Entity` is just a typedef which points to the class for the currently used platform. With this structure we achieve two important goals:

1. It is simple to implement new Entities. New Entities can be platform independent if they inherit `Entity` or can be platform specific if they inherit `POSIXEntity` or `OMNetEntity`.
2. We avoid redundant code. All data structures and procedures which are platform independent are implemented in the common base class `AbstractEntity`.



Names of zero length have a defined meaning in all Compartments and are used as a name that does not belong to any Entity and can be used as placeholder.

Names additionally have so-called flags associated with them. Flags are used to give names a special meaning, e.g., that a name can be used multiple times (e.g., multicast names in the simplest case). One part of the flags, like the multiple flag, has a well defined meaning for all Compartments while another part of flags can be used freely for Compartment internal usage.

Node Compartment Names

Names inside the Node Compartment are defined as human readable, valid UTF-8 strings without NUL-termination and a non-zero length. A name must uniquely identify a single Entity and can not be used for multiple Entities at the same time. The name `core` is reserved for the Core but all other names can be used by normal Entities.

A.2.1.4 Packets

Packets are the generic form of data, which can be used by Entities to communicate with each other. Although this seems to restrict all data exchange to a packet-oriented communication paradigm, this is not true as Entities/Endpoints can implement any desired paradigm.

A generic packet only contains a 2 byte long type tag and the length of the immediately following data. The possible types are divided into 4 different categories.

- Invalid (0x0000) and Reserved for future usage (0x8000): These packets should not be used.
- Node Compartment communication packets (0x8001 to 0x8ffe): These packets are used for generic control communication of Entities inside the Node Compartment. Examples are packets for creating and releasing Hooks or for joining and leaving a Compartment.
- Generic Compartment communication packets (0x0001 to 0x0ffe): These packets are used for generic control communication between Entities. They be used inside the Node Compartment and every other Compartment but the sender and receiver must be member of the same Compartment. Examples are packets for resolution, GP creation and release and status packets.
- Data packets (0xffff): Data packets can be used for Compartment specific communication and their content only has a meaning to the Compartment for which they are.

The Node Compartment communication packets and generic Compartment communication packets are both used for generic control communication between Entities and as such should guarantee some degree of reliability. To ensure data correctness they contain a 32 bit CRC (as defined in ISO 3309 and ITU-T V.42) and a randomized 64 bit sequence number to allow finding the response corresponding to a request.

Node Compartment Communication Packets

Node Compartment communication packets contain the sender name at the beginning and type specific data following.

Generic Compartment Communication Packets

Generic Compartment communication packets can be split into two categories. The first half of the type space (0x0001 to 0x3fff) all contain the source and destination in the originating Compartment and



the source and destination in the transport Compartment, i.e., a Compartment, acting as a servicing Compartment, actually transferring data from one node to another. These information are used to allow correct forwarding to the destination and sending status back to the source.

The second half of the type space (0×4000 to $0 \times 7ffff$) contains packets for GP creation. These packets contain the GP label stack (see Section A.2.4.1) to allow correct forwarding.

Data Packets

Data packets can optionally contain a destination name and a destination port ID which can be used for Compartment specifics. After this two fields a data packet can contain arbitrary data, which is used for the Compartment internal communication.

For complete details about all types of packets, their fields and their meanings please take a look at the source code documentation.

A.2.1.5 Serialization

Packets need to be serialized to a platform-independent byte string for transfer between Entities and over node boundaries. For this, the serialization of a few, basic types is defined in this section and the serialization format of the complete packets is defined as the aggregation of these serialized basic types.

All data types must have a size that is a multiple of 8 bits or 1 byte.

Fixed Size Integer Types

All fixed size integer types are serialized in big endian byte order, i.e., most significant byte first. Unsigned integer types are stored as is, signed integers must be stored in two's complement.

Variable Size Integers

Variable size integers are used in places where small numbers are very common but very large numbers should still be supported without wasting space in the common case.

The variable size integers are designed similar to UTF-8 and are constructed from two parts: a size prefix and the actual integer. To store an integer of n bits the first $s = \text{ceil}(\frac{n}{7})$ bits are used as the size prefix, followed directly by the actual integer stored the same way as fixed size integers. The size prefix is defined as $s - 1$ ones followed by a single zero and the length of the size prefix is equal to the number of bytes required to store the complete variable size integer.

Example: `110xxxxx xxxxxxxx xxxxxxxx` has the size prefix `110`, which is 3 bits long. The following `x` are the 28 bit which contain the actual integer value.

Arrays

Arrays are serialized by first serializing the number of elements as variable size integer followed by exactly that number of array elements.



Names

Compartment names and names inside a Compartment are serialized by first serializing their length in bytes as a variable length integer followed by the serialization of the name itself. For Compartment and Node Compartment names this is an UTF-8 string, for all other names this is defined by the Compartment.

As such names are serialized the same way as an array of bytes.

A.2.2 Capabilities

To recapitulate, Capabilities are intended to provide a way of expressing the needs for services when establishing a communication path between 2 up to n communication partners via GP. These needs will be expressed via sets of *Capabilities*, whereas each Capability is represented by a name and a set of *Properties*. A Property essentially is a key-value pair with the addition of a type. The name of Capabilities describes the general communication type that this Capability represents while the Properties describe the different variants or options of the Capability.

There are 8 pre-defined Property types to choose from:

Type	Description
integral types (INT, DOUBLE)	self-explanatory
strings (STRING)	strings, represented as a sequence of characters
ranges (RANGE_INT,RANGE_DOUBLE)	lower and upper bound (ℓ and u , resp.) of the specified type, forming a closed range $[\ell, u]$
sets (SET_INT,SET_DOUBLE,SET_STRING)	non-empty sets of the specified type, containing $n < \infty$ elements

The reason behind representing Capabilities itself as sets of Properties becomes explicit by introducing *set operations*, namely *union* and *intersection*. Assume Entity A needs Capability $C_1^{(A)}$ and Entity B likewise provides Capability $C_1^{(B)}$. In order to figure out if B provides at least the Properties needed by A , an intersection has to be performed, i.e., $C_{\text{inter}} := C_1^{(A)} \cap C_1^{(B)}$. Obviously, if $C_1^{(A)} \subseteq C_{\text{inter}}$, then B provides with $C_1^{(B)}$ at least the Properties needed by A .

Note. Of course, the condition $C_1^{(A)} \subseteq C_1^{(B)}$ does exactly the same in above's situation, but it is only a specialized case compared with the one above.

To understand the need for union of Capabilities, assume that Entity B provides Capabilities $C_1^{(B)}$ and $C_2^{(B)}$. As Capabilities can be combined to a *list of Capabilities*, called `CapabilityList`, B returns a list of Capabilities produced by joining $C_1^{(B)}$ and $C_2^{(B)}$. This feature is used by the Core to produce a list of Capabilities, provided by the set of resolved Entities.

A.2.3 Resolutions

As described in Section A.1.2.3, a generic mechanism of name resolution exists that allows to resolve a name in the own Compartment into a name in a different Compartment, which can be used to contact the resolved name. This is, however, not completely correct: Instead of getting a name inside another Compartment as resolution, an index inside a resolution table is returned. The resolution table is local to a node and acts as a resolution cache. The reason for not returning the actual resolutions is, that no Entity should ever know names of different Compartments (Information Hiding).



Resolutions are handled in a generic way from three different points of view.

A.2.3.1 Resolution handling inside the source node

The following list describes the actions that happen if an Entity wants to resolve a name inside its own Compartment to contact this name somehow.

1. To start a resolution, an Entity first has to send a `WhoHas` packet to the Core, which contains the name which should be resolved and optional capabilities for the resolutions. The `WhoHas` packet is sent to the Core, because it might already know the resolutions from its cache and to ensure the information hiding. It is important that the name has the correct flags because the Core and other Entities will use them during the resolution process. For example if a name does not have the multiple flag only the first resolution is returned instead of waiting for all resolutions.
2. The Core now tries to answer the resolution in three different ways
 - a) It tries to answer the resolution by finding a resolution inside the same Node Compartment. This is done by static knowledge about the Node Compartment. Entities and by asking these Entities with another `WhoHas` packet. If an Entity gets a `WhoHas` packet from the Core it answers this `WhoHas` packet with an `IHave` packet if it has this name or knows the resolution, e.g., is a resolver.
 - b) It tries to find the answer in a resolution cache, which was created from previous resolutions.
 - c) It tries to find a resolver inside the static resolver table, which can resolve names from the requested Compartment into names inside a Compartment, which can be used by the asking Entity for communication. If such a resolver is found a new `WhoHas` packet, with the target Compartments filled in, is send to the resolver. This is done over one or more Entities inside the Node Compartment, which are also member of a Compartment in which the resolver can be reached.
3. Now the Core waits for all answers from the above steps or until a timeout is reached ². The aggregation of resolutions simplifies resolution handling in the Entities in many cases, because all answers come at the same time and the entity knows that no other answers will come later. In some cases this is suboptimal though.
4. All answers are aggregated in the Core and a single answer is sent to the resolving Entity, containing the indexes of the resolutions in the resolution table and Node Compartment names of Entities, that could be used to contact the resolutions.

In the end the resolving Entity can send control packets to the resolved Entities by sending them to the Node Compartment names from the resolution and passing the index as destination (which is a name inside the Resolution Compartment). These packets contain the source and destination inside the originating Compartment and two other names in the so-called encapsulation Compartment, which is used as service for transferring the packet. The source inside the encapsulation Compartment is the Node Compartment name of the Entity, that gets the packet, the destination is the index in the Resolution Compartment.

The Node Compartment Entity can now ask the Core for the real name of the resolution inside the Compartment by using the index and can replace the encapsulation source with its own name inside that Compartment and the encapsulation destination with the resolution.

²In future versions, the resolving Entity could request the Core to forward answers as soon as they are available



A.2.3.2 Resolution handling inside the destination node

The following list describes the actions that happen if a `WhoHas` packet somehow arrives inside a node and the receiving Entity is the encapsulation destination of the packet.

1. The encapsulation destination sends the `WhoHas` packet to the Core.
2. At the Core all possible destinations are searched
 - If the Core knows about the destination from its internal Compartment tables these are used.
 - If the Core knows about Entities that are in the destination Compartment all these Entities are used.

The packet is sent to all possible destinations and only if there is no possible destination inside the Node Compartment the Core immediately sends a `Status` packet for the failed resolution to the previous encapsulation entity in step 1.

3. The Core aggregates the answers and waits until all answers are received.
4. If some of the resolutions contain Node Compartment names, the Core replaces these Node Compartment names with suitable names of Entities that could act as encapsulation Entities. See case (1) in section A.2.3.3 for details.
5. The Core sends all resolutions in a single `IMHave` packet back to the previous encapsulation destination.
6. The encapsulation Entity now forwards the `IMHave` or `Status` packet to the destination, i.e., the Entity asking for the resolutions.

A.2.3.3 Resolution handling inside the destination Entity

The following list describes the actions that happen, if a `WhoHas` packet somehow arrives at an Entity and this Entity is the destination of the packet. This is only allowed if the packet is sent from the Core.

1. If the Entity also owns the name that should be resolved it directly sends an `IMHave` packet back to the Core, with its Node Compartment name as answer. The Core will then replace the Node Compartment name with the name of some other node Entity that could act as encapsulation Entity for the Entity that answered the `WhoHas`.
2. Otherwise the Entity must be a resolver and has special knowledge about names inside the resolution Compartment. It prepares an `IMHave` packet with all known resolutions and sends it back to the Core. In this case the Core simply uses these resolutions without filtering or changes.

A.2.3.4 Example

For further insight in the way how resolutions work, a simple example will be explained here. The situation is illustrated in figure A.5. Entity `app:abc` wants to resolve the name `app:de*` in the `app` Compartment. Asterisks in this `app` Compartment are defined as a single character wildcard. The expected resolutions in this case would be `app:def` and `app:deg` together with a resolution table index for their transport Entities and the Node Compartment name of `transport:1.1.1.1`. In this scenario the `app` Compartment uses multicast resolution and the `transport` Compartment provides a way for multicast resolution.

1. Entity `app:abc` sends a `WhoHas` packet to the Core in node 1, with `app:de*` as name to resolve.
2. The Core of node 1 does not know a resolution for this name but knows that the `app` Compartment has a resolver with the name `app:*` and that this resolver can be reached via `transport:255.255.255.255` (which has the meaning of a broadcast name in the `transport` Compartment).

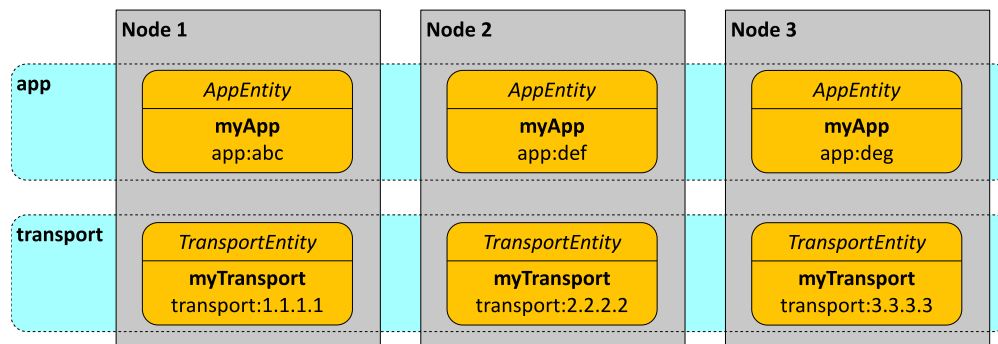


Figure A.5: Resolution Scenario

3. The Core sends a WhoHas packet to the transport Entity, which then broadcasts the packet to `transport:255.255.255.255`.
4. `transport:2.2.2.2` and `transport:3.3.3.3` receive the WhoHas packet and send it to the Core in their own node.
5. The remote Core now try to find a resolution, but only know that they have an Entity in the app Compartment, to which the WhoHas packet is now send.
6. The two remote app Entities receive the WhoHas packet and send a IHave packet back to their core, with their Node Compartment name as resolution.
7. The remote Core replace the Node Compartment names of the remote app Entities with the names of the transport Entities and send it back to `transport:1.1.1.1` through their transport Entity.
8. The transport Entity in node 1 receives the two IHave packets and sends them to the Core of node 1.
9. The Core of node 1 receives the two IHave packets and stores the resolutions in its internal resolution table. Then it sends a IHave packet to `app:abc`, which contains the Node Compartment name of the transport Entity in node 1 and the index of the resolutions in the resolution table.

A.2.4 Generic Paths & Endpoints

An *Endpoint* (EP) is (in some sense) your side's instantiation of a Generic Path (GP) and will actually do the GP's work; it is where the GP's functionality lives. Hence, an EP is *not* independent of the associated GP type, but its implementation and the GP API will, from a programmers point of view, be an EP API. Otherwise, if the GPs functionality would live in a separate GP object, an Endpoint for that GP would hardly be more than a layer of indirection for accessing the GP it depends on. It would simply wrap the interfaces for implementing the GPs communication paradigm (e.g., send/receive or pub/sub). Hence (roughly speaking)

$$GP = (EP_1, EP_2, \dots, EP_n),$$

i.e., a GP can be interpreted as the sum of Endpoints and the functionality provided by the combination of these Endpoints.

A.2.4.1 GP Label Stack

When establishing a new GP between Entities, a special packet called *CreateGPPacket* will be send. As it passes different Compartments (see Figure A.6 for an example) it accumulates the necessary information to (a) be aware at each time of the next hop and (b) to know about the upper Compartments in order to find its way back to the right Compartment, which is the one where the source Entity lives). This information is

accumulated to prevent a recursive approach of establishing GPs, which would have more round trips and as a result increases the network traffic and latency.

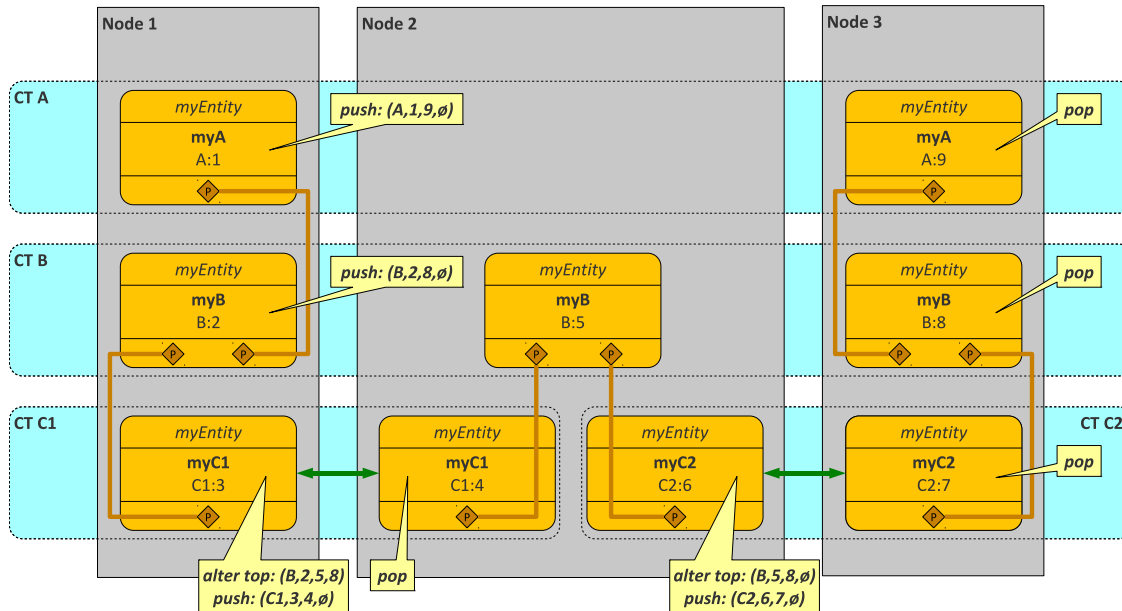


Figure A.6: Accumulating path information, stored in the GP Label Stack. Here, Entity 1 establishes a GP to Entity 9.

To be able to accumulate this information, we introduced a so-called *Label Stack*. The Label Stack is a simple stack of 4-tuples (called *GP Labels*):

GP Label := (source Entity, next hop Entity, list of Capabilities, protocol specific data) .

The *list of Capabilities* carries the Capabilities required by the source Entity in order to successfully establish the requested GP. This GP is then established by utilizing other Entities inside the same node within other Compartments, e.g., to establish an application GP some kind of data stream transport GP has to be established. These servicing Entities must satisfy the Capabilities requested by the source Entity, otherwise they cannot be utilized in the GP creation process. The *protocol-specific data* field is needed to enable protocols to store additional information, e.g., data needed to perform a forwarding decision or security information.

Note. The *next hop* field always carries the next Entity to contact during the GP creation process (an *intermediate destination*), while the overall destination Entity will be extracted from the *protocol-specific data* field within the top GP label.

The Label Stack handling only inspects the top element, acting on it as follows: If the top element is a *ResolutionCompartmentName*, then start asking the Core for a resolution within the Node Compartment. When receiving the answer, pop the *ResolutionCompartmentName* which is on top of the Label Stack and push the resolution on it, i.e., replace the top of the stack with the resolution. Afterwards start over with the Label Stack handling. For a detailed examination of *ResolutionCompartmentName*, see Section A.2.3.

In all other situations, the Entity's names will be compared to both names stored in the top label on the Label Stack. For the following description, let *A* denote an arbitrary Entity.

A's name(s)	stack size > 1	stack size = 1
equal(s)		



source	The <i>next hop</i> has to be determined based on the data stored in the <i>protocol specific data</i> field. Afterwards, the top GP label will be updated with this information. If possible, the packet will now be forwarded directly to the next hop Entity (e.g., by using an existing GP). Otherwise, a servicing Entity (that has to be determined by using the GP's list of required Capabilities) gets the packet.	
intermediate destination	Essentially the same as in the first case in which <i>A</i> is the source Entity.	
destination	<i>A</i> is the destination Entity as indicated by the top label. Hence, removing the top label yields a new label wherein the <i>next hop</i> Entity E_n lives within <i>A</i> 's Node Compartment. Therefore, a resolution for E_n 's Node Compartment name will be triggered and afterwards the packet will be handed over.	<i>A</i> is the overall destination Entity, i.e., the destination for the initial GP creation. Therefore, the GP creation will now be confirmed.

A.2.4.2 Instantiation and Removal

The creation of a new GP is comprised of several steps. But before the *CreateGPPacket* can be forwarded to the next Entity (and in the end to its overall destination), it has to be figured out how to reach the next hop Entity.

Therefore, let's assume Entity A_1 , living in Compartment C_A (non Node Compartment), is going to send a message to Entity A_2 , (living within the same Compartment C_A). Furthermore, assume that A_1 and A_2 are not yet connected by a GP. Denote by P the capabilities (see Section A.2.2 for an explanation) required by A_1 in order to send the packet. Then, in general, the following steps have to be performed in order to gather all the information needed to reach Entity A_2 and finally establish the GP.

1. Ask the Core for a resolution of A_2 with required capabilities P regarding the resolution(s).
2. Assume the Core provides a list $L := \{(N_1, E_1, P_1), \dots, (N_k, E_k, P_k)\}$ of *NodeCompartmentName-Entity-Capability* triples, i.e., Entity E_i offers Capability P_i with $P \subseteq P_i$, whereas N_i names an Entity, living in the same node as A_1 . Furthermore assume that A_2 does not live in the same node as A_1 . Now, an Entity out of L will be selected, acting as a servicing Entity, forwarding the *CreateGP-Packet*. W.l.o.g., let (N_1, E_1, P_1) be the selected triple.
3. The Label Stack of the *CreateGPPacket* will be updated with the selected resolution, and a Hook between A_1 and N_1 will be created.
4. The packet will be send by the A_1 to N_1 over the formerly created Hook. N_1 now acts as described in Section A.2.4.1, i.e., updates the Label Stack and also tries to figure out how to reach the destination, being part of the top label on the Label Stack.

After reaching the GP's destination, namely A_2 , an *EstablishedGPPacket* will be send by A_2 to A_1 , confirming the GP creation. In A_1 a callback method will be invoked, indicating the status of the GP creation process (*success* if done, *failure* if something went wrong).

On the other hand, removing an existing GP G works just by sending a special packet, called *ReleaseGP-Packet*. Conceptually, underlying GPs should also be released if they are obsolete because of G 's removal. The releasing of underlying GPs is currently not implemented since it would require at least a basic version of an *Introspection API*.

A.2.4.3 Relocation

Triggering a GP relocation (i.e., starting to move an existing GP from one Entity to another) is implemented for two situations as shown in Figure A.7.

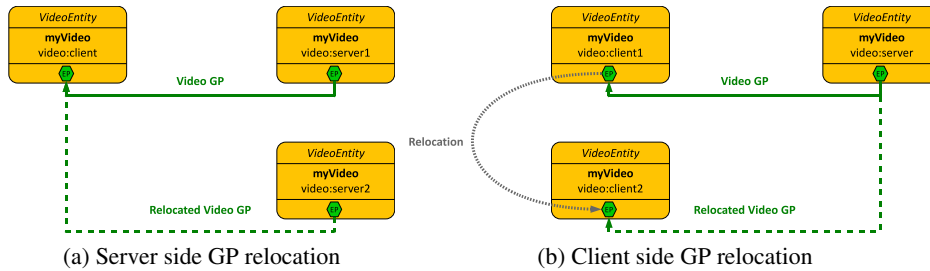


Figure A.7: Situations the GP relocation is implemented for

For the upcoming discussion, denote by C the client and S_i server i in situation as shown in Figure A.7a. Accordingly, denote by S the server and C_i client i in situation shown in Figure A.7b.

Server Side GP Relocation

As shown in Figure A.7a, server S_1 triggers a relocation of GP (S_1, C) to server S_2 , forming a modified GP (S_2, C). The relocation process can be broken up into the following sequence of steps:

1. S_1 's Endpoint will be serialized – or more precisely its status. This is needed for a seamless relocation of the GP. Developers have to implement it by overwriting two dedicated methods. For further details, see Section A.3.2. Afterwards, the serialized Endpoint will send to server S_2 along with information about the relocation, i.e., the involved Endpoints and Entities.
2. S_2 starts a GP creation to C , but instead of creating a clean Endpoint it initializes a new one with the received serialized Endpoint of S_1 . This especially transfers the status of the communication between S_1 and C to S_2 (at least the portion S_1 is willing to share).
3. C receives the GP creation request from S_2 and confirms it.
4. After the GP between S_2 and C is established, a *RelocateGPPacket* will be sent by S_2 to C , updating the newly created GP to be essentially a relocated version of the old GP between S_1 and C .
5. C receives the *RelocateGPPacket*, sent by S_2 , and updates its old Endpoint (for the GP from S_1) and EndpointTable accordingly. The new Endpoint, namely the one for the GP from S_2 , will be deleted since the old one is now an exact copy of it. It's necessary to keep the old Endpoint instead of the new one, because C itself might hold several references to the old Endpoint, but the relocation process should be transparent for all participants.
6. After successfully establishing the new GP between S_2 and C , the old one, namely the GP between S_1 and C , will be released by S_1 .

Note. The relocation information especially contains a **unique** identifier of C 's Endpoint, associated with GP (S_1, C). C 's old Endpoint has to be determined somehow (this is where the unique identifier is used) because it has to be updated with the status of C 's new Endpoint (see step 5 above for details).

One conceptually existing ingredient is missing in the currently implemented server side relocation process:

- Cleanup of servicing GPs. When relocating a server side GP from S_1 to S_2 some underlying GPs might become obsolete and therefore should be removed. In order to decide whether an underlying



GP (or *servicing GP* if you will) became obsolete an (at least basic form of) *Introspection API* is needed. It allows to ask if a specific GP is currently in use or if it can be safely removed.

Client Side GP Relocation

As shown in Figure A.7b, client C_1 triggers a relocation of GP (S, C_1) to client C_2 , forming a modified GP (S, C_2) . The relocation process can now be broken up into the following sequence of steps:

- Send a *RelocateGPPacket* with the Endpoint ID and the serialized Endpoint, associated to the GP that shall be relocated, from C_1 to S .
- Initiate a GP creation from S to C_2 and wait till C_2 confirms it.
- S sends a *RelocateGPPacket* to C_2 , containing the formerly received serialized Endpoint.
- C_2 receives the packet and updates the Endpoint, associated to the GP between S and C_2 , with C_1 's old Endpoint data.
- After successfully establishing the new GP between S and C_2 , based upon the old GP between S and C_1 , the old one will be released by S .

Two conceptually existing ingredients are missing in the currently implemented client side relocation process:

- Backtracking. Assume the path of the old GP between S and C_1 is of the form $S =: E_0 \rightarrow E_1 \rightarrow E_2 \rightarrow \dots \rightarrow E_n \rightarrow C_1$. When initiating the GP relocation, an index $0 \leq i \leq n$ has to be determined such that (a) there exists a path from E_i to C_2 and (b) the index i is maximal with this property. This means we are only going back on the old path as far as needed and keep the rest (namely $S \rightarrow E_1 \rightarrow \dots \rightarrow E_i$) of it. It's a heuristic that hopefully conserves time and bandwidth.
- Cleanup of servicing GPs. See the open issues part of explaining the server side relocation for this issue.

A.2.4.4 Hooks & Ports

Hooks and Ports are specializations of the more general terms of Generic Paths and Endpoints, as described in Sections A.1.2.4 and A.1.2.5. Hooks are only used for communication inside the Node Compartment and can be created between any two Entities inside the same Node Compartment.

Hooks provide a packet oriented, single-hop, end-to-end Generic Path, which guarantees that the packets are received without changes and in the same order as sent on the sender side. They are implemented by using platform-specific communication methods for exchanging the data between Entities.

Note. In our implementation, Hooks and Generic Paths are still implemented differently and establishing and releasing of Hooks is done by using different control packets. Nonetheless, this will change in the future and only has historic reasons.

A.3 Developing with the ForMux/GP Testbed

A.3.1 Installing & Building

A.3.1.1 Prerequisites

All platform require the following packages/libraries to be installed:



- GCC Build Toolchain
(Debian/Ubuntu: build-essential)
- Boost Library 1.37 or newer (<http://www.boost.org>)
(Debian/Ubuntu: libboost1.37-dev, libboost-thread1.37-dev)
- log4cpp (<http://log4cpp.sourceforge.net>)
(Debian/Ubuntu: liblog4cpp5-dev)

If you want to build from a HG snapshot (and not from a tarball release) you also need

- Autoconf & Automake
(Debian/Ubuntu: automake, autoconf, libtool, pkg-config, perl)

To run the prototype using OMNeT++ you also need

- OMNeT++ 4.0 or newer (OMNeT++ 3.X is not supported!)

To build the prototype on POSIX you also need

- Additional Boost Libraries 1.37 or newer (<http://www.boost.org>)
(Debian/Ubuntu: libboost1.37-dev, libboost-system1.37-dev, libboost-date-time1.37-dev, libboost-regex1.37-dev, libboost-thread1.37-dev, libboost-program-options1.37-dev)
- Google Protobuf (<http://code.google.com/p/protobuf/>)
(Debian/Ubuntu: libprotobuf-dev, protobuf-compiler)
- GTK
(Debian/Ubuntu: libgtk2.0-dev)

If you intend to run the unit tests you also need

- Google-Test (<http://code.google.com/p/googletest/>)
(Debian/Ubuntu: libgtest-dev)
- Valgrind
(Debian/Ubuntu: valgrind)
- GDB
(Debian/Ubuntu: gdb)

A.3.1.2 Building with OMNeT++

If OMNeT++ 4.0 or newer and all prerequisites are installed correctly, the prototype can be built as follows:
(Be sure to include the OMNeT++ bin directory in your PATH environment variable and the lib directory in the LD_LIBRARY_PATH! Alternatively, you can use the setenv script in the OMNeT++ install directory.)

1. Run `./configure --with-target=omnet`
2. Import folder formux as a project (File>Import>Existing Projects into Workspace)
3. Build the project (Project>Build Project)
4. Run simulations by creating your own ini files or using an existing one from src/Omnet. (Right-Click and Run as>OMNeT++ Simulation)

A.3.1.3 Building on POSIX

If all prerequisites are installed correctly, the prototype can be built as follows:

1. Run `./configure --with-target=posix`



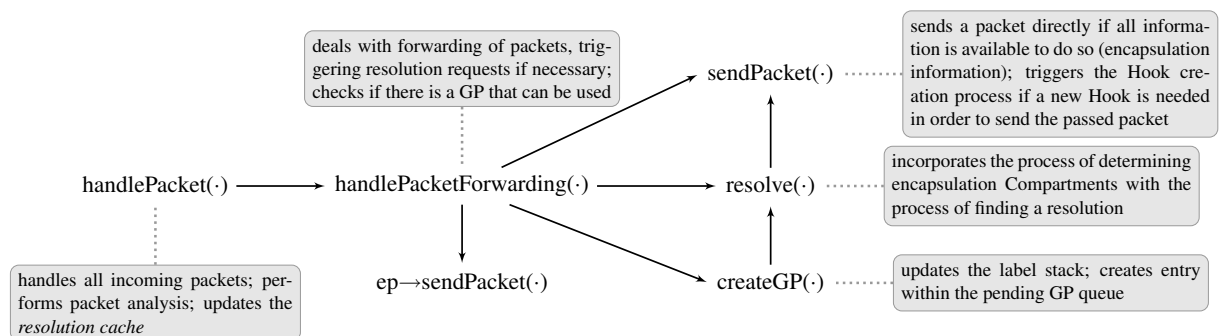
2. Run make

Section A.3.3 gives a brief introduction on the existing demo scenarios, which can now be started. Additionally the following commands may be useful:

- `make check` - Run all tests
- `make testname.check` - Run a specific test
- `make testname.gdb` - Execute a specific test under GDB
- `make check-valgrind` - Execute all tests under Valgrind
- `make testname.valgrind` - Execute a specific test under Valgrind
- `make check-coverage` - Measure code-coverage of all tests

A.3.2 API along with exemplary implementations

Before writing Entities, one must be aware of how packets are being processed within the prototype. That's of high importance because some methods have to be overloaded in order to implement the functionality one needs. The following figure gives you a sense of where the functionality lives and how information (i.e., packets most of the time) will be processed.



Let m_1, m_2 be two methods. Then the expression (visualization) $m_1 \rightarrow m_2$ means that m_1 calls m_2 and therefore passes information to m_2 .

Note. The graph drawn is a simplified view on the things that are happening in real. For example, callback methods are omitted, and `resolve(.)` does not call `sendPacket(.)` directly, but triggers a callback that eventually calls `sendPacket(.)`.

A.3.2.1 Creating an Entity

Create a new class, derived from `Entity` (or any subclass of it that you find adequate).

```

1 #include <ForMux/Core/ForMux.h>
2 #using namespace ForMux;
3 class MyEntity : public Entity

```

At least implement the three standard constructors, provided by every entity.

```

1 public:
2     // Let the Entity join the Node Compartment with the passed name.
3     MyEntity (const Name* nodeCtName) : Entity (nodeCtName) {
4         // ...
5     }
6

```



```
7 // Let the Entity join the Node Compartment as well as some other
8 // Compartment(s) with the corresponding name(s)
9 MyEntity (const Name * nodeCtName, const Compartment* compartment,
10          const Name* name) : Entity (nodeCtName, compartment, name) {
11     // ...
12 }
13 MyEntity (const Name * nodeCtName,
14          const vector<pair<const Compartment*, const Name *> >* names) :
15     Entity (nodeCtName, names) {
16     // ...
17 }
```

Overwrite `registered()` if you want to get informed when the Core has become aware of your Entity's existence. Very handy if you want to start some actions immediately after you have been recognized within your node (Compartment).

```
1 protected:
2     virtual void registered (bool success);
3     virtual void unregistered ();
```

If you want to run your Entity everywhere (e.g., in OMNeT++, native, in tests), you have to implement `createInstance` which basically is a constructor.

```
1 public:
2     static Entity * createInstance (const Name * nodeCtName,
3                                     const vector<pair<const Compartment*, const Name *> >* names) {
4         return new MyEntity (nodeCtName, names);
5     }
```

Deciding if a given name addresses a certain Entity cannot be done without internal knowledge about the name's structure. Just as an example, consider multicast names. Therefore, this name-matching decision has to be implemented in some sort of method (here: `amI`).

```
1 protected:
2     virtual bool amI (const Name& name);
```

You have to overwrite `createEndpoint()` to instantiate the correct Endpoint type based on the data available within the `GPLabel`. It is recommended to pass the serialized Endpoint in your Endpoint's constructor. The parameter `usableAfterCreation` must be set properly by you! If set to `true`, the created Endpoint can be used right after the method returns to send packets.

For example, consider the client-server paradigm. When the client initiates a GP creation to the server it creates an Endpoint, tries to establish a GP to the server and if everything went well, the client's Endpoint will henceforth be able to send packets over its associated GP. Setting `usableAfterCreation` to `true` on the client side would indicate that the Endpoint could be used even before the GP would be finally established.

As a side remark, don't insert the created GP manually into the `EndpointTable`; the generic code does take care of it.

```
1 protected:
2     virtual Endpoint::pointer createEndpoint (const GPLabel & lbl,
3                                               const pair<uint8_t *, size_t>& serializedEP,
4                                               bool & usableAfterCreation) {
5         // ...
6         if (amI (lbl.source)) usableAfterCreation = false;
7         else if (amI (lbl.destination)) usableAfterCreation = true;
8         // ...
9     }
```



By overwriting `getSuitableEndpoint()` you can (and should) decide which Endpoint to return based on the passed GPLabel. Search for example the *EndpointTable* to find the right Endpoint.

```
1 protected:
2     virtual Endpoint::pointer getSuitableEndpoint (const GPLabel & lbl);
```

(optional) Overwrite `getSenderName()`. It deals with the problem that some names of your Entity should not be passed to other Entities, e.g., a broadcast address you (surely) could be reached over, but on the downside resulting in a much higher network load if someone uses this name to send you packets. Those names should be filtered out in here. Exemplary implementation:

```
1 protected:
2     virtual Name getSenderName (const Compartment & ct) {
3         if (ct == Compartment (any_of_my_compartments)) {
4             vector<Name>* names = AbstractEntity::getSenderName (ct);
5             names->erase (remove (names->begin (), names->end (),
6                                 a_name_of_mine)), names->end ());
7             // return one of the remaining names
8         } else {
9             return AbstractEntity::getSenderName (ct);
10        }
11    }
```

(optional) Overwrite `forwardingDecision()`. As indicated by the method's name, your protocol specific forwarding decisions (based on routing tables or whatever) should be implemented (or at least called) in here. It's recommended to encode your forwarding information in the *protocol specific data* field, provided by a GPLabel, and therefore extract it from there. If you don't overwrite this method, `lbl.destination` will be returned by default.

```
1 protected:
2     virtual Name forwardingDecision (const GPLabel & lbl) {
3         return apply_some_forwarding_rule(name);
4     }
```

(optional) Overwrite `handlePacketForwarding()`. Deals with packets for which this Entity acts as a servicing Entity, i.e., the Entity is part of a servicing Compartment. For example, decisions like sending a packet over a cable or via WiFi could go in here. The ID of the Port over which a packet has been received is stored in `destinationPortID`.

```
1 protected:
2     // @returnval true iff we dealt with the packet.
3     virtual bool handlePacketForwarding (const ForMuxPacket *p,
4                                         PortID destinationPortID) {
5         // ...
6     }
```

OMNeT++ Specific Implementation Details Please note: We assume that you are already familiar with OMNeT++. Otherwise please read its basic introduction first.

First, create a network by properly laying out `.ini` and `.ned`-file(s). In order to initialize the "GP enabled" network nodes with names you have to fill `**your_node.os.initString` with content.

```
1 **your_node.os.initString = "
2     entity_A_class_name:omnet_specific_entity_A_name,
3     compartment_A_1:name_in_compartment_A_1,
4     ...,
5     compartment_A_m:name_in_compartment_A_m;
```



```
6     entity_B_class_name:omnet_specific_entity_B_name,
7     compartment_B_1:name_in_compartment_B_1,
8     ...,
9     compartment_B_n:name_in_compartment_B_n;
10    ..."
```

(optional) If you need resolvers in your network, the `*.resolveConf`-string is the place where you should insert them.

```
1 **.your_node.core.resolveConf = "
2     resolver_1_compartment:
3     resolver_1_name,
4     resolves_into_compartment_1,
5     compartment_reachable_by_1:
6     name_reachable_by_1"
```

Entities are sending packets over servicing Compartments (also called "encapsulation Compartments"). Since we do not have capabilities yet, the `encapRules`-string determines where Compartment A sends its packets over, i.e., that Compartment A chooses Compartment B as a servicing Compartment.

```
1 **.your_node.core.encapRules = "
2     compartment_1,servicing_compartment_for_1;
3     ...;
4     compartment_n,servicing_compartment_for_n"
```

A.3.2.2 Handling Resolutions

Identifying an Entity by a given (set of) Name(s) is essential, at least for answering *Who Has*-requests properly. Since Names are Compartment-specific, every Entity has to decide by its own if a given set of Name(s) matches any one of its Names. You should implement your Name-matching in `amI`.

```
1 protected:
2     virtual bool amI (const Name& name);
```

Incoming resolutions might contain more than you really need. Therefore, by implementing a generic callback method, satisfying the `ResolveCallback` signature, you can filter and prioritize resolutions by altering the received list of resolutions.

```
1 public:
2     void _abstractResolveCallback (const Name& toResolve,
3         const list<triple<pair<Name, NodeCompartmentName>,
4             CapabilityList, uint32_t> >& resolutions) {
5         // filter the list of resolutions such that it only contains items
6         // you want to keep. Similarly, you can add resolutions that you are
7         // already know.
8     }
```

An Entity can be enabled to act like a resolver, i.e., it stores resolutions found so far and answers to requests from other Entities. In this case, `resolve` has to be overwritten in order to ask the Entity's local "resolution cache" before asking, for example, the Core.

```
1 protected:
2     void resolve (const Name& toResolve, const ResolveCallback& callback);
```



A.3.2.3 Capabilities

```
1 Capability::pointer cap = Capability::create(  
2     <unique capability identifier>,  
3     <property name l>, Property::value_t::<TYPE>, param a1, ..., param an,  
4     ...,  
5     <property name k>, Property::value_t::<TYPE>, param z1, ..., param zm,  
6     NULL  
7 );
```

The number and kind of values to pass for each property are depending on the type of property you specify. Be careful and only pass appropriate parameters, otherwise this method might produce some non-predictable results.

Note. The capability identifier is a simple string representing a point within the hierarchy to insert this capability in, e.g., `common.transfer`.

Note. The last parameter to `Capability::create` always has to be `NULL`. Don't forget it, otherwise the method will act in a non-deterministic fashion.

A.3.2.4 Creating an Endpoint

Create a new class, derived from `Endpoint` (or any subclass of it that you find adequate).

```
1 #include <ForMux/Core/Endpoint.h>  
2 #using namespace ForMux;  
3 class MyEndpoint : public Endpoint
```

Implement both constructors, one only taking a pointer to its hosting Entity, the second one additionally taking a serialized Endpoint as a parameter. The second constructor therefore acts as a deserialization function, i.e., it should deserialize the serialized Endpoint and initialize the new one with this data. Attention: Always keep in mind to check the `data.first` pointer if it equals `NULL`. In this case, just act like the first constructor to guarantee well-behavedness.

```
1 public:  
2     MyEndpoint (AbstractEntity * host) : Endpoint (host) {  
3         // ...  
4     }  
5     MyEndpoint (AbstractEntity * host,  
6         const pair<uint8_t*, size_t>& data) : Endpoint (host) {  
7         if (data.first != NULL && data.second > 0) {  
8             // ...  
9         }  
10        // ...  
11    }
```

In addition to the deserialization constructor, a serialization method is needed. Everything a newly created Endpoint needs to pick of and resume the other Endpoint's work should be serialized, i.e., its status.

```
1 public:  
2     virtual pair<uint8_t*, size_t> serialize () const;
```

Our "protocol" for sending and handling packets is it to wrap everything into a *DataPacket*. Therefore, before a packet, handed over to an Endpoint by calling `endpoint->send*Packet()`, can get on its way out, it has to be encapsulated. The *DataPackets* payload will be a serialization of your data, e.g., a header with standard payload in the case of TCP. Hence, the *DataPacket* acts as a *container* for your data.



When, on the other hand, a packet has been received, it is wrapped in a *DataPacket*. To have a chance to perform some side actions, `decapsulatePacket()` has to be overwritten, at least with the code for unpacking the passed packet.

```
1 public:
2     virtual DataPacket * encapsulatePacket (const ForMuxPacket * packet) {
3         pair<uint8_t *, size_t> serializedPacket = packet->serialize ();
4         DataPacket * p = new DataPacket (packets_destination_name,
5             serializedPacket.first, serializedPacket.second);
6         free (serializedPacket.first);
7         return p;
8     }
9     virtual ForMuxPacket * decapsulatePacket (const DataPacket * packet) {
10        if (packet->payload == NULL || packet->length == 0) {
11            ForMuxPacket * p = new ForMuxPacket ();
12            FORMUX_WARNING ("ill-formated payload");
13            return p;
14        }
15        ForMuxPacket * p = ForMuxPacket::deserializePacket (
16            packet->payload, packet->length);
17        return p;
18    }
```

Endpoints, as part of the GP architecture, should at least be able to send *ControlPackets*. In addition to that, *DataPackets* should be sendable somehow as the most general form of packets (they can carry any kind of information) in our prototype. A template for both methods looks as follows:

```
1 public:
2     virtual void sendDataPacket (const DataPacket * packet) {
3         DataPacket * pSend = this->encapsulatePacket (packet);
4         // ...
5         ((AbstractEntity *) host)->sendPacket (... , pSend);
6     }
7     virtual void sendControlPacket (const ControlPacket * packet) {
8         // could be the same as in the method above
9     }
```

When relocating a GP, old Endpoints will be updated by newly created ones. For a more detailed explanation, see Section A.2.4.3. The `update()` method should therefore perform a deserialization and initialization, analogous to what happens in the deserialization constructor.

```
1 public:
2     virtual void update (const pair<uint8_t *, size_t> & data);
```

A.3.2.5 Generic Paths

The following subsections show how to implement and trigger the most basic (not to be confused with trivial!) operations like creating, relocating and removing GPs, aside with comments about *do's* and *dont's*.

Instantiating GPs Start the Generic Path creation by calling `createGP()` with suitable parameters. Since you might want to get informed when the Generic Path creation has been finished, you should also implement a callback method. It tells you the status (success, failure) and you side's Endpoint.

```
1 #include <ForMux/Core/ForMux.h>
2 #using namespace ForMux;
3 public MyEntity : public Entity {
```



```
4 // ...
5 void someMethod() {
6     // ...
7     AbstractEntity::createGP (&ourCompartment, &myName,
8         &destinationName, boost::bind(
9             &MyEntity::_createGPCallback, this, _1));
10    // ...
11 }
12
13 private:
14 bool _createGPCallback (EndpointTable::Entry::pointer entry) {
15     // ...
16     // Return true to remove callback from the callback queue
17     return true;
18 }
19 // ...
20 }
```

Releasing GPs Simply call `releaseGP()` to start releasing an existing GP. The Endpoint used to initiate the process (and therefore the whole GP) shall be used no more when the removal was started.

```
1 #include <ForMux/Core/ForMux.h>
2 #using namespace ForMux;
3 public MyEntity : public Entity {
4     // ...
5     void someMethod() {
6         // ...
7         releaseGP (an_endpoint, boost::bind (
8             &MyEntity::_releaseGPCallback, this, _1));
9     }
10
11 private:
12 void _releaseGPCallback (bool success) {
13     // ...
14     // Return true to remove callback from the callback queue
15     return true;
16 }
17 }
```

Relocating server side GPs Simply call `relocateServerSideGP()`. Conceptually the passed Endpoint will automatically be locked, i.e., it won't be used anymore to transport data. Of course, that's necessary to ensure that the new Endpoint on the new server will pick up the connection just at the right "position".

```
1 #include <ForMux/Core/ForMux.h>
2 #using namespace ForMux;
3 public MyEntity : public Entity {
4     // ...
5     void relocateServerSideGP (Endpoint::pointer ep, const Name& newSvr,
6         const Name& client, const RelocateGPCallback& cb) {
7         AbstractEntity::relocateServerSideGP (ep, newSvr, client, cb);
8     }
9 }
```



A.3.3 Included Demo Applications

Our prototype comes with two ready to use demos included.

A.3.3.1 ResolvePingTool

The ResolvePingTool can be found at `src/Posix/ResolvePing/formux-resolve-ping`. It is a simple tool to test broadcast resolution and pings via GPs. It requires the following parameters:

- `--verbose`: print debug messages on stdout (optional)
- `--node-compartment-name <string>`: Node Compartment name of the ResolvePingEntity
- `--this-name <string>`: name in ResolvePing Compartment
- `--ip-address <ip address>`: local IPv4 address
- `--port <port number>`: port

A.3.3.2 Video-GPs

The Video-GP demo can be started via the NetInfEntity with `integration/NetInfEntity/formux-netinf-entity`. It requires the following parameters:

- `--verbose`: print debug messages on stdout (optional)
- `--ip-address <ip address>`: local IPv4 address
- `--netinf-port <port number>`: Port for local socket to NetInf (control socket)
- `--servermode`: start in server mode
- `--testmode`: start in test mode
- `--nivname <string>`: (partial) NetInf name for video-transfer
- `--video <filename>`: video file name/path (default is `./bbb.avi`)

There are two different ways to use the NetInfEntity either standalone, without a running NetInf node or integrated with a running NetInf node. In both cases it is possible to relocate the video GP.

For running the standalone mode:

- set parameter `--testmode`
- on the first server node set `--servermode` and `--nivname server`
- on the second server node (for GP relocation) set `--servermode` and `--nivname server2`
- on the client node set `--nivname client`
- set all other required parameters and check if a video file is present on all servers

For running the integrated mode:

- on all server nodes set `--servermode`
- do not set `--nivname` on any node
- set all required parameters and check if a video file is present on all servers

All names will be set from the NetInf node itself! It is also possible to mix both modes, i.e for running the servers in standalone mode and only the clients in integrated mode.

The GP relocation can be triggered from the NetInf node or with the `relocate-util`, which requires the following parameters:



- `--netinf-port <port number>`: Port for local control socket
- `--target <string>`: target name for the relocation



Document:	FP7-ICT-2007-1-216041-4WARD/D-5.3		
Date:	June 14, 2010	Security:	Public
Status:	Final	Version:	1.0

B GP-based Transport Services Selection for LQO-based Routing

B.1 Introduction

In this section we use GP-architecture to model sophisticated transport services in a wireless mesh network (WMN). In doing so, *compartmentalisation* of functions (i.e. presenting them as separate compartment-like blocks) is the modelling vehicle that is applied in particular to the routing function of the mesh network. This may support a heap-based approach to dynamically set-up appropriate stack of functions in the network and contrasts with traditional approaches where preconfigured stacks are typically used. We assume functional constructs provided by the UPB framework A, and the example is based on LQODV routing protocol, both having been developed within 4WARD.

B.2 LQODV Protocol - General Idea

Figure B.1 contains a snapshot of routes selected by Link Quality Ordering-based Distance Vector (LQODV) routing protocol. The general rule is that a node distributes to its neighbours its cumulative distance towards a destination node (the component used by classical schemes) along with the metric value of its outgoing link towards that destination node (additional component of LQODV). Thus, the previous node can evaluate LQO component comparing its own metric of outgoing link with the metric of the outgoing link of the next node. Following the example in Figure B.1 we can see that node *I5* disseminated a routing report about its distance to *D* and its metric value of its outgoing link towards *D*. For in-detail explanation how distances to destinations are calculated and routes are selected according to LQO approach refer to Annex C.2.7 of D5.2 deliverable of 4WARD project.

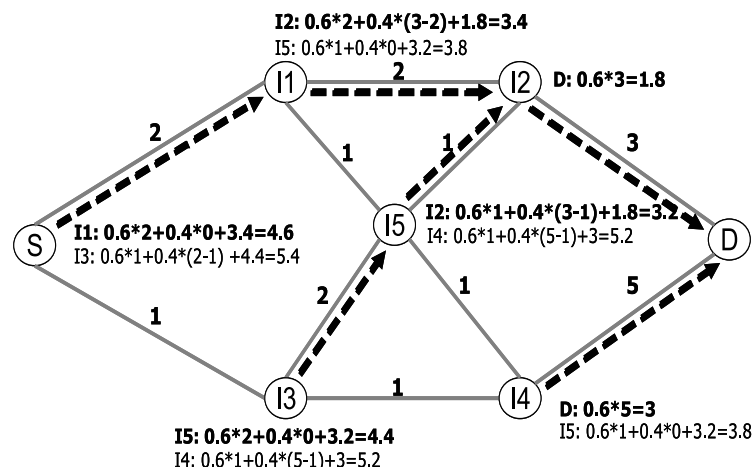


Figure B.1: LQODV protocol - example

It is obvious that working with dynamic and adaptive link metric systems the routing reports ought to be sent periodically and in a timely fashion. So, the Routing Update Interval (RUI) is an important parameter of the routing protocol that impacts signalling overhead and network performance. Simulation experiments of our implementation of LQODV protocol in OmNet++ environment show that setting optimal value of RUI parameter is difficult. The problem stems not only from the known trade-off between protocol signalling overhead and network performance (the shorter RUI the larger the overhead but also faster routing state propagation), but also from the fact that report messages, usually broadcasted in WMN, can be lost without the sender knowing about this. Such losses may result in out-of-date routing data which may degrade network performance.

It is easy to conceive that a routing protocol can use various (available) transport services to disseminate its messages, and based on the features of those services it can adjust its parameters such as RUI or even change its dissemination policy.

B.3 Modelling using GP-architecture and UPB's framework

Our LQODV implementation assumes that WMN network layer employs IP forwarding and each node sends periodic reports about its distance to a destination and metric value of its outgoing link towards the destination. The reports are sent to one-hop neighbours (in terms of radio-visibility).

Let us assume that LQO report sending policy depends on the transport service used to contact one-hop neighbours. Also, let there be confirmed and unconfirmed one-hop neighbour notification service instances available that can be used by LQODV.

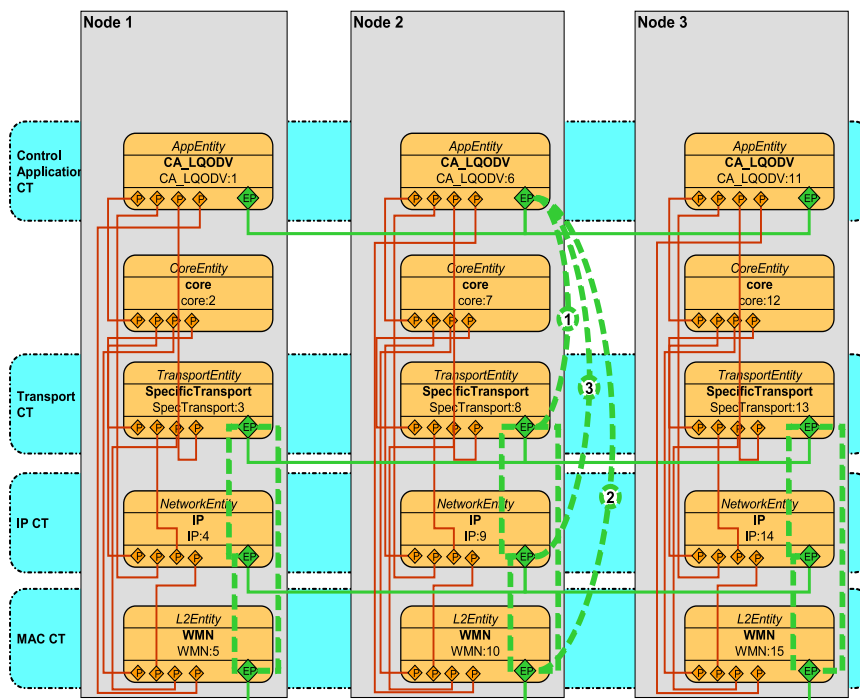


Figure B.2: LQODV protocol vs. GP Architecture

Following the example in figure B.2 and applying Generic Path Architecture framework we model LQODV process as an entity operating in a Control Application Compartment that populates the routing tables of



nodes in a specific forwarding compartment (IP CT in our case). Therefore, LQODV entity has internal hook to the IP Entity. In order to contact LQODV entities in neighbouring nodes, such LQODV entity needs appropriate transport service. Such LQODV entity may request that the node Compartment (its Core Entity) provides a list of Compartments through which the neighbouring LQODV entities are reachable. Additionally, the requesting entity may specify the requirements related to the service it needs (e.g. confirmed or unconfirmed delivery).

Depending on the particular choice of the one-hop notification service, LQODV entity can create a GP realised by either (1) Specific Transport entity or (2) MAC entity (see figure B.2). Depending on the realisation (cases 1 or 2) the LQODV entity can adjust its routing information dissemination policy. In case (2) LQODV entity broadcast its messages using the broadcast mode of MAC. Here, LQODV entity sends its routing reports every RUI (say 15s) without taking care of the result of the transmission. Of course, the appropriate value of RUI has to be precalculated somehow which is however beyond the scope of the example.

Case (1) requires a specialised transport service that is much more sophisticated than the simple broadcast offered by MAC CTs. The EP of Specific Transport entity takes for both broadcasting the packets received from the upper entity (possibly using underlying services of MAC) and collecting the acknowledgements, the latter being used to confirm successful delivery of routing reports to the routing entities. In this way LQODV entity is able to control the right value of RUI based on routing report delivery ratio. Again, the GP of Specific Transport entity can be created based on MAC CTs.

The transport service can also be provided by IP CTs using IP broadcast - shown as case (3) in the figure. Similarly as in case (1) above, it is up to IP entity whether or not it will use MAC broadcasting capability for efficiency reasons.

We notice that the use of GP architecture as sketched above is proposed mainly for implementation purposes - from purely functional point of view the routing function is still present in (functionally) WMN compartment. However, modelling and implementing communication between processes (entities) in such a way allows for more flexibility in the implementation phase than using traditional, rigid protocol stack layout. As in the example, it makes it easier to support various communication regimes when developing WMNs that can adapt to actual connectivity conditions. We note that a possible application of this general approach can be found in WP4 Deliverable D-5.4 (In Network Management for Generic Path) where multi-protocol routing framework is proposed.



Document:	FP7-ICT-2007-1-216041-4WARD/D-5.3		
Date:	June 14, 2010	Security:	Public
Status:	Final	Version:	1.0

C Dynamic Functional Composition at the Mediation Point

C.1 General

We start from the premise that in the future Internet there will be strong demand for a capacity to modify dynamically the data plane or control plane, if possible at runtime in response to emerging conditions. Currently, ad-hoc dynamic tuning of existing functionality is being advocated in different cross-layer optimisations and in the past a number of frameworks have been proposed, mainly in active networking research, for enabling incremental extension of existing functionality. In both cases the degree of adaptability is rather limited.

In the GP model an adaptation facility for functionality in-band to (GP) service paths is advocated through the Mediation point. Our initial consideration of the Mediation point functionality involves a rather static and explicitly controlled binding capability between GPs. However, the Mediation point functionality, like the GP functionality, is provided in object-oriented terms through a class, the service of which can be extended and enriched to provide more sophisticated and elaborate mechanisms for its semantic services.

In this chapter we progress one step further and advocate a more dynamic operation for a mediation point that provides a holistic approach of adapting data or control plane functionality to satisfy different requirements and to serve orthogonal adaptation processes. We present a framework that is able to support dynamic composition, optimisation and customisation of in-path network services through dynamic selection and on-demand modification of network functions at the inter-stitches of generic paths. We have implemented and evaluated a prototype of this framework, but we have not integrated yet in the GP node architecture. The aim of this evaluation has been to explore the space of potentials for such a mechanism in terms of flexibility and performance.

Despite the high configurational capacity provided and the cost incurred by a user space implementation, we show that it is possible to sustain lightweight virtualisation and maintain near line-speed performance. This makes the proposed mechanism particularly attractive for virtual node environments.

The presented design is based on work on functional composition in the ANA [14] project. The main idea here is to show how the functionality of this (and other – see related work) adaptational facilities can be captured, generalised and abstracted in the GP model through the Mediation point construct, and used to enable in-path control of GP services. At the same time we aim to highlight how the 4WARD project and the GP model can help bring together research contributions from different infra-structure projects in area of future Internet engineering.

C.2 Functional composition framework

We first provide a high level overview of our prototype design and its main building blocks, while deferring the explanation of their detailed roles to subsequent sections. This provides context for the subsequent presentation of the fundamental concepts and components of our design.



C.2.1 System Overview

Figure C.1a presents a high-level view of the main building blocks of the prototype implementation of the functional composition framework (a full description of which is provided in [107]) and their interactions.

The *framework manager* operates at the control plane, and is responsible for creating and controlling instances of the *runtime service engines*, which in turn execute the data plane operation and provide a premise for lightweight resource virtualisation. At the same time the framework manager orchestrates access to runtime interfaces for customisation of service engines and for modifying the control flow graph and the respective service classifier, which is the main functional composition foundation. In other words the manager module implements a programmable network type of access to the functional composition (adaptation) process. It is access to this interface that indirectly enabled and encapsulated in a Mediation point abstraction.

As service functionality is decomposed in a set of functional blocks, for every runtime service engine instantiated within the functional composition framework a new (virtual) functional block is registered that may then be employed as a service module in the composition of more complex services. The co-existence of multiple simultaneous runtime engines is thus the main virtualisation premise enabling cooperation between multiple composed service.

A service engine classifier is created at instantiation time of a composite service using primitives provided by a *service composition plugin*. A plugin also provides primitives for generating a service graph from specifications in XML [108], and for subsequently enabling modification of the service composite (by means of installing and/or removing filters and access to event registers, as described in subsequent sections). The use of plugins therefore allows the flexibility to accommodate different modes of access to the service composition process and different algorithms for generating classification graphs. We will show in the evaluation section how the structure of a service graph can be optimised for trading flexibility versus performance.

Finally, the framework provides a client-side static library that implements the message passing communication interface to which a functional block is expected to link.

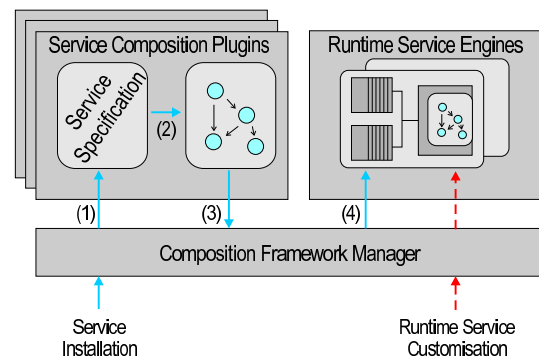
Next, we follow a bottom-up approach to explore the fundamental concepts of our design and show how the different building blocks come to existence and what purpose they serve.

C.2.2 Modularisation: Component based, message passing

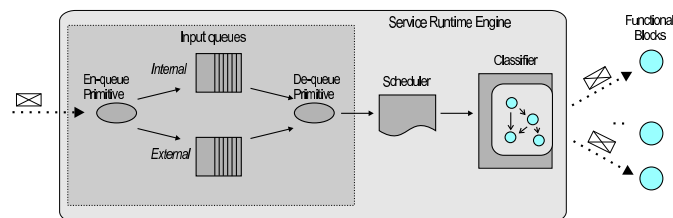
Modularity allows for separation and distribution of functional concerns and for individual parts of a composite service to evolve independently. Relating to this we have made two design and implementation decisions for leveraging functional composition.

The first regards the de-composability and organisation of service functionality in modules that represent discrete algorithmic (functional) building blocks of a service or protocol. Thus the atomic unit of adaptation is an opaque finite state machine implementing some data processing function (algorithm). We do not consider composition of services at the instruction level. In the GP node framework this is facilitated by means of distributing and attributing service functionality in *Entities*, that satisfy this exact requirement for modularity.

The second decision regards the communication (interface) among building blocks of a composed service. In order to allow dynamic insertion or removal of functionality in the composite, the different service components need to communicate asynchronously. At the same time in order to permit the re-use of algorithms at different locations of the service path without interface compatibility problems, a universal interface needs to be in place. This requirement is also satisfied in the GP node framework since entities communicate with each other by exchanging messages in a well specified way. We simply had to prescribe the semantics (message format) extending this interface as shown in figure C.1a).



(a) Main building blocks of the functional composition framework



(b) Block diagram of a service engine

In summary these two preconditions reinforce two goals:

- the Internet communications paradigm is adopted at the node level making network wide (distributed) composition attainable using one composition model, and
- it allows us, as we see later, to adopt a flexible composition model, founded on well established packet classification and metric based routing techniques.

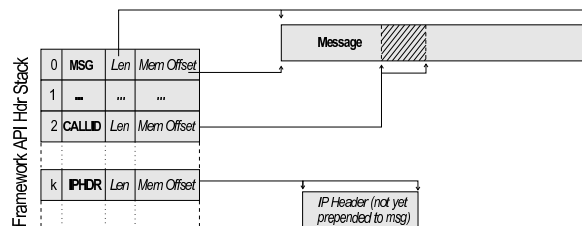
This meta-interface contains a number of type fields that hold:

- Topological/addressing information for heuristically assisting the routing of messages among Entities within the current composition (service) context.
- An identifier specifying a backtracking *report* channel for sending synchronisation and error reporting messages between Entities across a processing path.
- A *composition context* identifier, which specifies in which composite service a message is currently handled. While a message may traverse composition contexts, maintaining a stack of the traversed contexts allows multilevel composition, whereby one composite service may be encapsulated in another composite service (figure C.1c).
- An embedded call stack (*header stack*), which Entities in a composite service may use to exchange parameters and data. From a procedural programming perspective this field enables parameter passing between functions by storing them as headers in stack frames. From a network engineering point of view this field provides the traditional packet header stacking facility. A message is thus handled interchangeably within a node or across a network. A legacy network packet format can be easily produced from a message by stripping the meta-data before the header stack field.

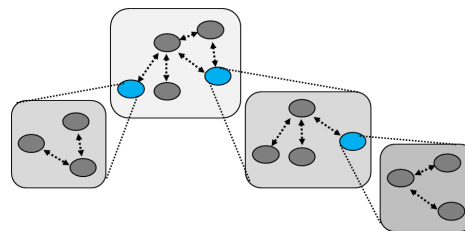
This inter-module message passing facility introduces a network communication paradigm at the node level, while on the other hand the use of this meta-format enables procedural programming semantics at the network level, thus taking advantage of both paradigms. In contrast, the lack of such sophistication in the inter-module communication of most active networking frameworks proposed to date has limited their compositional scope to a small set of simple applications.

Type	Addr. Info & Rprt channel	Service Context	Header Stack
Node / Net	{Rpt, Src, Dst}	0th composite service context	Fr. 0 : Msg Data
		1st composite service context	Fr 1 : Param data
		...	Fr 2 : Param data

(a) Message meta-format



(b) Header stack



(c) Multi-level composition

C.2.3 Composition model: Federating and organising functionality

In this model a composite service is constructed by superimposing service modules (Entities) on a *control flow* or *classification graph*, which provides a skeleton for the composition of the service. The nodes of the graph however, do not represent service modules but instead binding points where more than one service module may attach. The structure of the graph thus dictates possible data paths followed by messages across Entities of the service, but does not reflect a conclusive service structure. It represents one possible semantic organisation of its comprising modules. An example is presented later in figure C.1 for a graph that organises Entities to provide a legacy TCP/IP network subsystem.

At each node of the control graph there are *filters*, which are applied to traversing messages. There are three different types of filter.

- *Message processing filters*, which associate Entities to graph nodes and select which messages will be subjected to processing by the respective service module. An Entity may thus use one or more such filters at various nodes of the graph to “pull” messages.
- *Graph filters*, which interconnect graph nodes through runtime evaluated conditions. They thus dictate the topology of the classification graph at runtime.
- *Dynamic meta-filters*, which are responsible for activating other filters when predefined conditions are satisfied. By analogy to a *late-binding* process they provide a degree of runtime self-organisation of the composition process.

Message classification begins at an entry node of the graph and proceeds through constant evaluation of the encountered filters. After each processing step is completed the message returns for re-classification at the last graph node and proceeds with the next filter. The use of filters provides a polymorphic structure for the composed service, subject to dynamic runtime conditions and flow dynamics. Message processing filters determine which messages are subjected to processing, and graph filters determine the set of Entities that messages are presented to ¹.

The structure of the control graph may thus reflect a layered protocol stack structure (this is rather intuitive for composing functionality using the current layered philosophy), although there is no special expectation for being so.

¹Note that pipelined processing and multicasting are inherently supported in this model, without need for special support.



Filters consist of *filter rules* that describe one or more patterns of interest anticipated in a message, which will be handled according to the type of filter. The syntax of filter rules follows the abstract syntax $\langle \text{anchor point} + \text{offset}, \text{bit pattern} \rangle$, whereby an *anchor point* refers to an identifiable location in a message and *offset* is a bit-distance from the anchor point. Anchor points are fast accessible through indexing at the header stack. For example, the filter rule $\langle \text{foci}\{\text{CALLID}\}+0, 0\text{xFF3A20} \rangle$ would refer to the indexed location (header stack) of the CALLID field in the message (figure C.1b) and for matching the value 0xFF3A20 . Multiple filter rules can be combined to form more complex expressions of disjoint patterns in a message.

Graph filters may additionally consist of *event masks* that test the activation of bit flags in a software event register at each graph node. This simple facility allows us to multiplex up to 256 external events with an 1-byte register, the semantics of which are not predefined so as to allow customisation of the classification process (and therefore service composition) based on external conditions which do not need to be interpreted in the context of the composition process.

Finally, since the evaluation of filters can be a heavyweight computational task, to avoid continuous re-evaluation of filter expressions we benefit from dynamic programming techniques to generate pre-compiled object code versions of the filter specifications which can be installed in the control flow graph.

C.2.4 Virtualisation through recursive composition and resource management

In order to support multilevel composition, as advocated in section C.2.2 and illustrated in figure C.1c, we need to enable the abstraction and use of composed services as single Entities. This, together with the re-use of Entities in multiple service contexts, enables virtualisation of service functionality.

Within the functional composition framework a runtime service engine provides a “container” for a composition context and instantiates a *virtual functional block*; which can be used and accessed from within another service composite.

As shown in figure C.1b a runtime service engine encapsulates a service classifier generated from the control flow graph and its superimposed filters), provides primitives for the management of the classification process, and has a fixed slice of system resources allocated for its operation. The design of a service engine is rather simple, providing lightweight virtualisation, and achieves good performance as reported in [109]. Two distinct queues identified as *internal* and *external* store the incoming messages for classification. The external queue holds messages that are to be classified for first time, while the internal queue holds messages already in transit through the composite service and awaiting re-classification. A queue scheduler is responsible for adjusting the relative priority that the two queues are serviced by the classifier. Early work in [110] concluded that this strategy of separating *in-service* message flows from *upcoming* ones provides better performance.

C.3 Evaluation of the functional composition framework

The evaluation of the functional composition framework has taken place in cooperation with researchers from the ANA project [111]. We present a short analytical study assessing the adaptation capacity in terms of functional flexibility and extensibility and a quantitative assessment of the performance achieved. The former demonstrates that the principle concepts are viable within a current and future network context where as the latter proves the feasibility of our work in terms of performance.

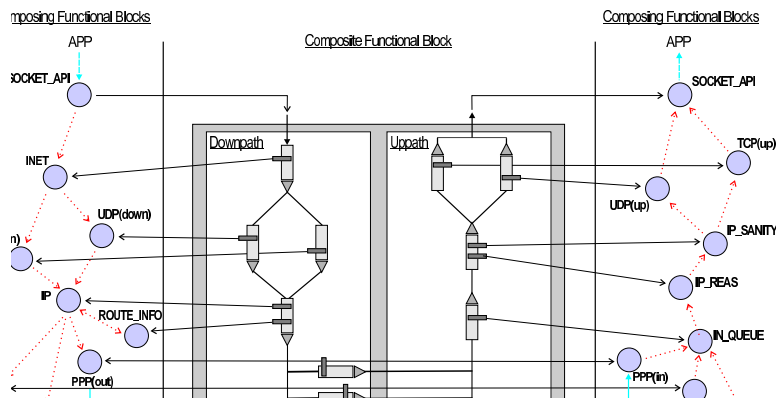


Figure C.1: Composite TCP/IP stack service

C.3.1 Adaptation capacity

The first part of the evaluation is an analytical assessment using some simple scenarios that investigate the effectiveness of the composition model to leverage flexible adaptation.

In [107] there is an extensive case study that shows how a “traditional” TCP/IP_{v4} network stack [112] may be modularised and implemented. We use this configuration as a baseline setup and show how adaptation may be served thereafter by incrementally integrating functional extensions. Figure C.1 shows the service graph providing the TCP/IP_{v4} stack configuration of a 4.4 BSD [112] system. The “composite Entity” in the middle of the figure represents the composed service (classification graph) that integrates the functions provided by Entities at the right and left of the figure. The structure of the service graph reflects the organisation of the current network subsystems in the *down-path* (left-part) and *up-path* operation (right-part). However, this structure is not definitive. For instance, an alternative cascaded composition strategy is possible, whereby the TCP/IP service may consist of multiple other service composites implementing the transport layer, network layer, and link-layer functionality separately; increasing opaqueness among the second-level service composites, and at the same time allowing independent resource management for each.

C.3.1.1 Network Gatewaying

Enabling IPv4 forwarding function in the composite service in figure C.1 involves only a simple extension of the service graph structure by an edge connecting the graph node where the IP_SANITY Entity resides, to the down-path (figure C.2a). This is accomplished by adding a graph filter that matches all traffic that is not destined to the IP addresses of the local node and diverts it to the down-path. This extension possible at runtime without affecting the existing functional blocks composition of the service. This demonstrates adaptation through graph topology changes without the need to introduce new service functions in Entities.

C.3.1.2 IP version 6 support

While the forwarding capability regards the enhancement of the existing functionality, we also look at the extension of functionality by reusing as much of the existing service (in Figure C.1) as possible thereby leveraging functional scalability. Version 6 of the IP protocol [17] is a good example of this as, in principle, it should not necessitate the installation of a second separate protocol stack (as required today).

Figure C.2b illustrates the required extensions. The down-path is extended by installing a message processing filter at the same graph node where the IPv4 Entity resides, to match the AF_INET6 domain identifier (in the respective *protocol control block* data structure) and dispatch the message to a new IPv6 Entity. The

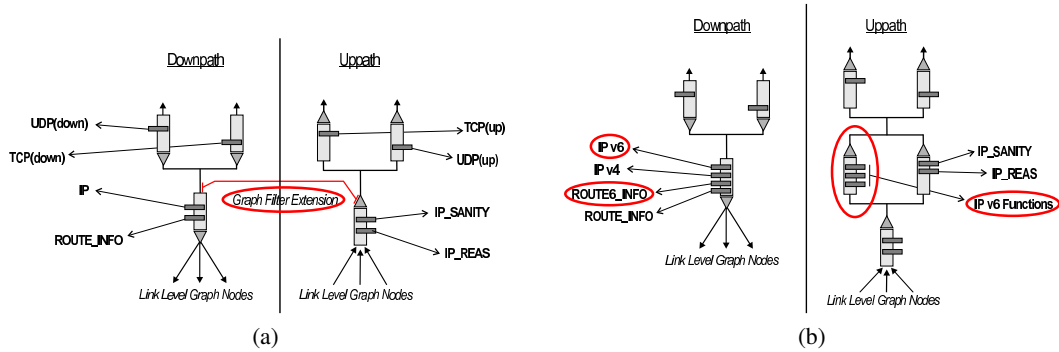


Figure C.2: Extensions to existing service *a*: Enabling IP forwarding *b*: Embedding IPv6 support.

checksumming algorithm of TCP and UDP functional blocks typically looks for fixed fields of the *sock-addr* data structure for the generation of the pseudo-header, and therefore can remain mostly unchanged². Entities providing IPv6 forwarding information, may additionally be installed on the same graph to service RT_QUERY6, RT_DELETE6, RT_UPDATE6 request types. The required extensions on the up-path are equally minimal, since all that is needed is IPv6 sanity checking and header processing functionality that can be hosted in a new branch after the IN_QUEUE processing graph node, or directly at the same graph node where the IP_SANITY and IP_REAS Entities reside (using analogous message processing filters).

Integration of IPv6 functionality is thus possible by adding to the existing service composite only three message processing filters and their back-end functionality providing Entity. All these enhancements can take place after the installation of the initial service and without interfering with the structure of the existing data paths.

C.3.2 Performance

The feasibility of functional composition depends largely on the performance overhead generated by the compartmentalised functions. In order to assess the performance we evaluate the different components of our design independently. We have conducted our measurements with a user space prototype running on a Mac OS X 10.5 installation on a Intel Core i7 (4-way) 2.8GHz CPU, 4GByte RAM, and 1 Gbit interfaces hardware configuration.

First we provide a statistical estimate of the latency introduced in the data path processing by the lightweight resource virtualisation provided by the service engines. Specifically we measure the delay overhead encountered for queueing, scheduling (for classification) and re-queueing of messages. A simple pass-through classifier that returns a message back immediately after receiving it allows us to exclude from our measurement the classification penalty (measured later). We generated a large number of time-stamped input messages (100byte each), alternating between constant rate and bursty traffic intervals so that at specific moments in time the queues get congested. Figure C.3a shows a window of the delay plot indexed by message sequence number. The encountered overhead remains on average constant at about 0.2 μ sec as we vary the input rate (we note for reference that the minimum packet inter-arrival time for a 100Mbit network interface is about 120 μ sec). In figure C.3b we repeated the same measurement several times and for all the collected data-sets we plotted the cumulative delay distribution. The steep initial step shows that 75% of the delays is concentrated below 10 μ sec in the x-axis, nearly 65% is less than 5 μ sec, and about 60% takes only a fraction of a μ sec (about 0.2 μ sec average). The occasional high values are attributed apart from the queueing congestion in the service engine, to operating system related factors (e.g. user space process scheduling).

²(This is only the case if they can operate on variable length fields)

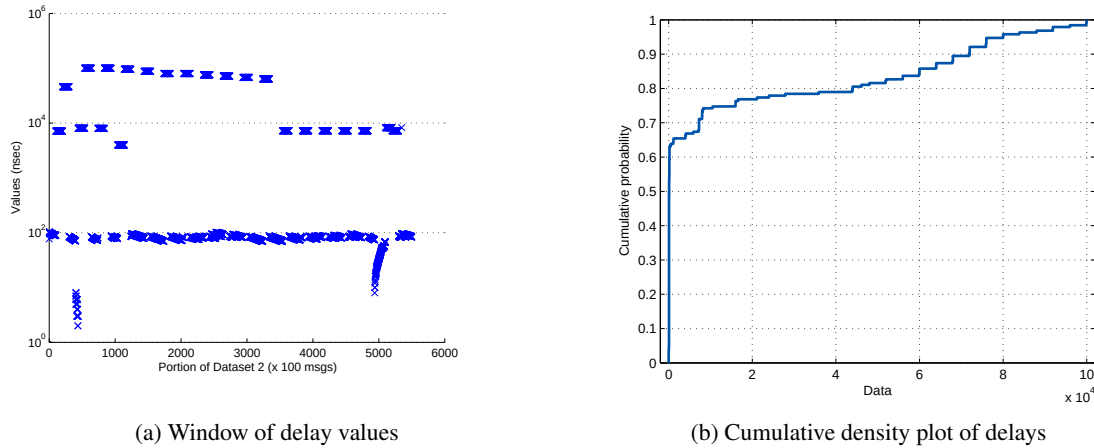
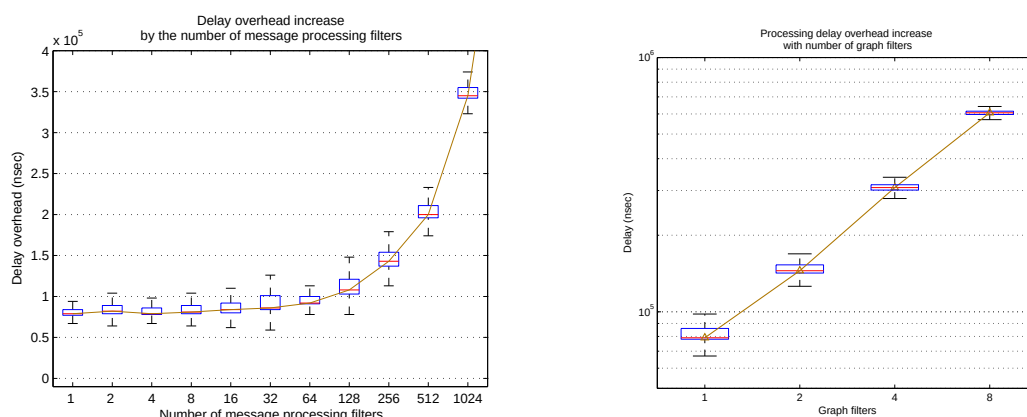


Figure C.3: Density and distribution of delays in a service engine

In the next step, we proceed to measure the cost of the different filter types. First, in order to assess the delay overhead incurred in the processing of graph filters we considered three different control flow graphs in which we incrementally increased the number of graph filters (graph edges) from 1 to 8 by a power of 2 in each step. This means that the messages had to traverse more graph nodes in every step. In every case, and for every configuration, we measured the increase of delay as a function of the increase of graph filters. Figure C.4b reports on the results, whereby a log-linear increase can be observed, implying that the delay cost is linearly proportional to the number of edges (fanout) from every graph node. This cost averages about $80 \mu\text{sec}$ every time the number of graph filters is doubled. We repeated the same measurements while having a varying density of message processing filters on the graph nodes. In this case there was little change in the results, being in the order of $10^2 \mu\text{sec}$. This implies that the impact of graph filter processing is substantially higher than that of message filter processing.

To measure the cost of message processing filters we created a simple composition graph with two graph nodes (1 graph filter connecting them) and one message processing filter installed on it running within a service engine. We then generated a large number of input messages with a pattern that matches the filter. In subsequent test runs we progressively increased the number of message processing filters by a power of two up to 1024 filters on one graph node. In figure C.4a we report the results of the delay overhead increase (average value and inter-quartile range) as the number of message processing filters increased. One can see that the average delay overhead incurred by up to 64 message processing filters remains below $100 \mu\text{sec}$ (where $80 \mu\text{sec}$ delay are attributed to the graph filter as reported earlier) and with a maximum variation in individual samples that does not deviate more than $15 \mu\text{sec}$ from the average value. The rate of increase in the overhead is around $0.22 \mu\text{sec}$ per message processing filter.

In summary, there is a trade-off in the composition process between using an expanded graph (which increases compositional flexibility and imposes stricter ordering of service functions) and a more compact graph (which provides better performance). An overall observation from these results is that the performance for enabling functional adaptation through our design, despite the user space implementation, is sustainable for 100Mbit line speeds. This makes our mechanism well suited for edge configurations and virtual machines. It strongly suggests that a kernel space solution for core systems could be even more promising. Hence, we conclude that functional composition for standard networks and protocols is a feasible option.



(a) Processing delay increase as a function of message processing filters

(b) Processing delay increase as a function of graph filters

Figure C.4:

C.4 The functional composition framework in the GP Node framework

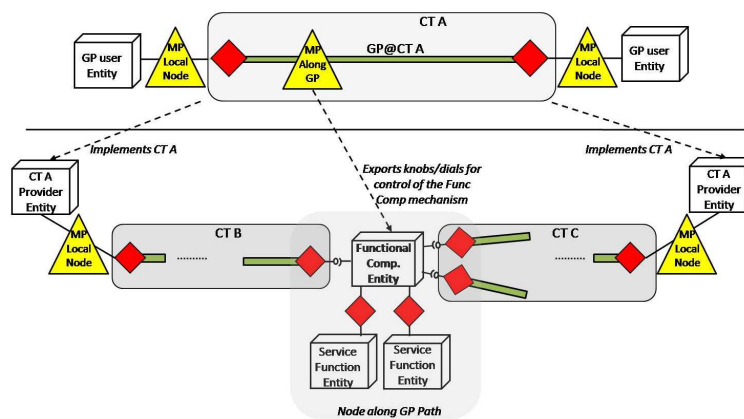
In the GP node framework the functionality and control abstracted through the Mediation Point construct relates to an access mechanism to the data structures that store the bindings (hooks) between Entities. From an implementation point of view such a mechanism may be distributed across Entities (hidden within the common static programming interface), or provided by a separate Entity that provides exclusive proxy access to the binding data structures. So far in the current prototype the latter approach has been followed. As a result the actual mechanism that instantiates the Mediation Point abstraction is provided in an Entity container.

In this implementation context, the functional composition framework provides an alternative, more dynamic and sophisticated, data structure than the binding table in the current GP node prototype for capturing and maintaining the relationships among Entities that compose Compartment and GP services.

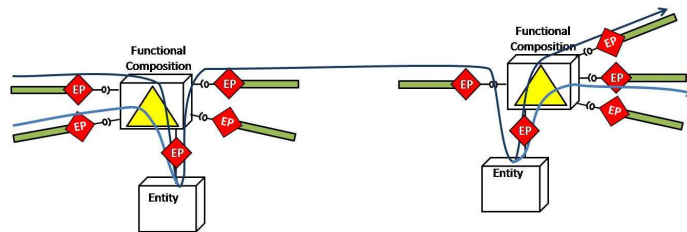
Figure C.5a therefore depicts schematically an Entity as a placeholder for the functional composition framework (lower part of the figure), and at the same time provides the MP interface abstraction to GP users (upper part of the figure). Subsequently in figure C.5 we show the operation envisaged through the functional composition framework for flow-driver dynamic manipulation such that it can enable network coding (figure C.5b), flow splitting and multi-path routing (figure C.5c), flow redirection, flow mobility and other.

C.5 Related work

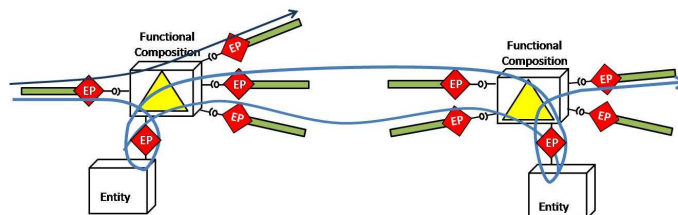
Functional composition is important in many recent research projects addressing the Future Internet. A few prominent examples include [113] where functional composition is employed to compose and adapt service functionality given a number of available service components and subject to awareness about the system's operational environment, [114] where a mechanism providing functional composition leverages the "inter-stitching" of service paths, [115] where a component composition model based on roles is advocated as a more dynamic, modular and extensible alternative to layers, and [116] where hierarchical multilevel node composition framework is proposed through role-specific service manager components. In



(a) Functional Composition implemented in an Entity and abstracted as a MP



(b) Network merging/coding by means of functional composition



(c) Multipath routing, load balancing, and flow splitting by means of functional composition

Figure C.5:



[117] functionality is semantically organised into planes and an overlay orchestration plane is responsible for the composition of services through management of interactions between functions from other planes. For most such architectures adaptation is addressed in respect to structural customisation of discrete service components and often follow up on research from active and programmable networks.

In the community of active and programmable networks a significant amount of work was invested in frameworks for enabling adaptation and dynamic service composition focusing on one or all of the following goals: deployment of new services, introduction of extended functionality into network nodes and customisation of services for different applications. Two main paradigms were considered. In the *integrated* approach [118] an *active capsule* replaces the traditional network packet and carries processing instructions or references to instructions [119] that are executed in every visited network node. This enables dynamic but uncoordinated deployment of new functionality. In the *discrete approach* [120], *active extensions* refer to dynamically loadable programs carried or referenced in data packets and loaded out-of-band. The objective is to extend the restricted programming enabled by the lightweight active packets (only small programs can be included in every packet) and provide a level of functionality close to that of a Turing machine. Building on one or both of these paradigms, subsequent research made advances in a number of related areas such as programming models and paradigms (e.g. [121, 122, 123]), compact languages for active network programming (e.g. [124, 125, 126]), composable platforms and node OS architectures (e.g. [127, 128, 129, 130, 131]), composition models (e.g. [110, 132, 133]) and execution environments (e.g. [134, 135, 130]). Various aspects of this work found their way to deployment within commercial solutions or became the subject of further research in areas such as autonomic networks and future Internet engineering. The work in this report builds upon active network research by extending the scope of adaptation: (a) beyond the current Internet fabric, (b) assuming functional composability of both the control plane and data plane, and (c) by addressing service optimisation as an orthogonal aspect for adaptation.

Functional composition by means of service optimisation on the other hand is currently a subject of extensive research in the area of cross-layering, which tries to overcome the opaqueness of protocol layers by extending the interactions of functions across layer boundaries. Cross-layering has a broad scope of applications including leveraging mobile IPv4 and IPv6 hand-offs through link layer triggers [136, 137], traffic management using differentiated services [138, 139], improving multimedia delivery [140, 141], optimising link layer throughput [142, 143], and improving TCP's performance on wireless environments [144, 103, 145]. Cross layer solutions deploy different techniques for acquiring intra-layer information and facilitating control loops across layers. In [146, 147] network traffic is used as a type of in-band (in the data path) shared memory for storing cross-layer information, piggybacked in ICMP packets or normal traffic. In [148, 149, 150] orthogonal data storing facilities are proposed for maintaining information advertised by the different layers, which can be accessed out-of-band by other layers to optimise their behaviour and operation. In [151] the authors enhance the layered network subsystem with a protocol API to each layer and an orthogonal execution environment for hosting inter-layer optimisation elements. An alternative approach in [102] proposes memory mapping of configuration state from one layer (with read-only access) to adjacent layers, and from there recursively to their adjacent layers. In [152] and [153] the authors propose a type of "direct signalling" between layers that need to interact in a more or less ad-hoc and on-demand manner through a message switching interface. Cross-layer information carrying messages are exchanged among layers in the same fashion that packets would be exchanged among network nodes in a fully meshed topology, or through a mediator entity.

The variety of designs in these frameworks reflect not only the diversity of requirements for different application classes, but also the orientation, perception and vision of the different research groups regarding the future Internet architecture. In our functional composition framework, although we make no assumptions about layers, we nevertheless facilitate the different techniques of information sharing between Entities as advocated in the cross-layering literature to leverage functional optimisations. For instance, in-band information exchange is elegantly provided through the header stack in the message meta interface in combina-



tion with the use of message processing filters. Out-of band direct signaling between Entities is provided by message communication along edges of the service graph reserved (by graph filters) for control-plane communication. Furthermore, the report channel identifier in the message meta-interface can be used to send information to an external database or logging facility (e.g. syslog in UNIX), or directly to an out-of-band interface of an Entity.

Finally, orchestrating functional composition is closely related to the process of classification and in our work even more explicitly to packet-based classification, where a substantial amount of effort has been invested in the past on achieving flexibility with sustainable performance. Prominent architectures that have provided ideas and inspired our work include the Berkeley packet filter [154], the dynamic packet filter (DPF) [155], the pathfinder classifier [156] as well as threaded indices [157] and tag switching [158].

D Optical Technologies: Simulating a Generic Network Environment

D.1 Introduction

The wide diversity of optical resources and technologies (particularly transmission techniques as shown in figure D.1) is very difficult to represent in a generic framework. It would be almost impossible to our opinion, to represent transmission resource in such a model, as transmission feasibility and techniques really depend on vendors, products, context.

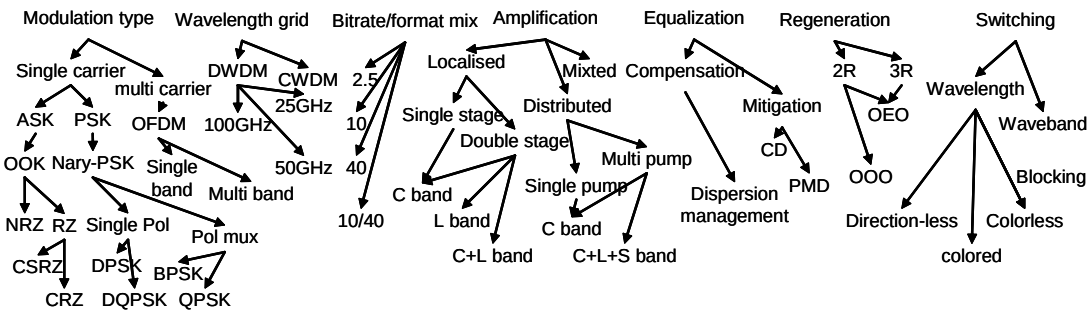


Figure D.1: schematic description of possible technologies used for standard OCS switching (transmission technologies and switching types).

Moreover each technique and vendor has its own engineering rules that are very complex to generalize and require a fine knowledge of physical equipments in order to be applied. We believe that this complexity should only be dealt at the optical physical layer where such expertise can be confined and invisible to higher layers. We propose an abstract representation of the optical physical resources that will mask technology specificities (e.g. physical constraints such as conditional multiplexing of 10 Gbit/s and 40 Gbit/s channels depending on modulation format ...). This abstract topology can also mask real topology to the client layers for security reason, and the abstraction will only provide attributes and functions less dependant on technology. Such a model of abstract objects has already been proposed in D5.2 deliverable. In the following subsections, we present the proposed optical resource ontology based on the classification of figure 5.12 to be used to build the abstract objects.

D.2 Infrastructure

Infrastructure is composed of cables themselves containing a set of fibres. Besides their own attributes, cables and fibres have "infrastructure" attributes (infrastructure inheritance) and parameters. They have relationship with equipments because equipments depend on their characteristics and they obey to rules (for example a rule can be : "fibre name is composed of cable name"). Fibres and cables are described in figure D.2.

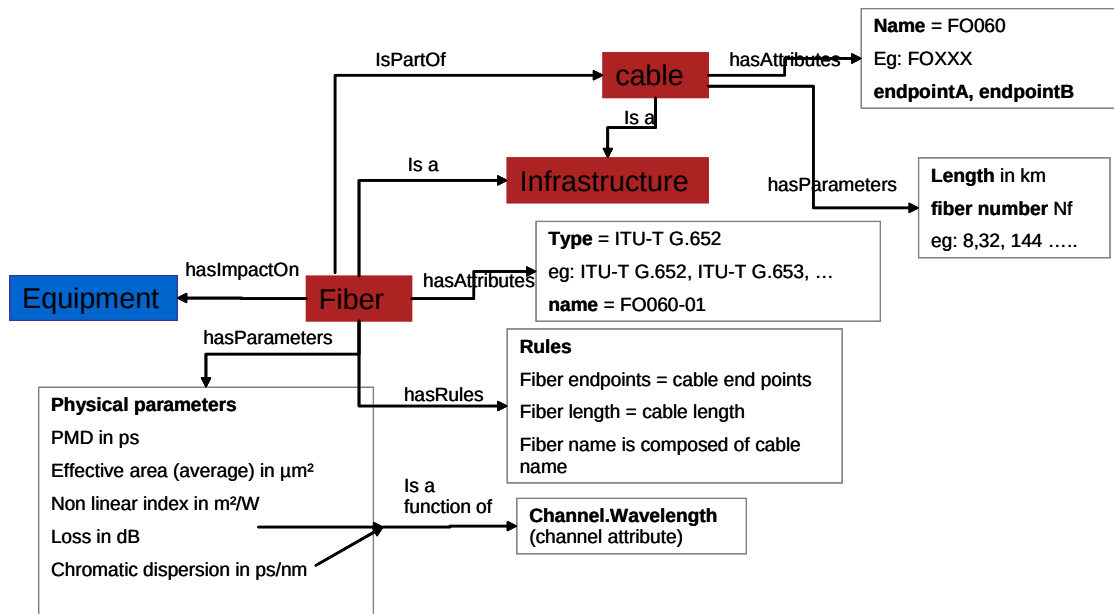


Figure D.2: Infrastructure description.

D.3 Equipments : transmission

Equipments are separated into transmission equipment and switching equipments. Transmission equipments are composed of amplifiers and transponders, both responsible for correct transmission of signal from one end point to the other. Transponders are also interfaces to the upper layer. Their use is restricted by rules (either applied at installation or during operation) in relation with infrastructure specificities and with transmission systems specificities. An important rule concerns transponder compatibility that allows a pair of transponders to operate only if they are compatible and if the transmission system they are connected to (set of amplifiers) is compatible with them. Transmission equipments are described in figures D.3 and D.4.

D.4 Equipments : switching

Switching equipments are used to switch optical signals on the optical layer. They can be traditional reconfigurable optical add-drop multiplexer (ROADM), photonic switches, optical burst switches or any possible switching technology that operates at the photonic level. Switches are composed of two parts : the switching fabric that handles data plane signals (wavelength for ROADM, Bursts, and so on...) and the controller that is responsible for the whole switch correct operation. The controller is in particular responsible for control message exchange and processing (for resource reservation, for topology distribution ...). The switching equipments are described in figure D.5.

D.5 The Abstract Resources

All previously described resources are then abstracted into a more generic description which is composed of Nodes, Links and Channels. Nodes are an abstract view of switching equipment, while Links are an abstract view of transmission equipment, with the particularity that a Link may encompass nodes bypasses. A Channel represents finally the end-to-end used resource between two transponder client interfaces. A

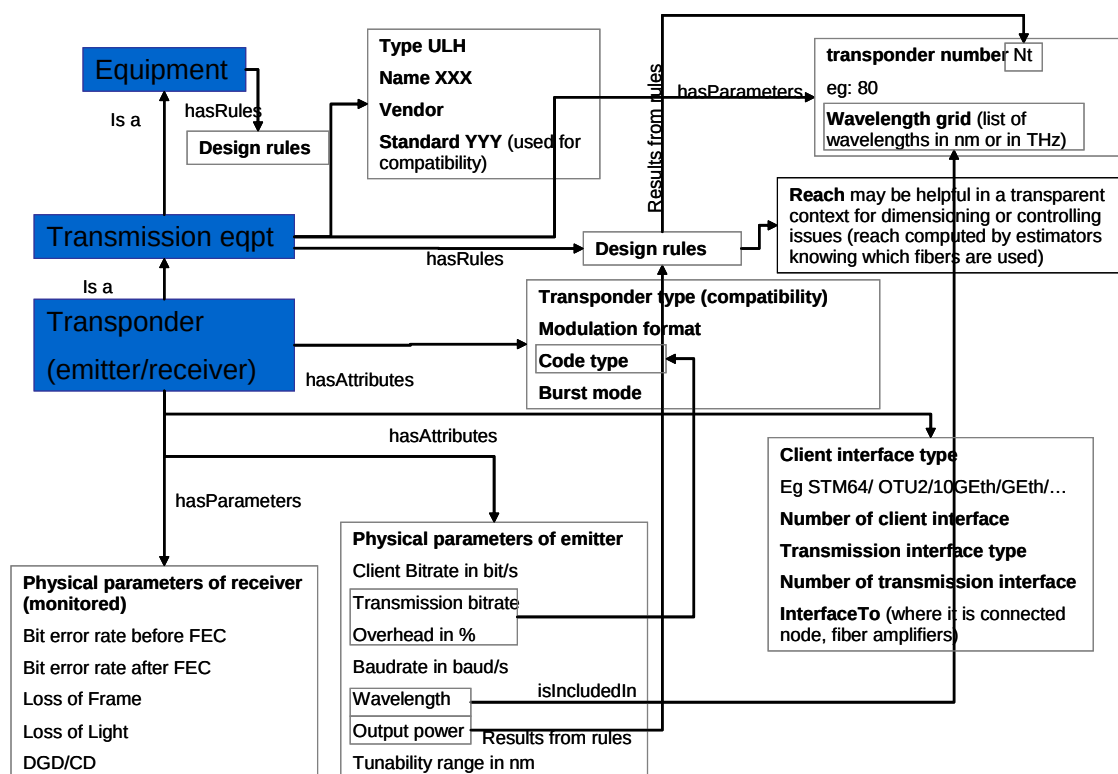


Figure D.3: Transmission equipments description: the transponder.

Channel represents the resources that support a Generic Path. Abstract objects are described in figure D.6 and D.7.

D.6 The Abstract Representation

The optical resource ontology has been used to build an abstract representation of the resources (as explained in section 5.1) on an example network (shown in figure D.8). The physical network is made of six nodes (A B C D E F). The network is a typical optical network where nodes are interconnected by links made of several spans with amplifiers placed following engineering rules of the transmission systems (following standard commercial systems in our example). Transponders are connected to add ports of the switching fabric in nodes. In this example, a channel can be established between two transponders, if they are compatible and if they are compatible with transmission systems (amplifiers and nodes) the optical path will use.

The result of the automatic abstract representation builder is shown in figure D.9

As can be seen on the example of this figure, the resources that are shown to the upper layer are very simplified: only useful bandwidths and delays are shown. No details are given on the path or on the used technology. In the example we have represented bandwidth resource as the ensemble of all possibilities offered by the combination of optical resources and available compatible transponders. The plurality of shown resources does not mean that all are available simultaneously or reserved. Indeed, a reservation in another node (for a channel between F and A for example) may change the set of available resources for F to C. The available resources are collected thanks to the mechanism proposed in section 5.1

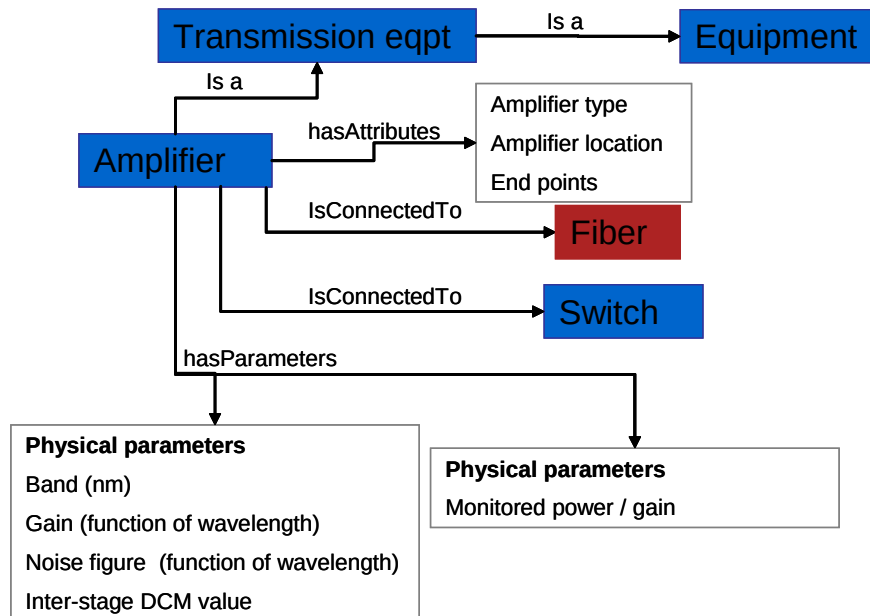


Figure D.4: Transmission equipments description: the amplifier.

D.7 Examples of NED code

The network simulation has been built based on the OMNeT++ environment [74]. The object-oriented capabilities of OMNeT++ (e.g. inheritance of components and their properties) have been used to model the ontology of the various optical components. For this purposes we used the NED language, which seems to be quite easy to translate into an abstract Unified Modeling Language (UML) representation [159].

The Network Description language (of OMNeT++) (NED) code [159] has been developed based on the developed resource description ontology. Each compound module represents a physical resource type (or concept) and each of them contain the following three simple modules

- a simple module that manages the elements gates and connections
- a simple module that manages the rules of the elements
- a simple module that manages its physical parameters or attributes

For simplification reasons the same gates have been used in the experimental work for control message exchange as for data plane. In this demonstration, the use of dedicated gates and physical support would not change the result as long as physical and control topologies remain congruent .

The following listings show NED codes for all modules of the description. Besides these modules, and for automatisation purpose, the simulation also contains a "start" module that is executed once and that creates an instantiation of all physical modules required to represent the physical network in test (for example network of figure D.8. During the initialisation for example, the correct number of fibres are created inside each cable, the correct number of switching ports in the switching fabrics and so on. All modules are interconnected at this step except if their interconnection is unknown (i.e. not yet configured). As a result the "start" initialization procedure "creates" a representation of a physical network based on the optical resource ontology.

The same generated representation is used to emulate message exchange generated by the automatic abstract representation builder that will then "build" progressively the node abstract representation of available resources in the network.

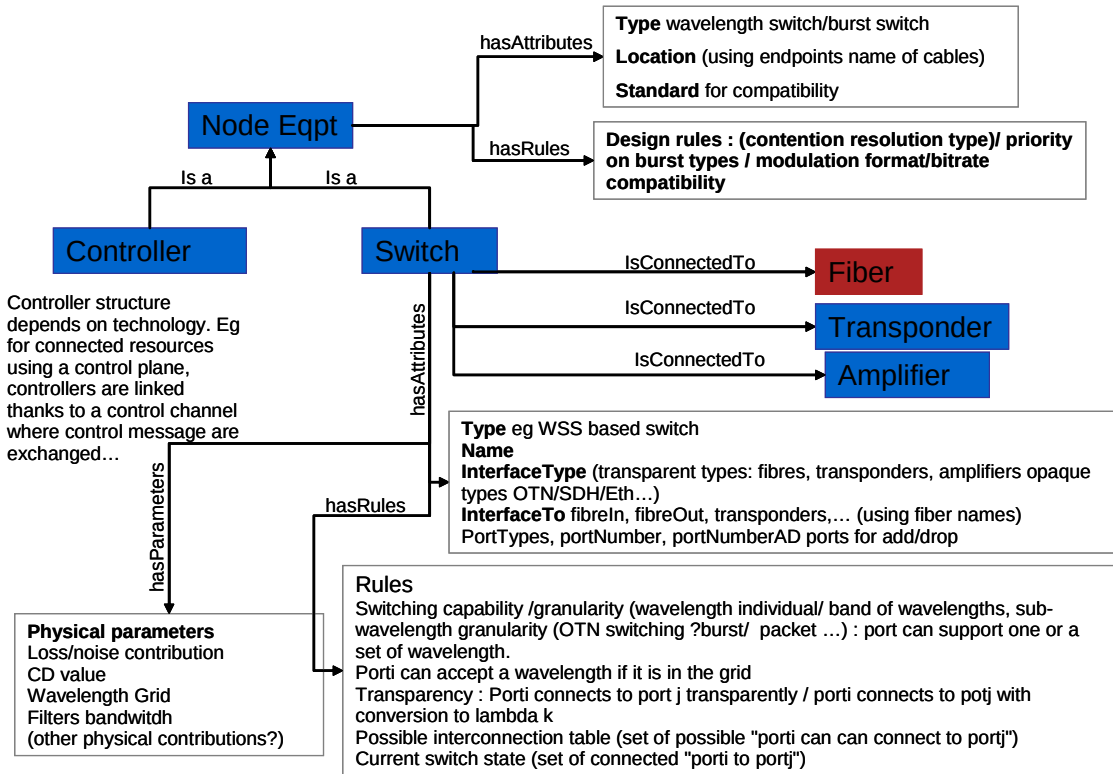


Figure D.5: Switching equipments description.

```
1 simple connectionDescription
2 {
3     gates: input In[];
4     output Out[];
5 }
```

Listing D.1: Simple module managing connections.

```
1 simple physicalParameters
2 {
3     parameters:
4         double length@unit(km);
5         double loss@unit(dB);
6         double PMD@unit(ps);
7         double CD@unit(ps/nm); //....
8         double effectiveArea@unit(microm*microm);
9         double nonLinearIndex@unit(m*m/W);
10        string ripples ;
11        string uniformity ;
12        string crosstalk ;
13        string wavelengthgrid@unit(nm); //or in THz = set of wavelengths centers of the grid
14        string wavelengthgridName;
15        string wavelength@unit(ps/nm); // string because can be set to "Tunable" or to a double value
16        double tunabilityRangeLow@unit(nm);
17        double tunabilityRangeHigh@unit(nm);
18        double bandLowLimit@unit(nm);
19        double bandHighLimit@unit(nm);
20        volatile double gain@unit(dB);
21        volatile double noiseFigure@unit(dB);
22        string Pin@unit(dBm); // signal power at the modules input with range (pinA PinB rangeA range B)
23        string Pout@unit(dBm); // signal power at the module output with range (PoutA PoutB range A rangeB)
24        double OSNRmin@unit(dB); // minimum OSNR required at receiver
25        double Pmin@unit(dBm); // minimum required power at receiver
26        double Pmax@unit(dBm); // maximum allowed power at receiver
```

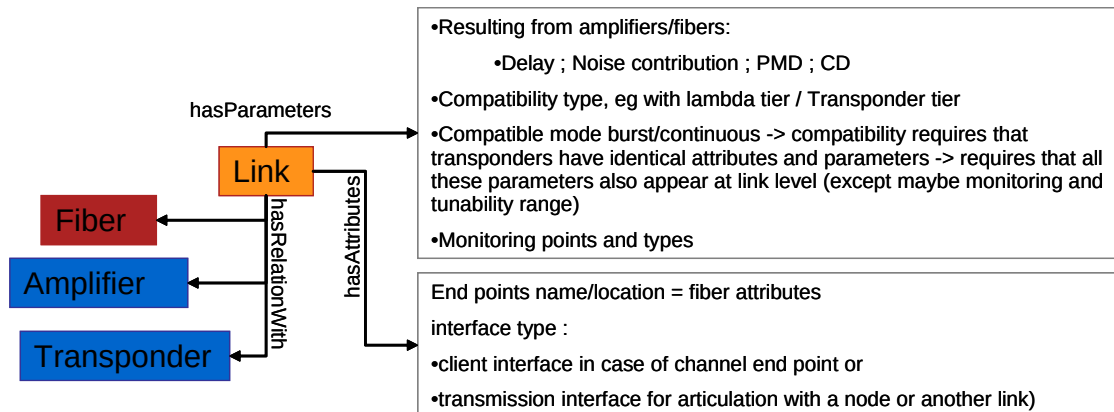


Figure D.6: Abstract resources description : Links.

```

27  int Nmax; //maximum number of wavelength in the system
28  double CDmax@unit(ps/nm); // maximum acceptable chromatic dispersion value at receiver
29  double PMDmax@unit(ps); // maximum acceptable PMD value at receiver
30  string powerPreemphasis@unit(dBm) ; //array of input power values for each wavelength in the WDM comb
31  }

```

Listing D.2: Simple module managing physical parameters.

```

1  simple rules
2  {
3  }

```

Listing D.3: Simple module managing rules. Rules are only represented by C++ code so module is empty at NED stage.

```

1  module cable
2  {
3      parameters:
4          double x;
5          double y;
6          int Id;
7          string name;
8          string endpointA;
9          string endpointB;
10         int fibreNumber;
11     gates:
12         input cableIn[];
13         output cableOut[];
14
15     connections allowunconnected:
16 }

```

Listing D.4: compound module cable.

```

1  module fibre
2  {
3      parameters:
4          double x;
5          double y;
6          int Id; // let set an ID internal to the representation
7          string fibreName; //has relation with cable name ->see cpp file
8          string endpointA;
9          string endpointB;
10         string type; //corresponds to standards
11     gates:
12         input fibreIn;
13         output fibreOut;

```

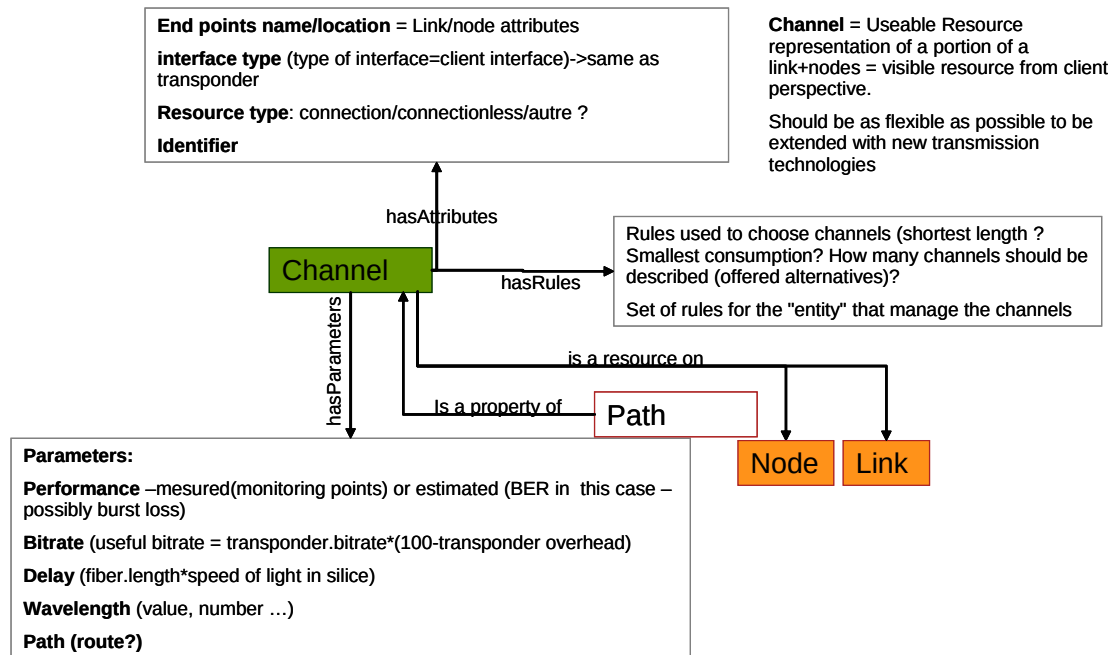


Figure D.7: Abstract resources description Channels and Nodes.

14 }

Listing D.5: Compound module: fibre.

```
1 module equipment
2 {
3     parameters:
4         int Id;
5         string equipmentName;
6         string equipmentVendor;
7         string equipmentStandard;
8         connections allowunconnected:
9 }
```

Listing D.6: Compound module: equipment.

```
1 module nodeEquipment extends equipment
2 {
3     parameters:
4         string nodeType;
5         string nodeName;
6
7     connections allowunconnected:
8 }
```

Listing D.7: Compound module: nodeEquipment.

```
1 module switchFabric extends nodeEquipment
2 {
3     parameters:
4
5         double x;
6         double y;
7         string switchName;
8         string fibreIn;
9         string fibreOut;
10        int portNumber;
11        int portNumberAD;
```

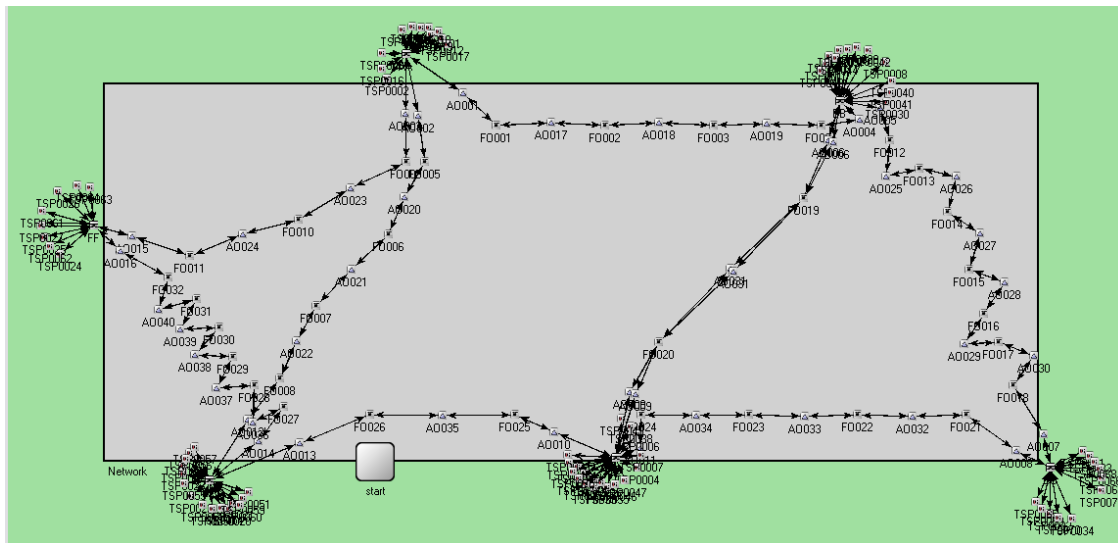


Figure D.8: Test network (6 optical nodes interconnected with optical fibres). B-D has two parallel fibre paths.

```

12     string switchState;
13     gates:
14         input switchIn[];
15         input switchAdd[] ;
16         output switchOut[];
17         output switchDrop[] ;
18
19     connections allowunconnected:
20 }

```

Listing D.8: Compound module: switchFabric.

```

1 module transmissionEquipment extends equipment
2 {
3     parameters:
4         string transmissionEquipmentType;
5         string transmissionEquipmentName;
6         //volatile double reach; /// to be defined ?
7         //int transponderNumber; // is total number of transponder usefull ?? to be defined as a function of sites
8
9     connections allowunconnected:
10 }

```

Listing D.9: Compound module: transmissionEquipment.

```

1 module amplifier extends transmissionEquipment
2 {
3     parameters:
4         //string modType;
5         double x;
6         double y;
7         string amplifierName;
8         string amplifierSite;
9         string fibreInA;
10        string fibreOutA;
11        string fibreInB;
12        string fibreOutB;
13        string amplifierType;
14        double DCMvalueA;
15        double DCMvalueB;
16
17        //monitoring

```

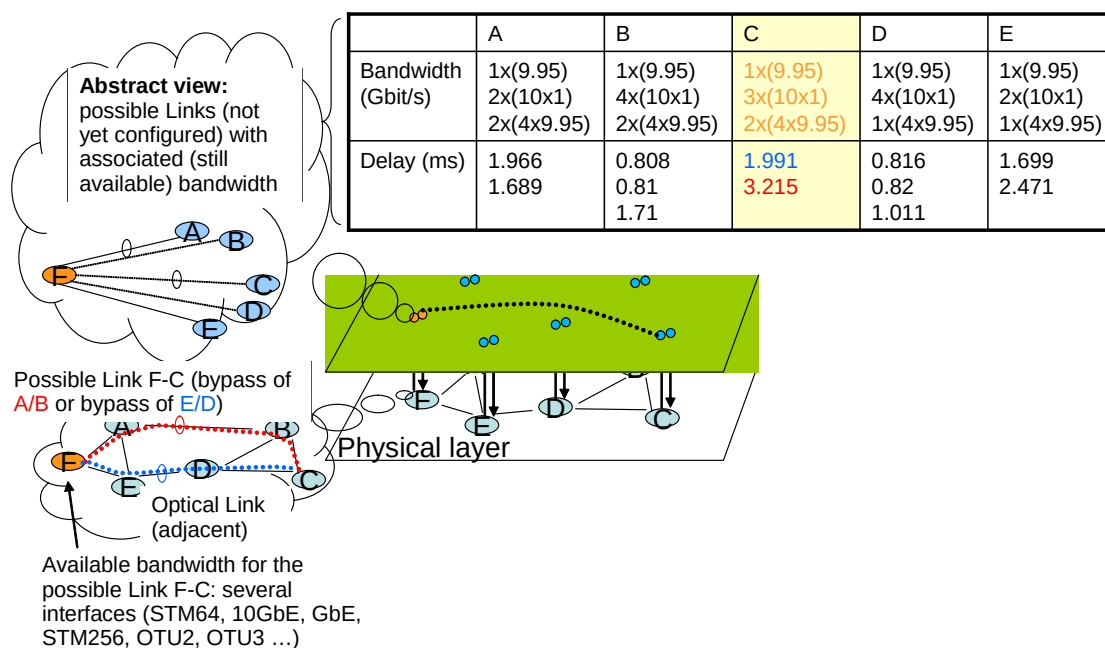


Figure D.9: schematic description of the automatic abstract representation builder result.

```

18     volatile double inputPower;
19     volatile double outputPower;
20     volatile double monitoredGain;
21     @display("bgl=2");
22     gates:
23         input amplifierIn[2];
24         output amplifierOut[2];
25     connections allowunconnected:
26 }

```

Listing D.10: Compound module: amplifier.

```

1 module transponder extends transmissionEquipment
2 {
3     parameters:
4
5     string transponderName ;
6     string interfaceTo ; // equipment or infrastructure to which it is connected
7     double transmissionBitRate;
8     double clientBitRate ;
9     double overhead;
10    double baudrate;
11    string clientInterfaceType;
12    int clientInterfaceNumber;
13    int transmissionInterfaceNumber;
14    string transmissionInterfaceType;
15    string modulationFormat;
16    string FECtype;
17    string burstMode;
18    //monitoring
19    volatile double preFECBER;
20    volatile double postFECBER;

```



```
21     bool lossOfFrame;  
22     bool lossOfLight;  
23     volatile double DGD;  
24     volatile double CD;  
25     double x;  
26     double y ;  
27  
28     gates:  
29         input clientInterfaceIn[];  
30         output clientInterfaceOut[];  
31         input transmissionInterfaceIn[];  
32         output transmissionInterfaceOut[];  
33     submodules:  
34  
35     connections allowunconnected:  
36  
37 }
```

Listing D.11: Compound module: transponder.

List of Abbreviations, Acronyms, and Definitions

3G	Third Generation
4G	Fourth Generation
AM	Anchorless Mobility
AN	Access Node
AODV	Ad hoc On-Demand Vector
API	Application Programming Interface
AS	Autonomous System
BGP	Border Gateway Protocol
BU	Binding Update
CAIDA	Cooperative Association for Internet Data Analysis
CCFW	Cooperation and Coding Framework
CMT	Concurrent Multipath Transfer
CSMA/CD	Carrier Sense Multiple Access/Collision Detection
CT	Compartment
DL	Down Link
DMA	Dynamic Mobility Anchoring
DNS	Domain Name System
E2E	End-to-End
EP	End Point
FER	Frame Error Rate
FIB	Forwarding Information Base
FI	Future Internet
ForCES	Forwarding /Control Element Separation
FTP	File Transfer Protocol
GMPLS	Generalized Multi-Protocol Label Switching
GPRS	General Packet Radio Service
GP	Generic Path
GSM	Global System for Mobile Communications
HA	Home Agent
hook	Hook
IETF	Internet Engineering Task Force



INM	In Network Management
IPv6	Internet Protocol version 6
IP	Internet Protocol
JSIM	JavaSim
LAN	Local Area Network
LQODV	Link Quality Ordering-based Distance Vector
LQO	Link Quality Ordering
LTE	Long Term Evolution
MAC	Medium Access Control
MAP	Mesh Access Point
MEE-GP	Multihomed End-to-End GP
MEEM	Multihomed End-to-End Mobility
MIH	Media Independent Handover
MIP	Mobile IP
MN	Mobile Node
MP-BGP	Multi-Protocol BGP
MPR	Multi-Path Routing
MP	Mediation Point
NDL	Network Description Language
NED	NEtwork Description language (of OMNet++)
NLRI	Network Layer Reachable Information
NW	Network
OCS	Optical Circuit Switching
OFDM	Orthogonal Frequency Division Multiplex
omnet	OMNeT
OSPF-TE	Open Shortest Path First - Traffic Engineering Extensions
OSPF	Open Shortest Path First
OTN	Optical Transport Network
PCE	Path Computation Element
(P)CN	Congestion and Pre-Congestion Notification
PHY	Physical Layer
PMD	Polarization Mode Dispersion
port	Port
QoS	Quality of Service
R-MAC	Radio Medium Access Control
RAID	Redundant Array of Inexpensive Disks
RDL	Resource Description Language
resct	Resolution Compartment



RIB	Routing Information Base
ROADM	reconfigurable optical add-drop multiplexer
RTT	Round-Trip Time
RUI	Routing Update Interval
SCTP	Stream Control Transmission Protocol
SDH	Synchronous Digital Hierarchy
SHIM6	Site Multihoming by IPv6 Intermediation
SIP	Session Initiation Protocol
SOCP	Second Order Conic Program
SONET	Synchronous Optical Network
TCP	Transmission Control Protocol
TDM	Time Division Multiplex
UDP	User Datagram Protocol
UML	Unified Modeling Language
UMTS	Universal Mobile Telecommunications System
VNet	Virtualized Networks
W-LLC	Wireless Link Layer Control
WiFi	alliance to support IEEE 802.11 WLAN standards family
WiMAX	Worldwide Interoperability for Microwave Access, Forum to support IEEE 802.16 standards family
WLAN	Wireless Local Area Network
WMN	Wireless Mesh Network
WP	Work Package
XML	eXtensible Markup Language



Document:	FP7-ICT-2007-1-216041-4WARD/D-5.3		
Date:	June 14, 2010	Security:	Public
Status:	Final	Version:	1.0

Bibliography

- [1] 4WARD (editor: Fabrice Guillemin). Architecture of a Generic Path, Jan 2009. Deliverable D-5.1 (FP7-ICT-2007-1-216041-4WARD/D-5.1).
- [2] 4WARD (editor: Sabine Randriamasy). Mechanisms for the Generic Path, May 2010. Deliverable D-5.2 (FP7-ICT-2007-1-216041-4WARD/D-5.2).
- [3] 4WARD (editor: Lcio Studer Ferreira). In Network Management for Generic Path, June 2010. Deliverable D-5.4 (FP7-ICT-2007-1-216041-4WARD/D-5.4).
- [4] H. Zimmermann. Osi reference model: The iso model of architecture for open systems interconnection. *IEEE Transactions on Communications*, 28(4):425–432, Apr 1980.
- [5] W.R. Stevens. *TCP/IP Illustrated, Volume 1: The Protocols*. Addison-Wesley, 1994.
- [6] 3GPP. Mobile radio interface layer 3 specification; core network protocols; general packet radio service (gprs). *Technical Specification 24.008*, 2008.
- [7] Amos E. Joel. Asynchronous transfer mode. *IEEE Press*, 1993.
- [8] ITU-T. *Recommendation X.25. Interface between Data Terminal Equipment (DTE) and Data Circuit-terminating Equipment (DCE) for terminals operating in the packet mode and connected to public data networks by dedicated circuit*, October 1996.
- [9] I. F. Akyildiz and X. Wang. Cross-layer design in wireless mesh networks. *IEEE Transactions on Vehicular Technology*, 57(2):1061–1076, 2008.
- [10] M. Chiang, S. H. Low, A. R. Calderbank, and J. C. Doyle. Layering as optimization decomposition: A mathematical theory of network architectures. *Proceedings of the IEEE*, 95(1):155–312, 2007.
- [11] J. Crowcroft, I. Wakerman, and Z. Wang. Layering considered harmful. *IEEE Network Magazine*, 6(1), 1992.
- [12] E. W. Dijkstra. The structure of the “THE” multiprogramming system. *Communications of the ACM*, 11(5):341–346, 1968.
- [13] E. Gamma, R. Helm, R. Johnson, and J. Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [14] G. Bouabene, C. Jelger, C. Tschudin, S. Schmid, A. Keller, and M. May. The autonomic network architecture (ana). *IEEE Journal on Selected Areas in Communications*, 28(1):4–14, Jan 2010.
- [15] Randriamasy S. et al. Mechanisms for generic paths (public report d5.2). Technical report, December 2009.
- [16] F. Baker. Requirements for IP Version 4 Routers. RFC 1812 (Proposed Standard), June 1995. Updated by RFC 2644.
- [17] S. Deering and R. Hinden. Internet Protocol, Version 6 (IPv6) Specification. RFC 2460 (Draft Standard), December 1998. Updated by RFC 5095.
- [18] M. Allen. Novell IPX Over Various WAN Media (IPXWAN). RFC 1634 (Informational), May 1994.
- [19] S. Kent and K. Seo. Security Architecture for the Internet Protocol. RFC 4301 (Proposed Standard), December 2005.



- [20] D. Pezaros. *Network Traffic Measurement for the Next Generation Internet*. PhD thesis, Lancaster University, Computing dept, 2005.
- [21] H. Schulzrinne, S. Casner, R. Frederick, and V. Jacobson. RTP: A Transport Protocol for Real-Time Applications. RFC 3550 (Standard), July 2003. Updated by RFC 5506.
- [22] J. Postel. Transmission Control Protocol. RFC 793 (Standard), September 1981. Updated by RFCs 1122, 3168.
- [23] T. Dierks and C. Allen. The TLS Protocol Version 1.0. RFC 2246 (Proposed Standard), January 1999. Obsoleted by RFC 4346, updated by RFC 3546.
- [24] S. Paul, R. Jain, and J. Pan. A vision of the next generation internet: A policy oriented view. pages 1–14, Sep 2008.
- [25] PlanetLab. open platform for planetary scale services.
- [26] GANT. pan-european data network for research and education.
- [27] R. Braden, T. Faber, and M. Handley. From protocol stack to protocol heap - role-based architecture. In *Proceedings of 1st ACM Workshop on Hot Topics in Networks*, Princeton, NJ, USA, Oct 2002.
- [28] J. Day. *Patterns in Network Architecture: A Return to Fundamentals*. Prentice Hall, 2007.
- [29] I. Stoica, D. Adkins, S. Zhuang, S. Shenker, and S. Surana. Internet indirection infrastructure. *IEEE/ACM Transactions on Networking (TON)*, 12(2):205–218, Apr 2004.
- [30] C. Tschudin and R. Gold. Network pointers. Princeton, NJ, USA, Oct 2002.
- [31] J. Touch, Y. Wang, and V. Pingali. Recursive network architecture. Technical report, ISI, Oct 2006.
- [32] J. Crowcroft, S. Hand, R. Mortier, and T. Roscoe. A warfield, plutarch: an argument for network pluralism. *Computer Communication Review*, 33(4):258–266, Oct 2003.
- [33] Huggle. A European Union funded project in Situated and Autonomic Communications.
- [34] Autonomic internet, eu-fp7-ist funded project in networks of the future.
- [35] K. Calvert, J. Griffioen, and L. Poutievski. Separating routing and forwarding: A clean-slate network layer design. In *Proceedings of the Broadnets 2007 Conference*, Sep 2007.
- [36] P. Bertin, S. Bonjour, and J.-M. Bonnin. Distributed or centralized mobility? In *Global Telecommunications Conference, 2009. GLOBECOM 2009. IEEE*, pages 1 –6, nov. 2009.
- [37] C. Perkins. IP Mobility Support for IPv4. RFC 3344 (Proposed Standard), August 2002. Updated by RFC 4721.
- [38] D. Johnson, C. Perkins, and J. Arkko. Mobility Support in IPv6. RFC 3775 (Proposed Standard), June 2004.
- [39] Luis M. Correia, Henrik Abramowicz, Martin Johnsson and Klaus Wnstel et al. *The 4WARD Book*. Springer, 1 edition, February 2010.
- [40] D. Farinacci, V. Fuller, D. Meyer, and D. Lewis. Locator/id separation protocol (lisp) draft-farinacci-lisp-12.txt, March 2 2009. Expires: September 3, 2009.
- [41] T. Biermann et al. Description of Generic Path Mechanisms. Deliverable D-5.2.0, 4WARD Project, 2009.
- [42] R. Stewart. Stream Control Transmission Protocol. RFC 4960 (Proposed Standard), September 2007.
- [43] E. Nordmark and M. Bagnulo. Shim6: Level 3 Multihoming Shim Protocol for IPv6. RFC 5533 (Proposed Standard), June 2009.
- [44] Eranga Perera, Roksana Boreli, Stephen Herborn, Eisl Jochen, Michael Georgiades, and Eleanor Hepworth. A mobility toolbox architecture for all-ip networks: An ambient networks approach. *IEEE Wireless Communications Magazine*, April 2008.



- [45] V. Typpo, O. Gonsa, T. Rinta-aho, and et al. Triggering multi-dimensional mobility in ambient networks. In *Proc. 14th IST Mobile & Wireless Communications Summit*, Dresden, Germany, Jun. 2005.
- [46] Maekelae J. and Pentikousis K. Trigger management mechanisms. In *Proc. Second International Symposium on Wireless Pervasive Computing*, pages 378–383, San Juan, Puerto Rico, USA, Feb. 2007.
- [47] IEEE 802.21. Ieee standard for local and metropolitan area networks, part 21: Media independent handover services, 11 November 2008.
- [48] Telemaco Melia, Antonio de la Oliva, Albert Vidal, Ignacio Soto, Daniel Corujo, and Rui L. Aguiar. Toward ip converged heterogeneous mobility: A network controlled approach. *Computer Networks*, 51(17):4849–4866, 2007.
- [49] OPNET modeler, wireless suite.
- [50] R. Fracchia, C. Casetti, C-F. Chiasserini, and M. Meo. A wise extension of sctp for wireless networks. In *IEEE International Conference on Communications, ICC'05*, 2005.
- [51] X. Zhang, T-M-T. Nguyen, and G. Pujolle. A cross-layer approach to optimize the performance of concurrent multipath transfer in wireless transmission. In *2nd IFIP Wireless Days (WD09)*, 2009.
- [52] J.R. Iyengar, P.D. Amer, and R. Stewart. Concurrent multipath transfer using sctp multihoming over independent end-to-end paths. *IEEE/ACM Transactions on Networking*, 14(5):951–964, 2006.
- [53] Figueiredo S., Bertin S., Lourenco J., Schefczik P., Wei G., and Zhang X. Generic paths technical mobility approaches. *submitted for the Third Joint IFIP Wireless and Mobile Networking Conference*, October 2010.
- [54] P.V. Mockapetris. Domain names - concepts and facilities. RFC 1034 (Standard), November 1987. Updated by RFCs 1101, 1183, 1348, 1876, 1982, 2065, 2181, 2308, 2535, 4033, 4034, 4035, 4343, 4035, 4592.
- [55] P.V. Mockapetris. Domain names - implementation and specification. RFC 1035 (Standard), November 1987. Updated by RFCs 1101, 1183, 1348, 1876, 1982, 1995, 1996, 2065, 2136, 2181, 2137, 2308, 2535, 2845, 3425, 3658, 4033, 4034, 4035, 4343.
- [56] J. Rosenberg, H. Schulzrinne, G. Camarillo, A. Johnston, J. Peterson, R. Sparks, M. Handley, and E. Schooler. SIP: Session Initiation Protocol. RFC 3261 (Proposed Standard), June 2002. Updated by RFCs 3265, 3853, 4320, 4916, 5393.
- [57] C. Perkins. IP Encapsulation within IP. RFC 2003 (Proposed Standard), October 1996.
- [58] Y. Rekhter, T. Li, and S. Hares. A Border Gateway Protocol 4 (BGP-4). RFC 4271 (Draft Standard), January 2006.
- [59] JavaSim. Official web.
- [60] Caida website; <http://www.caida.org>.
- [61] A. Gunnar, M. Johansson, and T. Telkamp. Traffic matrix estimation on a global IP backbone - a comparison on real data. In *Proc. ACM SIGCOMM Internet Measurement Conference*, pages 149–160, October 2004.
- [62] A. Soule, A. Lakhina, N. Taft, K. Papagiannaki, K. Salamatian, A. Nucci, M. Crovella, and C. Diot. Traffic matrices: Balancing measurements, inference and modeling. In *Proc. ACM SIGMETRICS*, June 2005.
- [63] Y. Zhang, M. Roughan, N. Duffield, and A. Greenberg. Fast accurate computation of large-scale IP traffic matrices from link loads. In *Proc. ACM SIGMETRICS*, San Diego, California, USA, June 2003.



- [64] M. S. Lobo, L. Vandenbergh, S. Boyd, and H. Lebert. Applications of second-order cone programming. *Linear Algebra and its Applications*, 284:193–228, 1998.
- [65] A. Gunnar and M. Johansson. Robust load-balancing under statistical uncertainty: models and polynomial-time algorithms. In *NGI 2009*, Aveiro, Portugal, July 2009.
- [66] A. Neto, Figueiredo S., L. Maal, and R. L. Aguiar. Qos-rrc: Integrated qos routing and resource provisioning mechanism for future internet. In (*Proc.IEEE Symp. Computers and Communications ISCC 2010*), 2010.
- [67] A. Neto, E. Cerqueira, A. Rissato, E. Monteiro, and P. Mendes. A resource reservation protocol supporting qos-aware multicast trees for next generation networks. In *Proc. 12th IEEE Symp. Computers and Communications ISCC 2007*, pages 707–714, 2007.
- [68] R. Braden, L. Zhang, S. Berson, S. Herzog, and S. Jamin. Resource ReSerVation Protocol (RSVP) – Version 1 Functional Specification. RFC 2205 (Proposed Standard), September 1997. Updated by RFCs 2750, 3936, 4495.
- [69] 4WARD (editor: Filipe Cardoso). Physical Layer Awareness, June 2010. Deliverable D-5.5 (FP7-ICT-2007-1-216041-4WARD/D-5.5).
- [70] International Telecommunication Union. Recommendation ITU-T G.709/Y.1331. Interfaces for the Optical Transport Network (OTN), 12/2009. Available at <http://www.itu.int/rec/T-REC-G.709>.
- [71] International Telecommunication Union. Recommendation ITU-T G.805: Generic functional architecture of transport networks, 03/2000. Available at <http://www.itu.int/rec/T-REC-G.805>.
- [72] International Telecommunication Union. Recommendation ITU-T G.806: Characteristics of transport equipment, Description methodology and generic functionality, 2009. Available at <http://www.itu.int/rec/T-REC-G.806>.
- [73] Wes Doonan and John Moore and Jerry Sobieski and Rick Summerhill and Chris Tracy. GMPLS Tutorial and R&E Network Implementation. Albuquerque, NM, February 5 2006.
- [74] OMNeT++ Community Site. <http://www.omnetpp.org/>.
- [75] International Telecommunication Union. Recommendation ITU-T G.809: Functional architecture of connectionless layer networks, March 2003. Available at <http://www.itu.int/rec/T-REC-G.809>.
- [76] Freek Dijkstra. Framework for Path Finding in Multi-Layer Transport Networks. *PhD Thesis Universiteit van Amsterdam*, June 2009. Available at <http://www.macfreek.nl/work/Dijkstra-multilayer-pathfinding.pdf>.
- [77] Jeroen van der Ham and Freek Dijkstra and Paola Grosso and Ronald van der Pol and Andree Toonk and Cees de Laat. A Distributed Topology Information System for Optical Networks Based on the Semantic Web. *Elsevier Journal of Optical Switching and Networking*, Vol. 5, Issue 2-3, pp. 85-93, June 2008. Available at <http://staff.science.uva.nl/vdham/research/publications/0703-ApplicationsOfNDL.pdf>.
- [78] Ilia Baldine and Yufeng Xin and Max Ott and Ivan Seskar. Experimenting with ontologies for multi-layer network slicing. 2009. Available at <http://groups.geni.net/geni/attachment/wiki/ORCABEN/ilia>
- [79] Maxwell E. Monteiro and Anilton Salles Garcia and Pedro Paulo F. Barcelos and Giancarlo Guizzardi. Ontology based model for the ITU-T Recommendation G.805: Towards the selfmanagement of transport networks. *International Journal of Computer Science and Information Technology*,



- Volume 2, Number 2, April 2010. Available at <http://airccse.org/journal/jcsit/0202csit13.pdf>.
- [80] D. Papadimitriou. Generalized Multi-Protocol Label Switching (GMPLS) Signaling Extensions for G.709 Optical Transport Networks Control. RFC 4328 (Proposed Standard), January 2006.
 - [81] 4WARD (editors: Stephan Baucke, Carmelita Grg). Virtualisation Approach: Concept, Oct 2009. Deliverable D-3.1.1 (FP7-ICT-2007-1-216041-4WARD/D3.1.1).
 - [82] P. Srisuresh and P. Joseph. OSPF-xTE: Experimental Extension to OSPF for Traffic Engineering. RFC 4973 (Experimental), July 2007.
 - [83] G. Raina and D. Towsley and D. Wischik . Part II: Control theory for buffer sizing, 2005.
 - [84] G. Carofiglio and L. Muscariello. On the impact of TCP and per-flow scheduling on Internet performance. In *In Proc. of IEEE INFOCOM 2010*, 2010.
 - [85] T. Bonald and L. Muscariello. Shortest queue first: implicit service differentiation through per-flow scheduling. Demo at IEEE LCN 2009.
 - [86] J. Beran and R. Sherman and M.S. Taqqu and W. Willinger. Long-range dependence in variable-bit-rate video traffic, 1995.
 - [87] The HTTP Apache project . <http://httpd.apache.org/>.
 - [88] The VideoLAN Project. <http://www.videolan.org/>.
 - [89] Linux advanced routing and traffic control. <http://lartc.org>.
 - [90] Tcpdump/Libpcap . <http://www.tcpdump.org/>.
 - [91] C. Demichelis and P. Chimento. IP Packet Delay Variation Metric for IP Performance Metrics (IPPM). RFC 3393 (Proposed Standard), November 2002.
 - [92] A. Kortebi and L. Muscariello and S. Oueslati and J. Roberts. Evaluating the number of active flows in a scheduler realizing fair statistical bandwidth sharing. In *Proc. of ACM SIGMETRICS*, 2005.
 - [93] IEEE. Telecommunications and information exchange between systems - local and metropolitan area networks- specific requirements part 11: Wireless lan medium access control (mac) and physical layer (phy) specifications. *IEEE Standard for Information technology - Std 802.11*, Jun 2007.
 - [94] IEEE. Local and metropolitan area networks - part 16: Air interface for fixed broadband wireless access systems. *IEEE Standard for Information technology - Std 802.16*, Dec 2001.
 - [95] IEEE. Telecommunications and information exchange between systems - local and metropolitan networks-specific requirements - part 15.5: Mesh enhancements for ieee 802.15 wpans. *IEEE 802.15.5 draft candidate*, Jul 2006.
 - [96] 3GPP. Utra-utran long term evolution (lte) and 3gpp system architecture evolution. *3GPP-ETSI*, May 2008.
 - [97] L.S. Ferreira, M.D. Amorim, L. Iannone, L. Berlemann, and L.M. Correia. Opportunistic management of spontaneous and heterogeneous wireless mesh networks. *Wireless Communications Magazine*, 17(2):41–46, May 2010.
 - [98] P. Kyasanur, J. So, C. Chereddi, and N. Vaidya. Multichannel mesh network: challenges and protocols. *Wireless Communications Magazine*, 3(2):30–36, Apr 2006.
 - [99] Jangeun Jun, Pushkin Peddabachagari, and Mihail L. Sichitiu. Theoretical maximum throughput of ieee 802.11 and its applications. In *In Proceedings of the IEEE International Symposium on Network Computing and Applications*, pages 249–257, 2003.
 - [100] Aravind Iyer and Catherine Rosenberg. Understanding the key performance issues with mac protocols for multi-hop wireless networks: Research articles. *Wirel. Commun. Mob. Comput.*, 6(6):745–760, 2006.



- [101] M. Chan and R. Ramjee. Improving TCP/IP performance over third generation wireless networks. In *IEEE INFOCOM*, 2004.
- [102] D. Kliazovich and F. Granelli. A cross-layer scheme for tcp performance improvement in wireless lans. In *IEEE GLOBECOM / Global Telecommunications Conference*, volume 2, pages 840–844, November 2004.
- [103] Y.-C.Shim, N.-C. Kim, and H.-S Kang. Improving tcp performance over mobile ipv6. In *Computational Science and Its Applications ICCSA 2004*, volume 3043/2004 of *Lecture Notes in Computer Science*, pages 1105–1114, April 2004.
- [104] Wen-De Zhong, Xiaomei Niu, Bin Chen, Sanjay K. Bose, and Yaohui Jin. Performance comparison of overlay and peer models in ip/mps over optical networks. *Photonic Network Communications*, 9(1):121–131, January 2005.
- [105] F. Idzikowski, S. Orlowski, C. Raack, H. Woesner, and A. Wolisz. Saving energy in IP-over-WDM networks by switching off line cards in low-demand scenarios. In *ONDM'10: Proc. of the 14th Conference on Optical Network Design and Modeling*, February 2010. Accepted paper.
- [106] A. Doria, J. Hadi Salim, R. Haas, H. Khosravi, and W. Wang and. Forwarding and control element separation (forces) protocol specification. IETF RFC 5810, march 2010.
- [107] M. Sifalakis. *Adaptation and Awareness in Autonomic Systems*. PhD thesis, Lancaster University, Computing Dept., 2009.
- [108] T. Bray, J. Paoli, C.M. Sperberg-McQueen, E. Maler, F. Yergeau, and J. Cowan. *Extensible Markup Language (XML) 1.1*, September 2006. <http://www.w3.org/TR/xml11/>.
- [109] M. Sifalakis, A. Louca, L. Peluso, A. Mauthe, and T. Zseby. A functional composition framework for autonomic network architectures. In *2nd IEEE International Workshop on Autonomic Communications and Network Management (IEEE NOMS/ACNM '08)*, April 2008.
- [110] S. Schmid, T. Chart, M. Sifalakis, and A. C. Scott. A highly flexible service composition framework for real-life networks. *Computer Networks, Special Issue on Active Networks*, 50:2488–2505, October 2006.
- [111] ANA project homepage. <http://www.ana-project.org/>.
- [112] M.K. McKusick, K. Bostic, M.J. Karels, and J.S. Quarterman. *The Design and Implementation of the 4.4 BSD Operating System*. Addison Wesley, 2003.
- [113] An Autonomic Network Architecture, 2006. An EU-IST funded project in Situated and Autonomic Communications, <http://www.anaproject.org>.
- [114] 4WARD – Architecture and design for the future Internet, 2008. An EU-IST funded project in The network of the future (ICT-2007.1.1), <http://www.4ward.eu>.
- [115] NewArch: Future-Generation Internet Architecture, 2003. An US DARPA funded project in The network of the future (ICT-2007.1.1), <http://www.isi.edu/newarch/>.
- [116] Hagggle, 2006. An EU-IST funded project in Situated and Autonomic Communications, <http://www.hagggleproject.org>.
- [117] Autonomic internet, 2007. An EU-IST 7th Framework funded project in Networks of the Future, <http://ist-autoi.eu>.
- [118] D.J. Wetherall and D.L. Tennenhouse. The ACTIVE IP option. In *7th ACM SIGOPS European Workshop*, Connemara, Ireland, September 1996.
- [119] J.K. Ousterhout, J.Y. Levy, and B.B. Welch. The safe-tcl security model. In *Annual conference on USENIX Annual Technical Conference*, Lecture Notes in Computer Science, 1998.



- [120] D. Alexander, W. Arbaugh, M. Hicks, P. Kakkar, A. Keromytis, J. Moore, C. Gunder, S. Nettles, and J.M. Smith. The switchware active network architecture. In *IEEE Network*, May 1998.
- [121] B. Schwartz, A.W. Jackson, W.T. Strayer, W. Zhou, R.D. Rockwell, and C. Partbridge. Smart packets: Applying active networks to network management. *ACM Transactions on Computer Systems*, 18(1):67–88, 2000.
- [122] M. Fry and A. Ghosh. Application level active networking. *Computer Networks*, 31(7):655–667, 1999.
- [123] J.H. Yeh, R. Chow, and R. Newman. A dynamic interdomain communication path setup in active network. In *First International Working Conference on Active Networks (IWAN)*, volume 1653, Berlin, Germany, 1999.
- [124] Y. Yemini and S. da Silva. Towards programmable networks. In *IFIP/IEEE International Workshop on Distributed Systems: Operations and Management*, Aquila, Italy, October 1996.
- [125] I. Wakeman, A. Jeffrey, R. Graves, and T. Owen. Designing a programming language for active networks. *Hipparch special issue of Network and ISDN Systems*, June 1998.
- [126] P. Menage. RCANE: A resource controlled framework for active network services. In *First International Working Conference on Active Networks (IWAN)*, volume 1653, pages 25–36, Berlin, Germany, 1999.
- [127] R. Cardoe, J. Finney, A.C. Scott, and W.D. Shepherd. Lara: A prototype system for supporting high performance active networking. In *First International Working Conference on Active Networks (IWAN)*, volume 1653, pages 117–131, Berlin, Germany, 1999.
- [128] L. Ruf, R. Keller, and B. Plattner. A scalable high-performance router platform supporting dynamic service extensibility on network and host processors. In *2004 ACS/IEEE International Conference on Pervasive Services (ICPS'2004)*, Beirut, Lebanon, July 2004.
- [129] G. Hjalmtysson. The Pronto platform - A flexible toolkit for programming networks using a commodity operating system. In *3rd Workshop on Open Architectures and Network Programming (OpenArch)*, Israel, 2000.
- [130] P. Tullmann, M. Hibler, and J. Lepreau. Janos: A java-oriented os for active networks. *IEEE Journal on Selected Areas of Communication*, 19(3), March 2001.
- [131] A. Campbell, H.G. De Meer, M.E. Kounavis, K. Miki, J. Vicente, and D. Villela. The Genesis Kernel: A virtual network operating system for spawning network architectures. In *2nd Workshop on Open Architectures and Network Programming (OpenArch)*, New York, USA, March 1999.
- [132] D. Decasper, Z. Dittia, G. Parulkar, and B. Plattner. Router Plug-ins: A software architecture for next generation routers. In *ACM SIGCOMM*, pages 229–240, September 1998.
- [133] D. Feldmeier, A. McAuley, J.M. Smith, D. Bakin, W. Marcus, and T. Raleigh. Protocol boosters. *IEEE Journal on Selected Areas in Communications, Special Issue on Protocol Architectures for 21st Century Applications*, 16(3):437–444, April 1998.
- [134] M.W. Hicks, P. Kakkar, J.T. Moore, C.A. Gunter, and S. Nettles. PLAN: A packet language for active networks. In *3rd ACM SIGPLAN International Conference on Functional Programming*, pages 86–93, 1998.
- [135] S. Merugu, S. Bhattacharjee, Y. Chae, M. Sanders, K. Calvert, and E. Zegura. Bowman and CANEs: Implementation of an active network. In *37th annual Allerton Conference on Communication, Control and Computing*, Monticello, Illinois, USA, September 1999.
- [136] Z. Fang and J. McNair. Cross layer design for mobile ip handoff. In *Vehicular Technology Conference*, pages 2255–2259, June 2005.



- [137] X. Lin, N.B. Shroff, and R. Srikant. A tutorial on cross-layer optimization in wireless networks. *IEEE journal on selected areas in communications*, 24(8):1452–1463, August 2006.
- [138] V. Sharma and F. Hellstrand. Framework for Multi-Protocol Label Switching (MPLS)-based Recovery. RFC 3469 (Informational), February 2003. Updated by RFC 5462.
- [139] G. Xylomenos and G. C. Polyzos. Quality of service support over multi-service wireless internet links. *Computer Networks*, 37(5):601–615, 2001.
- [140] S. Khan, Y. Peg, E. Steinbach, M. Sgroi, and W. Kellerer. Application driven cross layer optimisation for video streaming over wireless networks. *IEEE Communications*, 44(1), 2006.
- [141] A Ksentini and M. Naimi. Towards an improvement of h.264 video transmission over IEEE 802.11e through a cross-layer architecture. *IEEE Communications*, January 2006.
- [142] Q. Liu, S. Zhou, and G. Giannakis. Cross layer scheduling with prescribed qos guarantees in adaptive wireless networks. *IEEE Journal on Selected Areas in Communication*, May 2005.
- [143] Q. Liu, S. Zhou, and G. Giannakis. Cross layer combining of adaptive modulation and coding with truncated arq over wireless links. *IEEE Transactions on Wireless Communications*, September 2004.
- [144] R. Krishnan, J.P.G. Sterbenz, M.W. Eddy, C. Partridge, and M. Allman. Explicit transport error notification (eten) for error-prone wireless and satellite networks. *Computer Networks*, 46(3):343–362, October 2004.
- [145] J. Wang, L. Li, S.H. Low, and J.C. Doyle. Cross-layer optimization in tcp/ip networks. *IEEE/ACM Transactions on Networking*, 13(3):582–568, June 2005.
- [146] P. Sudame and B. R. Badrinath. On providing support for protocol adaptation in mobile wireless networks. *Mobile Networks and Applications*, 6(1), 2001.
- [147] R. Winter, J.H. Schiller, N. Nikaein, and C. Bonnet. Crosstalk: cross-layer decision support based on global knowledge. *IEEE Communications Magazine*, 44(1):93–99, 2006.
- [148] Al-Omari S, J. Du, and W. Shi. Score: a sensor core framework for cross-layer design. In *QShine '06: 3rd international conference on Quality of service in heterogeneous wired/wireless networks*, New York, NY, USA, 2006. ACM.
- [149] K. Chen, S. H. Shan, and K. Nahrstedt. Cross-layer design for data accessibility in mobile ad hoc networks. *Wireless Personal Communications*, 21(1), 2002.
- [150] M. Conti, G. Maselli, G. Turi, and S. Giordano. Cross layering in mobile ad hoc network design. *IEEE Computer Society*, 27(2):48–51, 2004.
- [151] V. T. Raisinghani and S. Iyer. Eclair: An efficient cross layer architecture for wireless protocol stacks. In *WWC*, 2004.
- [152] Q. Wang and M. A. Abu-Rgheff. A multi-layer mobility management architecture using cross-layer signalling interactions. In *5th European Personal Mobile Communications Conference*, 2003.
- [153] D. G. Sachs, W. Yuan, C. J. Hughes, A. Harris, S. V. Adve, D. L. Jones, R. H. Kravets, and K. Nahrstedt. Grace: A hierarchical adaptation framework for saving energy. In *ACEED*, 2005.
- [154] S. McCanne and V. Jacobson. The BSD packet filter: A new architecture for user level packet capture. In *Usenix '93 Winter Conference*, pages 259–270, 1993.
- [155] D. R. Engler and M. F. Kaashoek. Dpf: Fast, flexible message demultiplexing using dynamic code generation. In *SIGCOMM '96 - annual conference on applications, technologies, architectures, and protocols for computer communications*, pages 53–59, 1996.
- [156] M. L. Bailey, B. Gopal, M. A. Pagels, L. L. Peterson, and P. Sarkar. Pathfinder: A pattern-based packet classifier. In *Operating Systems Design and Implementation*, pages 115–123, 1994.



- [157] G Chandranmenon and G. Varghese. Trading packet headers for packet processing. *ACM/IEEE Transactions in Networking*, 17(1):21–29, 1996.
- [158] Y. Rekhter, B. Davie, D. Katz, E. Rosen, and G. Swallow. Cisco Systems’ Tag Switching Architecture Overview. RFC 2105 (Informational), February 1997.
- [159] OMNeT++ version 4.0, User Manual.
<http://www.omnetpp.org/doc/omnetpp40/manual/usman.html/>.